

LIFE Beyond Individual Intelligence: Surveying Collaboration, Failure Attribution, and Self-Evolution in LLM-based Multi-Agent Systems

Shihao Qi^{†,1,2}, Jie Ma^{†,✉,1,3}, Rui Xing^{†,1,4}, Wei Guo^{†,1,5}, Xiao Huang^{†,1,4}
 Zhitao Gao^{2,6}, Jianhao Deng², Jun Liu^{1,2,6}, Lingling Zhang^{1,2,6}, Bifan Wei^{1,6}
 Boqian Yang², Pinghui Wang^{1,5}, Jianwen Sun⁷, Jing Tao^{1,3}
 Yaqiang Wu^{2,8}, Hui Liu⁸, Yu Yao⁹, Tongliang Liu⁹

¹ MOE KLINNS Lab

² School of Computer Science and Technology, Xi'an Jiaotong University

³ School of Cyber Science and Engineering, Xi'an Jiaotong University

⁴ School of Software Engineering, Xi'an Jiaotong University

⁵ School of Control Science and Engineering, Xi'an Jiaotong University

⁶ Shaanxi Provincial Key Laboratory of Big Data Knowledge Engineering

⁷ Laboratory for AI and New Forms of Education, Central China Normal University

⁸ Lenovo AI Technology Center, CTOO, Lenovo

⁹ Sydney AI Centre, The University of Sydney

[†] Equal contribution ✉ Corresponding author

Abstract

LLM-based autonomous agents have demonstrated strong capabilities in reasoning, planning, and tool use, yet remain limited when tasks require sustained coordination across roles, tools, and environments. Multi-agent systems address this limitation through structured collaboration among specialized agents, but tighter coordination also amplifies a less explored risk: errors can propagate across agents and interaction rounds, producing failures that are difficult to diagnose and, even once identified, rarely translate into structural self-improvement. Existing surveys have separately covered individual agent capabilities, multi-agent collaboration, or agent self-evolution, but treat these topics in isolation—leaving the causal dependencies among them largely unexamined. This survey provides a unified and comprehensive review organized around four causally linked stages, which we term the **LIFE** progression: **L**ay the capability foundation, **I**ntegrate agents through collaboration, **F**ind faults through attribution, and **E**volve through autonomous self-improvement. We review the capability foundations of individual agents, the organizational mechanisms of multi-agent collaboration, the methodological landscape of failure attribution, and the hierarchical design space of self-evolution. Throughout, we formally characterize the dependencies between adjacent stages, revealing how each stage both depends on and constrains the next. Beyond synthesizing existing work, we identify open challenges at the boundaries between LIFE stages and propose a cross-stage research agenda for closed-loop multi-agent systems capable of continuously diagnosing failures, reorganizing collaborative structures, and refining agent behaviors, thereby extending current human-engineered coordination frameworks toward more self-organizing and resilient forms of collective intelligence. By bridging these previously fragmented research threads into a coherent progression, this survey aims to offer both a systematic reference for current research and a conceptual roadmap toward autonomous, self-improving multi-agent intelligence.

Keywords: large language model-based agents, multi-agent systems, multi-agent collaboration, failure attribution, self-evolution, survey.

GitHub:  <https://github.com/mira-ai-lab/awesome-mas-life>

Contents

1	Introduction	4
2	Individual Intelligence	6
2.1	From LLM to LLM-based Agent: An Architectural Overview	6
2.2	Reasoning	9
2.2.1	Input-Stage Enhancement	9
2.2.2	Reasoning-Process Enhancement	10
2.2.3	Output-Stage Regulation	10
2.3	Memory	11
2.3.1	Memory Formation	13
2.3.2	Memory Maintenance	14
2.3.3	Memory Retrieval and Utilization	14
2.4	Planning	15
2.4.1	Decomposition-Based Planning	15
2.4.2	Search-Based Planning	17
2.5	Tool Use	17
2.5.1	Tool Capability Acquisition	18
2.5.2	Tool Invocation	19
2.5.3	Tool Generalization	20
2.6	Evaluation	21
2.7	Discussion	22
3	Multi-Agent Collaboration	22
3.1	Role	25
3.1.1	Role Capability	25
3.1.2	Role Allocation	27
3.2	Communication	29
3.2.1	Communication Modes	29
3.2.2	Communication Protocols	30
3.3	Orchestration	31
3.3.1	Centralized Orchestration Topology	32
3.3.2	Distributed Orchestration Topology	32
3.3.3	Hybrid Orchestration Topology	33
3.4	Interaction	33
3.4.1	Information Flow	34
3.4.2	Interaction Patterns	35
3.5	Evaluation	36
3.6	Discussion	37
4	Multi-Agent System Failure Attribution	39
4.1	From Collaboration to Diagnosis	39
4.1.1	Formal Expression of the Attribution Process	40
4.2	Failure Taxonomy	41
4.2.1	System Structure	41

4.2.2	Execution Stages	41
4.2.3	Causal Lifecycle	41
4.3	Failure Attribution Methods	42
4.3.1	Data-driven Method	42
4.3.2	Constraint-Guided Method	44
4.3.3	Causal-inference Method	45
4.4	Failure Attribution Evaluation	47
4.5	Discussion	49
4.5.1	Core Challenges	49
4.5.2	Future Research Direction	51
5	Multi-Agent System Self-Evolution	51
5.1	From Attribution to Evolution	51
5.2	Formal Definition of MAS Self-Evolution	52
5.3	MAS Self-Evolution Taxonomy	54
5.3.1	Agentic Self-Evolution	54
5.3.2	Systemic Self-Evolution	56
5.3.3	Meta Self-Evolution	58
5.4	Analyzing Evolutionary Dynamics	59
5.5	Evaluation	62
5.6	Discussion	62
6	Conclusion	64

1 Introduction

Large language models (LLMs) have rapidly evolved from fluent text generators into systems capable of complex reasoning, long-horizon planning, and interaction with external environments [1, 2, 3, 4]. This progression has been accompanied by the emergence of chain-of-thought capabilities [5, 6], advances in scalable instruction tuning [7], and reinforcement learning–driven reasoning [8], which together have substantially expanded the ability of LLMs to tackle mathematical problem-solving, competitive programming, and professional examinations [8, 9]. Despite these advances, individual LLMs still struggle to maintain coherent behavior over extended interactions, adapt to rapidly changing task conditions, and coordinate multi-step actions that span diverse tools and environments [10, 11]. These gaps become especially pronounced in real-world deployments that demand sustained autonomy, including autonomous software engineering [12, 13], AI-driven scientific discovery [14, 15], and embodied robotic control [16, 17], where success hinges not on a single inference pass but on persistent observation, iterative decision-making, and closed-loop adaptation. This recognition has catalyzed the emergence of LLM-based autonomous agents: systems that augment foundation models with explicit modules for perception, memory, planning, and tool use, operating through continuous cycles of reasoning, action, and reflection [10, 11, 18, 19].

While these capabilities have significantly raised the ceiling of what a single agent can accomplish, a fundamental tension emerges as tasks grow in scope and duration: the diverse competencies they demand become increasingly difficult to concentrate within a single agent without encountering conflicts in prompt design, context allocation, and behavioral objectives [20]. Multi-agent collaboration offers a principled resolution by decomposing complex tasks into specialized roles that operate through structured communication and coordinated orchestration [21, 22]. Recent systems have moved well beyond early explorations: role-specialized agent teams now collaboratively solve tasks that require sustained multi-step coordination across planning, execution, and verification [23, 24], while standardized communication protocols such as MCP and A2A are beginning to establish the infrastructure for open, interoperable agent ecosystems [25, 26]. These developments signal a shift from building individual capable agents to engineering the organizational structures that govern how agents coordinate at scale. Yet this very capacity for tight coordination introduces a less examined consequence: as agents become more deeply interdependent, the system’s collective behavior grows increasingly difficult to predict, diagnose, and improve when failures occur.

In tightly coupled multi-agent systems, a localized error such as a hallucinated fact, a misrouted message, or an incorrect tool invocation can propagate through successive interaction rounds, triggering cascading failures that obscure the original root cause [20, 27]. As execution trajectories grow longer and inter-agent dependencies deepen, the causal chain linking a failure to its downstream consequences becomes increasingly opaque, rendering manual identification of the responsible agent and the decisive step both inefficient and unscalable [28, 29, 30]. Meanwhile, even when root causes are successfully identified, existing multi-agent systems largely lack the ability to translate diagnostic insights into structural adaptation, whether by reorganizing coordination topologies, revising role assignments, or refining collaboration policies in light of observed failure patterns [31, 32]. These two deficits in attribution and in self-correction are deeply intertwined: without reliable diagnosis, improvement efforts lack direction; without the ability to act on diagnostic results, attribution yields limited practical value [33]. This coupling exposes a fundamental asymmetry: the capacity to build increasingly sophisticated multi-agent systems has far outpaced the machinery available for understanding why they fail and enabling them to improve autonomously. Bridging this gap requires a closed-loop framework

in which failure diagnosis directly informs structural self-improvement.

The growing significance of LLM-based agent systems has prompted a substantial body of survey work, yet existing reviews remain compartmentalized along the boundaries identified above. One line of surveys systematically examines the core capabilities of individual agents, covering reasoning, memory, planning, and tool use as modular components of a single decision-making system [10, 11, 34, 35, 36]. A second line shifts focus to multi-agent collaboration, analyzing how role specialization, communication protocols, orchestration topologies, and interaction patterns enable collective problem-solving [22, 37, 38, 39, 40, 41]. A third, more recent line addresses agent self-evolution, investigating how agents can autonomously refine their prompts, memory, tools, and workflows through environmental feedback [31, 42]. These lines collectively cover considerable ground, but they leave the post-deployment failure loop largely unexamined. Notably, no existing survey provides a systematic treatment of multi-agent failure attribution, despite its emergence as a distinct and rapidly growing research area [20, 27, 33]. More fundamentally, no prior work has examined the complete operational lifecycle of multi-agent systems as an integrated whole: from foundational single-agent capabilities, through collaborative organization, to failure diagnosis and autonomous structural improvement. This survey is motivated by precisely this gap.

To the best of our knowledge, this is the first survey that covers the complete operational lifecycle of LLM-based multi-agent systems, from foundational single-agent capabilities, through collaborative organization, to failure diagnosis and autonomous structural improvement. Figure 2 provides a visual overview of the operational lifecycle and the representative work reviewed at each stage. We organize this lifecycle as a causally linked progression termed **LIFE**, and summarize our main contributions below.

Contributions

- **LIFE-oriented perspective.** We organize LLM-based multi-agent systems through the LIFE progression, a unified analytical lens that connects individual intelligence, collaboration, failure attribution, and self-evolution as four causally linked stages, moving beyond prior surveys that largely treat these topics separately.
- **Stage-wise structured synthesis.** Within each stage, we provide systematic taxonomies and comparative analyses of the field, covering core capabilities for individual intelligence, organizational mechanisms for collaboration, methodological families for failure attribution, and hierarchical scopes for self-evolution.
- **Attribution-driven evolution loop.** Enabled by the LIFE perspective, we highlight an attribution–evolution closed loop: failure attribution not only explains what went wrong, but also narrows the search space for targeted self-improvement. In turn, collaborative structures shape what failures can be observed and attributed, a coupling that remains underexplored in prior survey literature.
- **Challenges, agenda, and resources.** We identify open challenges that arise at stage boundaries, outline a cross-stage research agenda, and curate a publicly maintained repository of structured literature, taxonomy visualizations, and comparative tables to support future research and community use.

2 Individual Intelligence

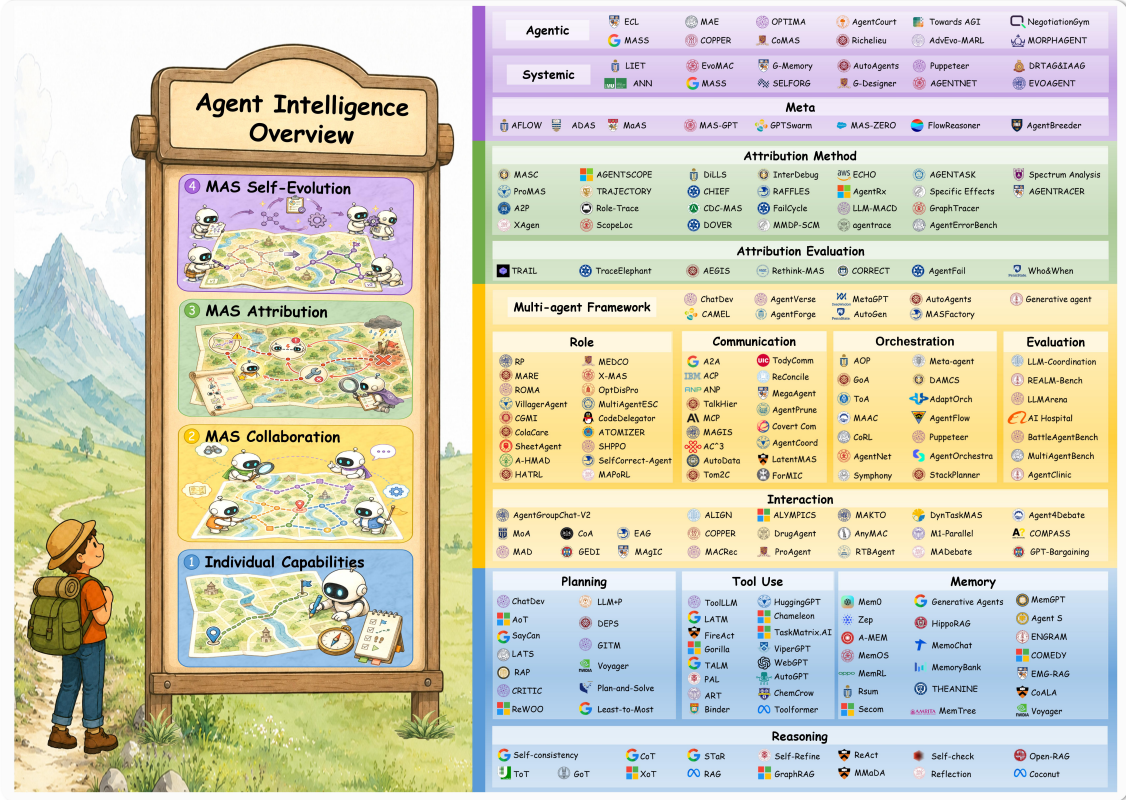


Figure 1: Overview of LLM-based multi-agent systems organized along an operational lifecycle. The right side summarizes representative topics and systems in each stage, and the left-side axis visualizes the cross-stage progression from action to evolution.

2.1 From LLM to LLM-based Agent: An Architectural Overview

While large language models have demonstrated remarkable capabilities in language understanding and generation [43, 44], their limitations become evident in complex real-world scenarios such as automated software development [12], scientific experiment design [15], and multi-step web navigation [45]. In these settings, LLMs operate as stateless response generators: they lack persistent memory, cannot actively perceive or act upon external environments, and remain confined to the static knowledge encoded during training. These constraints motivate the transition from reactive text generators to LLM-based autonomous agents [10], which extend the foundation model with the ability to perceive environmental states, formulate goals, execute actions through external tools, and adapt behavior based on feedback—operating through an iterative loop of observation, reasoning, action, and reflection [18, 19]. To systematically characterize the capabilities that enable such autonomy, we organize the core competencies of an individual agent into four dimensions: *reasoning*, *memory*, *planning*, and *tool use* [10, 46]. Below we formalize their interactions within a unified decision-making framework.

We model an LLM-based agent as a structured sequential decision-making system with explicit internal functional decomposition. The agent a is defined by the tuple:

$$a = (\mathcal{X}, \mathcal{U}, \mathcal{O}, \mathcal{M}, R, \mathcal{P}, T), \quad (1)$$

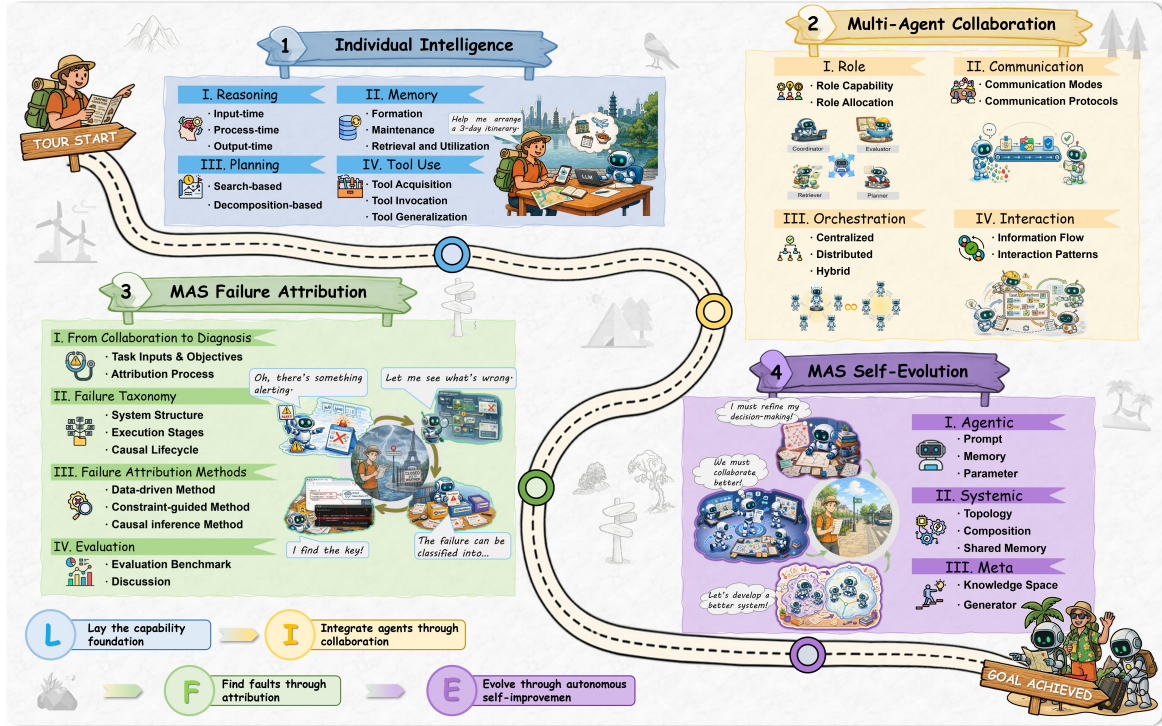


Figure 2: Illustrative overview of the operational lifecycle for LLM-based multi-agent systems, using a collaborative travel-planning scenario as a running example. The winding pathway connects four causally linked stages from START to DESTINATION: ❶ Individual Intelligence, ❷ Multi-Agent Collaboration, ❸ Multi-Agent Attribution, and ❹ Multi-Agent System Self-Evolution. Each stage is accompanied by a taxonomy panel summarizing the key dimensions and sub-topics reviewed in this survey.

where $\mathcal{X}$ denotes the observation space, $\mathcal{U}$ the action space, $\mathcal{O}$ the space of tool-returned observations, and $\mathcal{M}$ the memory state space. The functions R , $\mathcal{P}$, and $\mathcal{T}$ correspond to reasoning, planning, and tool execution, respectively. At each discrete time step $t = 1, 2, \dots$, the agent receives an observation $x_t \in \mathcal{X}$ and maintains an internal memory state $m_t \in \mathcal{M}$. The decision-making process is factorized into the following stages.

First, the agent retrieves task-relevant context from its memory state:

$$c_t = \phi(x_t, m_t). \quad (2)$$

Second, a reasoning module produces an intermediate representation based on the current observation and retrieved context:

$$r_t = R(x_t, c_t). \quad (3)$$

Third, a planning module maps a high-level goal g and the inferred context into a (possibly multi-step) action plan:

$$u_{t:t+k} = \mathcal{P}(g, r_t, c_t). \quad (4)$$

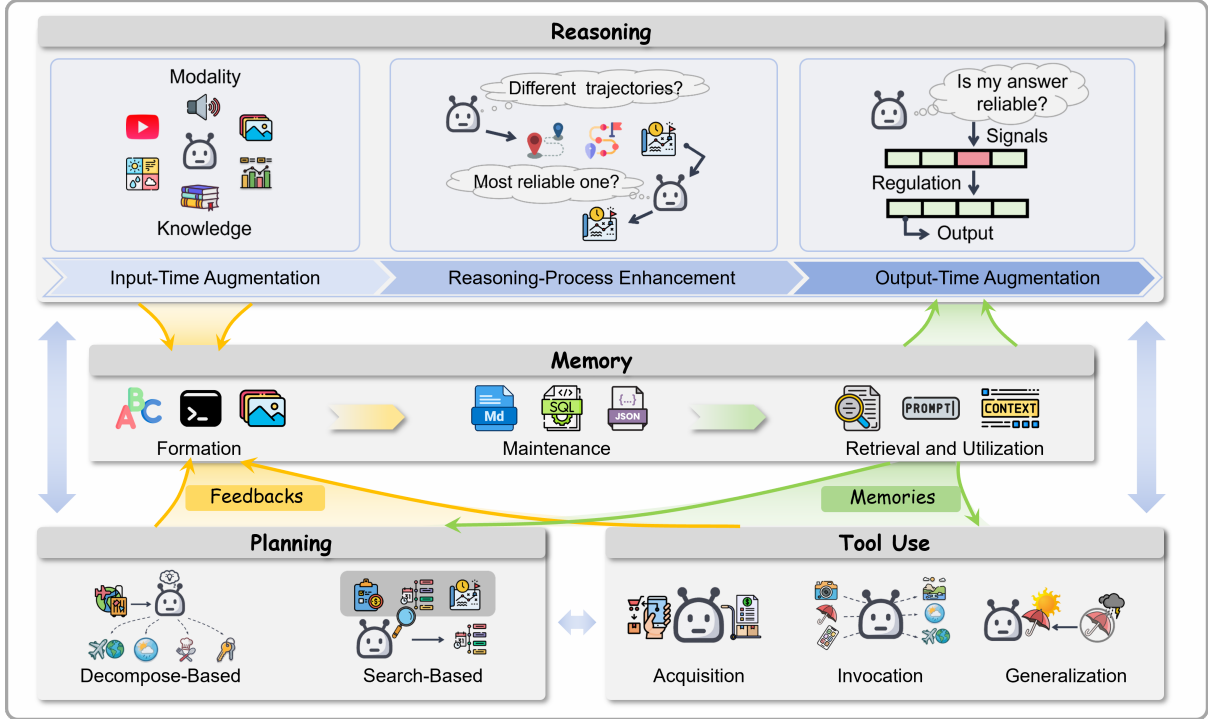


Figure 3: Architecture of individual agent capabilities organized along the reasoning pipeline. Memory serves as a cross-cutting module throughout this process, providing retrieved context and incorporating execution feedback. Planning and tool use are further elaborated as core components of the processing stage.

An executable action is then selected from the plan via an individual action policy that may incorporate stochasticity:

$$u_t \sim \pi_{\text{act}}(\cdot \mid u_{t:t+k}, r_t, c_t). \quad (5)$$

The selected action is executed through external tools, yielding an observation:

$$o_t \sim \mathcal{T}(\cdot \mid u_t). \quad (6)$$

Finally, let $h_t = (x_t, u_t, o_t)$ denote the interaction tuple. The memory state is updated by incorporating the new experience:

$$m_{t+1} = \psi(m_t, h_t). \quad (7)$$

Overall, the agent defines a closed-loop system in which the four capabilities interact dynamically rather than operating in isolation: reasoning guides tool selection, planning coordinates multi-step tool invocations, memory preserves intermediate results and learned strategies to inform future decisions, and tool execution grounds the agent’s actions in external environments. Figure 3 illustrates this architecture. The resulting individual action policy π_{act} is no longer a monolithic input-output mapping, but is factorized through the composition of functionally specialized modules, enabling interpretable and structured decision-making.

2.2 Reasoning

Reasoning constitutes the cognitive foundation of LLM-based agents, underpinning their capacity for decision-making, problem-solving, and coherent inference across complex task scenarios [10, 46]. Chain-of-Thought (CoT) prompting [5] elicits intermediate reasoning steps through carefully designed prompts and substantially improves performance on complex tasks without modifying model parameters. Building on this insight, a rich body of subsequent research has explored diverse strategies to further strengthen the reasoning ability of LLM-based agents [47, 48]. In the context of agent systems, the intrinsic reasoning capability embedded within the foundation model is generally treated as a given prior; the central challenge thus shifts to how external mechanisms can bridge the gap between a model’s static parametric knowledge and the dynamic demands of real-world tasks. Following this perspective, we organize existing methods along the information flow of the reasoning pipeline, categorizing them into three stages: (1) *input-stage enhancement*, which expands the informational boundary accessible to the model through external knowledge retrieval and multimodal perception; (2) *reasoning-process enhancement*, which optimizes the reasoning trajectory via search space expansion and path verification; and (3) *output-stage regulation*, which improves the reliability and factual accuracy of generated results through post-hoc evaluation and adaptive correction.

2.2.1 Input-Stage Enhancement

Input-stage enhancement expands the information available to the model before reasoning begins, shifting from sole reliance on parametric knowledge toward dynamic integration of external information. This category encompasses two main directions: knowledge augmentation and modality augmentation.

Knowledge Augmentation Retrieval-Augmented Generation (RAG) [49] couples a parametric generator with a non-parametric retrieval module, enabling the model to fetch relevant documents from external corpora at inference time. Self-RAG [50] improves upon this by training the model to adaptively decide when to retrieve and to critique both retrieved passages and its own generations through reflection tokens. Beyond the retrieve-then-read paradigm, several alternative strategies have emerged. On the structured knowledge side, SubgraphRAG [51] extracts query-relevant subgraphs from large-scale knowledge graphs, and Think-on-Graph [52] enables LLMs to interactively explore knowledge graphs through iterative beam search over entity–relation paths, tightly coupling retrieval with multi-hop reasoning. On the knowledge generation side, Knowledge Card [53] trains domain-specialized small models as modular knowledge repositories that generate background documents on demand, augmenting closed-source LLMs without relying on retrieval infrastructure. RuAG [54] takes a compression-oriented approach, distilling external data into compact first-order logic rules via LLM-aided Monte Carlo Tree Search and injecting them directly into prompts.

Modality Augmentation Many real-world reasoning tasks involve non-textual evidence that cannot be adequately captured in words alone. Multimodal-CoT [55] first demonstrates that fusing visual features into chain-of-thought reasoning yields substantial gains on science QA, even with sub-billion-parameter models. However, it treats visual input as a static feature. Visual Sketchpad [56] lifts this restriction by equipping models with a drawing canvas for creating intermediate visual artifacts during reasoning, and CogCoM [57] internalizes step-by-step visual manipulations, such as region grounding and zooming, while producing interpretable reasoning traces.

2.2.2 Reasoning-Process Enhancement

Input-stage methods enrich the information available before inference begins; the techniques discussed below instead intervene on the reasoning trajectory itself. We organize them along two dimensions: *search-space expansion*, which increases the coverage of candidate solutions, and *path verification*, which provides the selection pressure needed to identify the most reliable outcome.

Search Space Expansion CoT prompting and its zero-shot variant [5, 58] activate step-by-step decomposition through appropriate prompts. Tree of Thoughts [59] and Graph of Thoughts [60] generalize the search topology from a single chain to tree- and graph-structured spaces supporting branching, backtracking, and thought merging. These methods, however, rely on hand-designed prompting protocols, leaving open how to scale reasoning more systematically.

Recent work addresses this from two angles. On the learning side, DeepSeek-R1 [61] trains a model via large-scale reinforcement learning to produce extended chains of thought autonomously, exhibiting emergent self-verification and spontaneous backtracking that arise purely from reward optimization. CPO [62] takes a distillation perspective, collecting step-level preference pairs from tree search and fine-tuning via DPO so that ordinary decoding reproduces the quality of deliberate search at lower cost. On the resource-allocation side, Snell et al. [63] show that routing easy prompts to lightweight revision and hard prompts to parallel sampling or PRM-guided search enables a smaller model to match a substantially larger one under comparable compute budgets.

Reasoning Path Verification Expanding the search space produces richer candidate solutions, yet without effective selection the additional diversity may introduce as many erroneous paths as correct ones. Self-Consistency [64] addresses this at the output level by sampling multiple chains and returning the majority answer, while Self-Refine [65] and Reflexion [19] implement iterative critique loops, with Reflexion further maintaining an episodic memory of past failures. These approaches improve reliability without external supervision but evaluate entire outputs rather than pinpointing which step first goes wrong. Process reward models (PRMs) address this limitation by scoring each reasoning step individually. Lightman et al. [66] train a PRM on human-annotated step-level labels and show that step-wise supervision substantially outperforms outcome-level reward models. Math-Shepherd [67] eliminates the annotation bottleneck by estimating step correctness through Monte Carlo rollouts, matching human-supervised PRMs while scaling to larger training sets. WizardMath [68] further integrates PRMs into both PPO training and inference-time verification, confirming that process supervision is most effective when it shapes both the learned policy and the selection mechanism.

2.2.3 Output-Stage Regulation

The strategies above improve reasoning quality from the perspectives of information supply and search mechanisms. However, model outputs may still contain factual errors—commonly termed hallucination [69]. Output-stage regulation addresses this risk along two complementary dimensions: *output reliability assessment*, which evaluates the factual consistency and trustworthiness of generated content, and *response behavior regulation*, which dynamically adjusts the model’s generation strategy based on reliability signals to keep error risk within acceptable bounds.

Output Reliability Assessment Reliability assessment aims to evaluate the factual trustworthiness of generated content. Evidence-based methods verify claims against external knowledge: FActScore [70] decomposes long-form text into atomic claims and checks each against a knowledge base, while FacTool [71] and Factcheck-GPT [72] extend this pipeline to code generation, mathematical reasoning, and

scientific literature with finer-grained error localization. An alternative line of work detects hallucination without external knowledge. SelfCheckGPT [73] detects hallucination by measuring agreement across multiple independent samples without any external reference, and Semantic Entropy [74] refines this by clustering semantically equivalent outputs before estimating uncertainty, capturing the model’s true confidence at the meaning level rather than the surface form.

Deeper approaches extract reliability signals directly from model internals. HaloScope [75] uses membership estimation scores from hidden states to distinguish factual from hallucinated generations in an unsupervised manner, while HalluLens [76] provides a systematic taxonomy distinguishing extrinsic from intrinsic hallucinations. Chain-of-Verification (CoVe) [77] takes an active approach: after generating an initial answer, the model automatically produces verification questions, independently answers them, and revises inconsistencies; RARR [78] follows a similar verify-then-revise pattern using retrieved external evidence.

Response Behavior Regulation Reliability assessment provides error-identification signals; response behavior regulation acts on these signals to constrain the model’s generation. Existing mechanisms divide into inference-time decoding adjustments and training-time alignment.

At the decoding level, contrastive decoding suppresses hallucination by amplifying differences between distinct signal sources. DoLa [79] contrasts logit distributions from higher versus lower Transformer layers to surface factual knowledge encoded in deeper representations. CAD [80] contrasts output distributions with and without context to strengthen reliance on retrieved evidence, and ITI [81] directly shifts attention-head activations along pre-identified truthfulness directions during inference. More recent extensions include SADI [82], which introduces semantically aligned dynamic intervention, and MCD [83], which extends contrastive decoding to a three-model architecture for tighter hallucination suppression.

At the training level, FactTune [84] constructs factuality preference pairs from automated evaluation scores and fine-tunes via DPO, requiring no human annotation. For queries exceeding the model’s knowledge boundaries, R-Tuning [85] explicitly teaches the model to output refusal signals on out-of-scope samples, and calibrated RL approaches [86] incentivize stochastic abstention when confidence is insufficient. CRITIC [87] takes a tool-interactive approach, having the model verify its own outputs against external tools such as calculators and search engines before committing to a final answer. SEAL [88] trains models to embed calibrated confidence directly into their natural-language responses, enabling downstream systems to assess reliability without probing internal states.

2.3 Memory

Reasoning, as formalized above, is inherently episodic. However, complex real-world tasks often span multiple episodes, requiring the agent to leverage lessons from prior interactions to improve future performance. To address this need, recent research has introduced explicit memory mechanisms that enable agents to record, consolidate, and recall their own task-execution experiences [108, 109]. In the agent’s cognitive flow, reasoning generates the raw material of experience, memory consolidates and organizes that material, and the consolidated memory in turn informs the planning and tool-use strategies discussed in subsequent sections. We specifically focus on *agent-level experiential memory*, rather than model-level parametric knowledge or static external knowledge bases already discussed under retrieval-augmented generation (Section 2.2.1). Under this scoping, memory serves as the substrate for a closed loop: interaction produces experience, experience is consolidated into memory, and memory is retrieved to inform better actions in subsequent interactions. This loop not only strengthens the

Table 1: Comparison of reasoning enhancement methods (Section 2.2), organized by intervention stage and sub-type.

Method	Sub-type	Task Scope	Prim. Source	Correction	Ext. Ret.	Tool	TF	TTC	Year
<i>A. Input-Stage Enhancement</i>									
ViperGPT [89]	PE	Mu	Inter	–	–	✓	✓	–	2023
RAPTOR [90]	KE	QA LF	Evid	–	✓	–	✓	–	2024
Self-RAG [50]	KE	QA LF	Evid	SF	✓	–	–	–	2024
LLaVA [91]	PE	Mu	Inter	–	–	✓	–	–	2024
TopViewRS [92]	PE	Mu	Inter	–	–	–	✓	–	2024
VisCoT [93]	PE	Mu	Inter	–	–	–	✓	–	2024
HippoRAG2 [94]	KE	QA LF	Evid	–	✓	–	✓	–	2025
Theanine [95]	KE	QA LF	Evid	–	✓	–	✓	–	2025
SubgraphRAG [51]	KE	QA	Evid	–	✓	–	✓	–	2025
<i>B. Reasoning-Process Enhancement</i>									
Self-Consistency [64]	VRP	Ma QA	Consis	SC	–	–	✓	✓	2023
LATS [96]	ESS	Mu	Inter	Refl	–	✓	✓	✓	2024
CoC [97]	ESS	Ma QA	Tool	–	–	✓	✓	–	2024
AnyTool [98]	ESS	Co	Tool	SF	–	✓	✓	–	2024
RAIN [99]	ESS	Ag	Tool	SF	–	–	✓	✓	2024
Math-Shepherd [67]	VRP	QA	Inter	SF	–	–	✓	✓	2024
CPO [62]	VRP	Ma	Log	–	–	–	–	✓	2024
rStar [100]	VRP	Ma	Log	–	–	–	–	–	2024
rStar-Math [101]	ESS	Ma	Consis	SC	–	–	✓	✓	2025
TTS [102]	ESS	Ma	Consis	SC	–	–	–	✓	2025
TOPS [103]	ESS	Ma	Log	–	–	–	✓	✓	2025
Rewarding Progress [104]	ESS	Ma	Log	–	–	–	✓	✓	2025
WizardMath [68]	VRP	Ma	Log	–	–	–	–	✓	2025
<i>C. Output-Stage Regulation</i>									
FActScore [70]	OV	LF	Evid	–	✓	–	✓	–	2023
FacTool [71]	OV	Mu	Evid	–	✓	✓	✓	–	2023
SelfCheckGPT [73]	OV	LF	Consis	–	–	–	✓	–	2023
Semantic Entropy [105]	OV	QA LF	Inter	–	–	–	✓	–	2023
RARR [78]	OV	QA LF	Evid	TV	✓	–	✓	–	2023
ITI [81]	RC	QA	Inter	–	–	–	✓	–	2023
Reflexion [19]	RC	Ag Co	Inter	Refl	–	–	✓	✓	2023
L2R [106]	RC	QA	Inter	–	–	–	–	–	2023
Factcheck-GPT [72]	OV	QA LF	Evid	–	✓	–	✓	–	2024
HaloScope [75]	OV	QA LF	Inter	–	–	–	✓	–	2024
DoLa [79]	RC	QA LF	Inter	–	–	–	✓	–	2024
CAD [80]	RC	QA LF	Evid	–	–	–	✓	–	2024
FactTune [84]	RC	QA LF	Evid	–	–	–	–	–	2024
CoVe [77]	RC	QA LF	Log	SF	–	–	✓	✓	2024
CRITIC [87]	RC	QA Ma	Tool	TV	–	✓	✓	–	2024
R-Tuning [85]	RC	Co	Tool	TV	–	✓	✓	–	2024
R-Tuning [85]	RC	QA	Inter	–	–	–	–	–	2024
HalluLens [76]	OV	Mu	Inter	–	–	–	✓	–	2025
SADI [82]	RC	QA	Inter	–	–	–	✓	–	2025
MCD [83]	RC	QA LF	Inter	–	–	–	✓	–	2025
SEAL [88]	RC	QA LF	Inter	–	–	–	–	–	2025
SMC steering [107]	RC	QA Co	Log	–	–	–	✓	–	2025

Sub-type: KE Knowledge Enhancement; PE Perceptual Enhancement; ESS Expanding Search Space; VRP Verification of Reasoning Paths; OV Output Verification; RC Response Correction. *Task Scope:* evaluation domains — Ma Math; QA Question Answering; Co Code; LF Long-form Generation; Ag Agent; Mu Multi-task. *Prim. Source:* primary information source — Evid = external evidence; Inter = internal states; Consis = cross-sample consistency; Log = logic/verification; Tool = tool-based. *Correction:* error-correction mechanism — SF = self-feedback; Refl = reflection; TV = tool-verify; SC = self-consistency. *Ext. Ret.:* uses external retrieval. *TF:* training-free. *TTC:* supports test-time compute scaling.

individual agent’s reasoning and planning but also underpins the multi-agent coordination, attribution, and evolution discussed in later sections.

This closed-loop perspective naturally suggests three research questions that correspond to the three stages of the memory lifecycle: how raw interaction traces are transformed into storable entries (*memory formation*), how accumulated memories are structured, updated, and selectively forgotten over time (*memory maintenance*), and how relevant memories are recalled and converted into actionable context at decision time (*memory retrieval and utilization*).

2.3.1 Memory Formation

Memory formation concerns how an agent transforms its stream of interactions into discrete, storable memory entries. Existing approaches differ primarily in the degree of abstraction applied during this transformation, and can be grouped into three categories: direct recording, abstractive distillation, and typed routing.

Direct Recording Direct recording retains raw interaction traces with minimal processing. Generative Agents [110] introduces the *memory stream*, a timestamped log of all observations the agent perceives, serving as the foundational data layer for higher-level memory construction. MemGPT [111] frames this process through an operating-system metaphor, writing interaction segments to persistent archival storage via self-directed function calls.

Abstractive Distillation Abstractive distillation compresses raw experience into more compact representations, spanning a spectrum of granularity. At the coarsest level, the *reflection* mechanism of Generative Agents [110] periodically synthesizes recent observations into higher-level insights that are stored alongside raw entries. MemoChat [112] fine-tunes LLMs to compose structured memos through iterative memorization–retrieval–response cycles. Recursive summarization [113] maintains a continuously updated global summary through hierarchical compression of session-level memories. At a finer granularity, Memo [114] extracts individual facts from each exchange via a two-phase extraction-and-update pipeline. SeCom [115] demonstrates that the segmentation granularity itself critically affects quality, showing that topically coherent segments with compression-based denoising outperform both turn-level and session-level alternatives.

Typed Routing Typed routing goes beyond deciding how much to abstract and additionally determines where each entry should be stored. This is a natural extension of abstractive distillation: once experience is processed, routing it into semantically distinct stores enables more targeted downstream retrieval. ENGRAM [116] classifies each turn via an LLM-based router into episodic, semantic, or procedural stores with normalized schemas; ablation experiments confirm that collapsing types into a single store substantially degrades retrieval quality. CoALA [117] provides the theoretical grounding for such designs, formally defining working, episodic, semantic, and procedural memory as distinct components of a language agent’s cognitive architecture.

Notably, the type of memory formed at this stage directly determines what downstream capabilities can draw upon: episodic traces support reasoning-time self-correction (Section 2.2), semantic knowledge informs experience-driven planning (Section 2.4), and procedural entries enable cross-task skill reuse (Section 2.4).

2.3.2 Memory Maintenance

As interactions accumulate, the agent’s memory grows without bound, yet the capacity to attend to stored content at any given moment remains limited. Memory maintenance addresses this tension through two complementary aspects: the static structure that organizes stored entries, and the dynamic mechanisms that keep the store current and manageable over time.

Storage Structure The choice of storage structure determines the representational capacity and retrieval affordances of the memory system. The simplest approach is a flat, append-only list, as exemplified by the memory stream of Generative Agents [110]. Flat storage, however, provides no mechanism for relating or aggregating entries, limiting its scalability.

Graph-based structures introduce relational organization. Zep [118] builds a temporally-aware knowledge graph with three hierarchical sub-graphs (episode, semantic entity, and community layers) and a bitemporal data model that tracks both event time and ingestion time. HippoRAG [119] draws on hippocampal indexing theory, constructing a schemaless knowledge graph via open information extraction and retrieving through Personalized PageRank. Editable Memory Graph [120] supports dynamic memory modification through explicit graph editing operations for personalized agent behavior.

Hierarchical structures offer multi-granularity abstraction. MemTree [121] organizes memories as a dynamic tree with varying abstraction levels across depths, routing new information to the most semantically similar branch. A-MEM [122] adopts a Zettelkasten-inspired note-based architecture with LLM-generated metadata and dynamically constructed inter-note links that evolve as new knowledge accretes.

Dynamic Maintenance Beyond static organization, long-lived agents require mechanisms to update, compress, or discard stored memories as facts change and relevance decays. Existing approaches span a spectrum from passive decay to active policy learning.

Decay-based methods model forgetting as a continuous process. MemoryBank [123] applies the Ebbinghaus forgetting curve, exponentially decaying each entry’s strength since last access and removing entries below a threshold. Timeline-based memory management [95] organizes dialogue history along a temporal axis for selective retention based on narrative relevance.

LLM-driven policies make maintenance decisions through explicit reasoning. Memo [114] uses an LLM to choose among four atomic operations (add, update, delete, or no-op) for each incoming fact. A-MEM [122] implements memory evolution, retroactively refining existing notes when new experience arrives. COMEDY [124] progressively compresses older interaction segments while preserving key factual content. MemOS [125] elevates memory to a first-class operating-system resource with unified lifecycle management primitives across multiple memory types.

Learned policies optimize maintenance through training signals. MemRL [126] trains a Q-function that estimates the future utility of each stored experience, retaining high-utility entries and discarding low-utility ones. MemGPT [111] takes a resource-management approach, evicting the least relevant segments from main context to archival storage when the context window approaches its limit, analogous to page swapping in virtual memory systems.

2.3.3 Memory Retrieval and Utilization

Stored memories become useful only when recalled at the right moment and in the right form. Memory retrieval and utilization closes the experiential loop by converting accumulated state back into actionable context.

Retrieval Mechanisms The most widely adopted baseline is dense vector retrieval, where memory entries and the current query are embedded and ranked by cosine similarity [111, 114, 123]. This approach treats all entries as interchangeable and ignores temporal or importance signals.

Multi-factor scoring addresses this limitation by combining orthogonal dimensions. Generative Agents [110] weight recency, LLM-assigned importance, and embedding relevance into a composite score, a design that remains influential across subsequent systems. Type-aware retrieval further exploits the structure introduced during formation: ENGRAM [116] retrieves top- k neighbors independently from each typed store before merging results, ensuring balanced representation across evidence modalities. Zep [118] combines semantic search, BM25 keyword matching, and graph traversal, routing queries through the most effective path for each type. Utility-driven retrieval moves beyond relevance to task-level value: MemRL [126] first filters candidates by similarity, then re-ranks by a learned Q-value estimating each memory’s contribution to task success.

Integration Patterns How retrieved memories enter the agent’s decision process varies substantially across systems, and can be characterized along a spectrum of integration depth.

Existing integration patterns can be characterized along a spectrum of increasing agent autonomy. *Context injection* serializes retrieved entries into the prompt for the LLM to consume via in-context learning [114, 123], requiring no architectural modification but offering the agent no control over what is injected or when. *Experience-conditioned reasoning* allows retrieved memories to actively shape the agent’s inference trajectory: Generative Agents [110] feed retrieved reflections into a planning module to generate proactive daily schedules, and Agent S [127] retrieves relevant past task completions to constrain its plan search space. *Skill execution* further elevates memory from informational input to direct action: Voyager [128] (Section 2.4) retrieves verified code-based skills and composes them to solve new tasks, bypassing deliberative planning entirely.

2.4 Planning

Planning refers to the ability of an agent to convert a high-level goal into a structured sequence of executable actions. Unlike single-step reasoning, planning must account for sub-task dependencies, environmental feedback, and potential execution failures over extended horizons [35, 129]. We organize existing methods along two paradigms: *decomposition-based planning*, which structures goals into sub-tasks prior to or during execution, and *search-based planning*, which explores alternative plans through systematic evaluation and backtracking.

2.4.1 Decomposition-Based Planning

Decomposition-based methods reduce a complex goal into manageable sub-tasks, differing primarily in *when* the decomposition occurs relative to execution.

Proactive Decomposition Proactive decomposition generates a complete plan before any action is taken. Least-to-Most prompting [130] introduced the idea of progressively breaking a problem into simpler sub-problems and solving them in sequence, while Plan-and-Solve prompting [131] replaces the generic “think step by step” instruction with an explicit planning phase followed by sequential execution. HuggingGPT [132] extends this idea to multi-modal task orchestration: given a user request, an LLM generates a task dependency graph whose nodes are dispatched to specialist models hosted on Hugging Face. LLMCompiler [133] draws an analogy to classical compilers, producing a directed acyclic graph (DAG) of function calls that are executed in parallel wherever dependencies permit.

When the action space is grounded in physical or simulated environments, proactive decomposition

Table 2: Comparison of memory enhancement methods (Section 2.3), organized along the memory lifecycle.

Method	Formation	Storage	Maintenance	Retrieval	Mem. Type	Task Domain	Year
Generative Agents [110]	Record, Distill	Flat	Append	Multi-factor	●●○	Simulation	2023
MemoChat [112]	Distill	Flat	Append	Similarity	●●○	Conversation	2023
Reflexion [19]	Distill	Flat	Append	Injection	●○○	General	2023
MemoryBank [123]	Record	Flat	Decay	Similarity	●●○	Conversation	2024
MemGPT [111]	Record	Tiered	Eviction	Autonomous	●○○	Conv, QA	2024
HippoRAG [119]	Distill	Graph	Incremental	Graph trav.	○○○	QA	2024
EMG [120]	Distill	Graph	Rewrite	Similarity	●●○	Conversation	2024
SeCom [115]	Distill	Flat	Append	Similarity	●●○	Conversation	2025
ENGRAM [116]	Routing	Typed	Append	Type-aware	●●●	Conversation	2025
Memo [114]	Distill	Graph	CRUD	Similarity	○○○	Conversation	2025
MemTree [121]	Distill	Tree	Compression	Tree trav.	○○○	QA	2025
A-MEM [122]	Distill	Graph	Evolution	Similarity	○○○	Conversation	2025
Zep [118]	Distill	Graph	Versioning	Hybrid	●○○	Conv, Ent	2025
RecurSum [113]	Distill	Flat	Compression	Similarity	○○○	Conversation	2025
COMEDY [124]	Distill	Flat	Compression	Similarity	●●○	Conversation	2025
MemOS [125]	Routing	Unified	Lifecycle	Routed	●●●	General	2025
MemRL [126]	Record	Flat	Learned	Utility-driven	●○○	General	2026

Formation: how raw experience becomes memory — Record = raw logging; Distill = abstractive summarization; Routing = typed classification. *Storage:* data structure holding memories — Flat flat store; Tiered main context + archival; Graph graph-structured; Tree tree-structured; Typed type-separated stores; Unified multi-type unified store. *Maintenance:* how memories are updated or discarded — Decay = time-based forgetting; Eviction = budget-constrained removal; CRUD = create/read/update/delete; Evolution = retroactive refinement; Versioning = bitemporal tracking; Lifecycle = unified management; Learned = RL-based selection. *Retrieval:* how relevant entries are recalled — Multi-factor = recency + importance + relevance; Injection = full buffer injection; Autonomous = self-directed via function calls; Graph/Tree trav. = traversal over corresponding structure; Type-aware = per-type top- k then merge; Hybrid = semantic + keyword + graph; Routed = multi-route retrieval; Utility-driven = similarity + Q-value re-ranking. *Mem. Type:* E (Episodic), S (Semantic), P (Procedural); filled = supported, hollow = absent. *Task Domain:* Conv = Conversation; Ent = Enterprise.

often takes the form of hierarchical task networks. LLM-Planner [134] generates few-shot grounded plans for embodied agents, while LLM+P [135] translates natural-language goals into PDDL specifications and delegates solving to a classical planner, combining the linguistic flexibility of LLMs with the optimality guarantees of symbolic planners. More recent work explores richer plan representations: Planning Anything [136] formulates planning problems as optimization programs that are solved by formal solvers, and Skeleton-of-Thought [137] first generates a plan skeleton and then fills in details in parallel to reduce latency. Plan-and-Act [138] specifically targets long-horizon web tasks by separating a high-level plan from low-level action execution, demonstrating that explicit proactive planning significantly outperforms reactive approaches on complex web benchmarks.

Progressive Decomposition Progressive decomposition interleaves planning with execution, refining or extending the plan based on intermediate observations. ReAct [18] established the foundational pattern by alternating reasoning traces with environment actions in a single prompt, enabling the agent to adjust its next step based on the most recent observation. The verbal self-reflection mechanism introduced by Reflexion [19] (discussed in Section 2.2) further augments this loop, allowing agents to generate natural-language critiques of failed episodes and consult them in subsequent attempts.

ADaPT [139] introduces recursive decomposition—when the executor fails on a sub-task, a planner module further decomposes it into finer-grained steps, continuing until each step is within the model’s execution capability. SelfGoal [140] dynamically constructs and refines a hierarchical goal tree (GoalTree) during interaction, selecting the most relevant sub-goals for the current state and decomposing them further when guidance is insufficiently specific. In open-world settings, Voyager [128] combines incremental planning with a growing skill library: the agent proposes sub-goals, writes executable code

to achieve them, and stores verified skills for future reuse, enabling continual capability expansion in Minecraft. CodeAct [141] reframes the action space itself as executable code, allowing the agent to express complex, multi-step plans as Python programs that interact with the environment through function calls, naturally supporting iteration and branching. DEPS [142] follows a Describe-Explain-Plan-Select loop in which the agent generates a plan, explains its rationale, and selects among alternatives based on environmental feedback. KnowAgent [143] injects domain-specific action knowledge into the planning loop, constraining the agent’s plan space to reduce hallucinated actions and improve plan feasibility.

2.4.2 Search-Based Planning

While decomposition-based methods commit to a single plan trajectory (possibly with replanning), search-based methods explicitly explore multiple candidate paths and select among them through evaluation, pruning, or backtracking. The tree search mechanisms introduced by Tree of Thoughts [144] and Language Agent Tree Search [96] (discussed in Section 2.2) provide the algorithmic foundation for this paradigm; here we focus on methods that apply and extend these ideas specifically to agent planning scenarios.

Step-Level Search Step-level search expands and evaluates candidates at each decision point. MCTS has proven particularly effective for LLM-based planning. LLM-MCTS [145] uses the LLM as a common-sense prior policy to guide MCTS rollouts in large-scale household task planning. PromptAgent [146] applies MCTS to search over the prompt space itself, treating prompt optimization as a planning problem with strategic exploration. RAP [147] formulates reasoning as planning with a world model: the LLM simultaneously serves as both the world model (predicting next states) and the reasoning agent, with MCTS balancing exploration and exploitation. More recently, REST-MCTS* [148] integrates process reward models to guide tree search, and AFlow [149] uses MCTS at a higher level of abstraction, searching over entire agentic workflows composed of reusable operators rather than individual actions. Beyond tree search, ToolChain* [150] formulates tool-use planning as an A* search problem over chains of API calls, and DiffuSearch [151] proposes implicit search via discrete diffusion modeling, bypassing explicit tree expansion by predicting future states to inform current action selection.

Plan-Level Search Rather than searching step by step, plan-level search generates and compares multiple complete or partial plans. This approach is natural when global coherence matters more than local optimality. The multi-plan selection paradigm identified by Huang et al. [152] represents an early systematization, where the LLM generates several candidate plans and a selection mechanism (voting, scoring, or verification) chooses the best one. PlanRAG [153] combines plan generation with retrieval-augmented decision making: the agent generates candidate plans, retrieves relevant evidence from a knowledge graph, and re-ranks plans based on retrieved information. Agent S [127] employs a hierarchical planning architecture with experience-augmented search, using a manager agent to generate high-level plans and a worker agent to execute individual steps, with an experience library that enables the agent to learn from past task completions.

2.5 Tool Use

Tool use extends an agent’s action space beyond text generation by enabling it to invoke external functions, APIs, code interpreters, web services, and other external systems [141]. Unlike reasoning or planning, which operate within the model’s internal representation, tool use requires the agent to cross the boundary between language and execution—generating structured calls, processing external returns, and integrating results back into the reasoning flow. We organize existing work along three

Table 3: Comparison of planning enhancement methods (Section 2.4), organized by planning paradigm.

Method	Strategy	Structure	Exec Mode	Replan	Task Domain	Year
<i>A. Decomposition-Based Planning</i>						
Least-to-Most [130]	Proactive	Seq	Seq	–	General	2023
Plan-and-Solve [131]	Proactive	Seq	Seq	–	General	2023
HuggingGPT [132]	Proactive	DAG	Hier	–	General	2023
LLM-Planner [134]	Proactive	Seq	Seq	✓	Robotics	2023
LLM+P [135]	Proactive	Seq	Seq	–	Robotics	2023
ReAct [18]	Progressive	Seq	Intlv	–	General	2023
DEPS [142]	Progressive	Seq	Intlv	✓	General	2023
LLMCompiler [133]	Proactive	DAG	Hier	✓	Tool	2024
Skeleton-of-Thought [137]	Proactive	Tree	Hier	–	General	2024
Voyager [128]	Progressive	Seq	Intlv	✓	General	2024
ADaPT [139]	Progressive	Tree	Intlv	✓	General	2024
CodeAct [141]	Progressive	Program	Intlv	✓	General	2024
KnowAgent [143]	Progressive	Seq	Intlv	✓	General	2024
Planning Anything [136]	Proactive	Program	Seq	–	General	2025
Plan-and-Act [138]	Proactive	Seq	Hier	–	Web	2025
SelfGoal [140]	Progressive	Tree	Intlv	✓	General	2025
<i>B. Search-Based Planning</i>						
LLM-MCTS [145]	MCTS	State	Seq	–	Robotics	2023
RAP [147]	MCTS	State	Seq	–	General	2023
PromptAgent [146]	MCTS	State	Seq	–	General	2024
ToolChain* [150]	A*	Action	Seq	–	Tool	2024
REST-MCTS* [148]	MCTS	State	Seq	–	General	2024
PlanRAG [153]	Retrieve	Plan	Seq	✓	QA	2024
AFlow [149]	MCTS	Plan	Seq	–	General	2025
DiffuSearch [151]	Diffusion	State	Seq	–	General	2025
Agent S [127]	Greedy	Plan	Hier	✓	Web	2025

Strategy: temporal relation between planning and execution (Part A) or search algorithm (Part B) — Proactive full plan before execution; Progressive interleaved plan-execute loop; MCTS Monte Carlo Tree Search; A* heuristic A* search; Retrieve retrieval-guided; Diffusion diffusion-based; Greedy greedy. *Structure:* plan representation (Part A) or search space (Part B) — Seq = sequential; DAG = directed acyclic graph; Tree = tree-structured; Program = programmatic; State/Action/Plan = corresponding search space. *Exec Mode:* Seq = sequential; Hier = hierarchical; Intlv = interleaved with planning.

dimensions that capture the lifecycle of this capability: *tool capability acquisition*, which concerns how models learn to use tools; *tool invocation*, which concerns the control-flow patterns through which tools are called during task execution; and *tool generalization*, which concerns how learned tool-use abilities transfer to unseen tools and environments.

2.5.1 Tool Capability Acquisition

Tool capability acquisition addresses how a model learns to recognize when a tool is needed, select the appropriate one, generate valid invocation parameters, and incorporate the returned results. Existing approaches can be broadly divided into trajectory-based learning, which internalizes tool-use behavior from demonstration sequences, and protocol-based alignment, which grounds the model’s understanding in tool interface descriptions.

Trajectory-Based Learning Toolformer [154] pioneered self-supervised tool learning by having the model annotate its own training corpus with API calls and retaining only those that reduce perplexity, enabling autonomous acquisition of tool-use behavior without task-specific supervision. ToolLLM [155]

scales this idea to 16,000+ real-world APIs by using ChatGPT to generate diverse instruction–trajectory pairs and training open-source models via supervised fine-tuning, while APIGen [156] further improves data quality through a three-stage verification pipeline covering format, execution, and semantic correctness. GPT4Tools [157] and ToolACE [158] explore complementary directions in data synthesis: the former uses self-instruction from GPT-4, while the latter introduces tool self-evolution to systematically expand API diversity and dialog complexity.

Beyond static supervision, reinforcement learning enables models to refine tool-use strategies through interaction. ToolPlanner [159] employs a two-stage RL framework with path planning and dual feedback mechanisms for task completion and instruction following. WebRL [160] applies self-evolving curriculum RL to web agent training, progressively generating harder tasks from failed attempts and using a learned outcome reward model.

Protocol-Based Alignment Rather than learning from execution traces, protocol-based methods align the model’s representation with tool interface specifications. Gorilla [161] introduces Retriever-Aware Training (RAT), which conditions fine-tuning on retrieved API documentation so that the model learns to ground its calls in up-to-date specifications and adapt to documentation changes at test time. ToolkenGPT [162] takes a representation-level approach, encoding each tool as a special token in the model’s vocabulary so that tool invocation becomes a natural extension of next-token prediction without modifying the frozen language model. EasyTool [163] addresses the upstream bottleneck of documentation quality by transforming diverse and verbose tool documentation into unified, concise instructions, significantly reducing token consumption while improving tool-use accuracy. At the ecosystem level, the Model Context Protocol (MCP) [25] standardizes agent-tool integration through a universal JSON-RPC interface, replacing per-tool custom connectors with a single interoperable protocol.

2.5.2 Tool Invocation

Once tool-use capability has been acquired, the central question shifts to how tools are invoked during task execution. We distinguish three control-flow patterns of increasing complexity: one-shot calling, closed-loop calling, and workflow-based orchestration.

One-Shot Calling In the simplest setting, a single user instruction triggers a single tool call whose result directly produces the final response. This pattern underlies the basic function-calling interfaces of commercial APIs [164] and is the primary evaluation target of benchmarks such as BFCL [165], which systematically assesses serial, parallel, and multi-language function calls using Abstract Syntax Tree matching. Toolformer’s single-step API insertion [154] and Gorilla’s syntax-aware call generation [161] both operate in this regime, where the model must select the right tool and produce well-formed parameters in a single generation pass.

Closed-Loop Calling Many tasks require multiple rounds of tool interaction, where each tool’s output informs the next decision. CRITIC [87] exemplifies this pattern: the model generates an initial response, verifies it against external tools such as calculators or search engines, and revises based on the feedback. AnyTool [98] implements a hierarchical, self-reflective closed loop over a large API pool, dynamically selecting and invoking tools across multiple rounds. ReAct [18] provides the foundational interleaving of reasoning traces and tool actions that underpins most closed-loop tool-use systems.

Workflow-Based Orchestration For complex goals requiring coordinated use of multiple tools, explicit workflow structures govern the invocation order and data flow. LLMCompiler [133] generates

a DAG of function calls with dependency tracking, enabling parallel execution wherever possible. WorkflowLLM [166] constructs a large-scale benchmark of real-world workflows from Apple Shortcuts and fine-tunes models to orchestrate complex multi-tool sequences with nested branches and loops. GPTSwarm [167] models the orchestration as an optimizable graph of LLM invocations, while AFlow [149] uses MCTS to automatically search over workflow structures composed of reusable operators.

2.5.3 Tool Generalization

A practical agent must handle tools it has never seen during training and retrieve appropriate tools from large, evolving libraries. Tool generalization addresses these challenges along two complementary directions: adapting to unseen tools and discovering relevant tools from large collections.

Unseen Tool Generalization Generalizing to previously unseen tools requires the model to rely on documentation comprehension rather than memorized call patterns. GenTool [168] systematically trains for this through two complementary strategies—zero-to-one generalization (handling entirely new tools) and weak-to-strong generalization (transferring from simpler to more complex tools). TOOLVERIFIER [169] improves generalization by generating contrastive questions that help the model self-verify its tool selection and parameter generation for unfamiliar APIs. Gorilla’s retriever-aware architecture [161] also contributes here, as its conditioning on retrieved documentation enables adaptation to test-time API changes without retraining. ToolLLM [155] demonstrates strong zero-shot generalization on the out-of-distribution APIBench benchmark despite being trained on a disjoint API set.

Table 4: Comparison of tool-use enhancement methods (Section 2.5).

Method	Training	Tool Interface	Tool Set	Control Flow	Gen Target	Task Domain	Year
Toolformer [154]	Self-sup	Func-Call	Closed	One-shot	In-dist	General	2023
ToolkenGPT [162]	SFT	Func-Call	Closed	One-shot	New-Task	General	2023
GPT4Tools [157]	SFT	Func-Call	Closed	One-shot	New-Task	General	2023
CRITIC [87]	Prompt	Func-Call	Open	Closed-loop	In-dist	General	2024
Gorilla [161]	SFT	Func-Call	Dynamic	One-shot	New-Tool	Code	2024
ToolLLM [155]	SFT	Func-Call	Open	Closed-loop	New-Tool	General	2024
AnyTool [98]	Prompt	Func-Call	Open	Closed-loop	New-Tool	General	2024
APIGen [156]	SFT	Func-Call	Open	One-shot	New-Tool	General	2024
ToolPlanner [159]	RL	Func-Call	Open	Closed-loop	New-Task	General	2024
Re-Invoke [170]	Prompt	Retrieval	Open	One-shot	New-Tool	General	2024
TOOLVERIFIER [169]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2024
MetaTool [171]	Prompt	Func-Call	Open	One-shot	New-Tool	General	2024
ToolACE [158]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2025
EasyTool [163]	Prompt	Func-Call	Dynamic	One-shot	New-Tool	General	2025
WebRL [160]	RL	Web	Open	Closed-loop	New-Env	Agent	2025
WorkflowLLM [166]	SFT	Func-Call	Open	Workflow	New-Tool	General	2025
GenTool [168]	SFT	Func-Call	Dynamic	One-shot	New-Tool	General	2025
BFCL [165]	—	Func-Call	Dynamic	One-shot	—	General	2025
ToolRet [172]	—	Retrieval	Open	—	New-Tool	General	2025

Training: learning paradigm — Self-sup = self-supervised; SFT = supervised fine-tuning; RL = reinforcement learning; Prompt = prompting only (no training). *Tool Interface:* interaction modality — Func-Call = function-calling; Retrieval = retrieval-based selection; Web = direct web interaction. *Tool Set:* tool availability at test time — Closed fixed set, known at training; Open extensible to unseen tools; Dynamic available tools change across tasks. *Control Flow:* invocation pattern — One-shot = single invocation; Closed-loop = iterative invocation with feedback; Workflow = multi-step orchestration. *Gen Target:* generalization target evaluated — In-dist = in-distribution only; New-Task = unseen tasks; New-Tool = unseen tools; New-Env = unseen environments.

Tool Discovery and Retrieval As tool libraries scale to thousands of APIs, identifying the right tool for a given query becomes a retrieval problem in its own right. ToolLLM [155] trains a neural

API retriever that recommends relevant APIs from a large pool, while Re-Invoke [170] proposes an unsupervised approach that enriches tool documents with synthetic queries and extracts user intents through multi-view similarity ranking, achieving strong zero-shot retrieval without any labeled query-tool pairs. ToolRet [172] provides the first comprehensive tool retrieval benchmark and reveals that existing information retrieval models, despite strong performance on general retrieval tasks, perform poorly on tool retrieval, highlighting a significant gap between conventional document retrieval and tool-oriented retrieval. MetaTool [171] addresses the upstream decision of whether to use a tool at all and which tool to select, framing these as meta-cognitive capabilities that precede the invocation itself.

A related line of work abstracts tool discovery to the level of reusable skills. SkillWeaver [173] enables web agents to autonomously synthesize reusable skills as APIs through iterative environment exploration, building transferable skill libraries. SAGE [174] enhances this paradigm with reinforcement learning, using sequential rollouts across similar tasks to improve skill acquisition.

2.6 Evaluation

The capabilities reviewed in Sections 2.2–2.5 require systematic benchmarking to identify where individual agents succeed and where they fall short. As summarized in Table 5, the evaluation landscape for LLM-based agents has expanded rapidly in recent years, exhibiting two notable trends. First, benchmarks have moved from testing isolated skills toward joint evaluation of reasoning, planning, and tool use within unified task settings. Second, evaluation criteria have shifted from purely outcome-based metrics (e.g., success rate or exact match) toward process-level signals such as action trajectories, subgoal completion, and execution traces, which provide finer diagnostic resolution.

Comprehensive benchmarks aim to stress-test multiple capabilities simultaneously. AgentBench [175] was among the first to evaluate agents across eight diverse environments with trajectory-level assessment. GAIA [176] and MINT [177] further combine web interaction and tool use within multi-turn settings. More recent efforts raise the bar in evaluation granularity: AgentBoard [178] introduces subgoal-based progress metrics that capture partial task completion, while TheAgentCompany [179] situates evaluation in realistic enterprise workflows requiring long-horizon coordination across web and code environments.

Beyond comprehensive suites, a large body of work targets specific execution environments or individual capabilities. In web environments, WebArena [45] constructs self-hosted websites for reproducible long-horizon evaluation, VisualWebArena [180] extends this to multimodal web tasks, and Mind2Web [181] provides large-scale cross-website coverage. BrowseComp [182] further challenges agents with deeply entangled, hard-to-find information retrieval. OSWorld [183] and AndroidWorld [184] bring evaluation to desktop and mobile operating systems, adopting state verification as the primary evaluation signal. In code and software engineering, SWE-bench [12] has become the de facto standard for assessing agents on real GitHub issue resolution over extended interaction horizons. For tool use specifically, ToolBench [185] offers large-scale API evaluation with over 12,000 instances, BFCL [165] introduces AST-level matching for precise assessment of function-call correctness, and AppWorld [186] and τ -bench [187] embed tool use within richer application ecosystems and dialogue contexts. Planning benchmarks such as ALFWorld [188] and ScienceWorld [189] evaluate long-horizon action sequences in text-based worlds. Memory evaluation remains comparatively nascent: LoCoMo [190] tests long-conversation memory through entailment-based metrics, LongMemEval [191] systematically probes multi-session recall, and MemoryAgentBench [192] couples memory assessment with application-level interaction.

Recent benchmarks have also expanded into domain-specific and safety-critical settings. In science and

machine learning, MLE-bench [193], CORE-Bench [194], and ScienceAgentBench [195] evaluate agents on automated experimentation and code-based research tasks, while PaperBench [196] assesses the ability to reproduce research papers using rubric-based subgoal evaluation. Enterprise benchmarks such as WorkArena++ [197] and CRMArena [198] test agents in realistic business applications, though both remain only partially open. On the safety front, Cybench [199] targets cybersecurity challenges and AgentHarm [200] evaluates susceptibility to harmful tool-use behaviors. Despite this breadth, several gaps persist. Cross-capability evaluation that examines how improvements in one module (e.g., memory) affect performance in another (e.g., planning) remains largely absent. Efficiency metrics such as token consumption, latency, and cost are rarely incorporated into benchmark design. These gaps suggest that future evaluation efforts should move toward integrated, resource-aware assessment protocols. Moreover, the capability bottlenecks revealed by single-agent benchmarks provide a direct motivation for the multi-agent collaboration strategies discussed in the next section.

2.7 Discussion

Modular optimization dominates, but inter-module coupling remains unstudied. The formalization in Section 2.1 shows that reasoning, memory, planning, and tool use are interdependent at every time step, yet existing methods almost exclusively optimize a single module in isolation. Capability mismatches across modules often become the actual bottleneck in end-to-end deployment [119, 155], but such failure modes are invisible to current benchmarks, which predominantly target one or two capabilities (Table 5).

Reasoning enhancement and hallucination suppression are converging but methodologically separated. Expanding the search space [59, 60] produces richer candidate paths while simultaneously increasing the surface area for errors. Process reward models [67] bridge the two concerns but scale poorly beyond mathematical domains. Most reliability assessment remains post-hoc [70, 73], leaving open how to dynamically steer search based on step-level confidence during inference.

Memory design is fragmented, with little understanding of how design choices interact. Table 2 shows diverse combinations of formation, storage, maintenance, and retrieval, yet no work systematically studies their interactions. Graph-based storage excels at relational retrieval but incurs high maintenance cost [118]; utility-driven maintenance requires extensive interaction to converge [126]; and storage formats are typically chosen without regard for downstream retrieval compatibility.

Capabilities are evaluated as static snapshots, overlooking continuous adaptation. All methods in this chapter assume frozen strategies and known tool sets, yet real deployment demands ongoing adaptation. Tool generalization and memory evolution treat adaptation as a one-time capacity rather than a continuous process.

3 Multi-Agent Collaboration

Multi-Agent Systems (MAS) are analogous to division of labour and cooperation in human societies, where agents collaborate to solve complex problems. By integrating agents with distinct capabilities, MASs can tackle tasks beyond the ability of any individual agent [201, 202]. In this paper, we formally define a MAS S as

$$S = (\mathcal{A}, \mathcal{E}, \mathcal{C}, \mathcal{G}, \Pi), \quad (8)$$

where

Table 5: Single-agent capability benchmarks, organized by evaluation domain.

Benchmark	Capability	Environment	Interaction	Observable Signal	Eval. Method	Scale	Open	Year
<i>A. Comprehensive</i>								
AgentBench	R P T	Mixed	Multi	ActTraj	SR	~1K	✓	2023
GAIA	R P T	Web + Tool	Multi	Outcome	EM	466	P	2023
MINT	R P T	Mixed	Multi	ToolSeq	SR	586	✓	2023
AgentBoard	R P T	Mixed	Multi	Subgoal	Comp	1,013	✓	2024
TheAgentCompany	R P T	Web + Code	Long	ExecTrace	Comp	175	✓	2024
<i>B. Web</i>								
WebArena	R P T	Web	Long	ActTraj	Exec	812	✓	2023
Mind2Web	R P T	Web	Multi	ActTraj	Comp	2,350	✓	2023
VisualWebArena	R P T	Web	Long	ActTraj	Exec	910	✓	2024
BrowseComp	R P T	Web	Multi	Outcome	EM	1,266	✓	2025
<i>C. Desktop & Mobile</i>								
OSWorld	R P T	Desktop	Long	StateVer	Exec	369	✓	2024
AndroidWorld	R P T	Mobile	Long	StateVer	SR	116	✓	2024
Mobile-Bench	R P T	Mobile	Multi	ActTraj	Comp	832	✓	2024
<i>D. Code & Software</i>								
SWE-bench	R P T	Code	Long	ExecTrace	Exec	2,294	✓	2024
Terminal-Bench	R P T	Terminal	Long	ExecTrace	Exec	89	✓	2025
<i>E. Tool Use</i>								
ToolBench	R P T	API	Multi	ToolSeq	LLM-J	12,657	✓	2023
API-Bank	R P T	API	Multi	ToolSeq	Comp	314	✓	2023
T-Eval	R P T	API	Multi	ToolSeq	Step Acc	553	✓	2024
AppWorld	R P T	App	Long	StateVer	Exec	750	✓	2024
τ -bench	R P T	Dialogue+Tool	Long	StateVer	Exec	168	✓	2024
BFCL	R P T	API	Multi	ToolSeq	AST+Exec	2,000+	✓	2025
ToolSandbox	R M P T	Dialogue+Tool	Long	Milestone	Comp	180	✓	2025
<i>F. Planning</i>								
ALFWorld	R P T	Text World	Long	ActTraj	SR	134	✓	2021
ScienceWorld	R M P	Text World	Long	ActTraj	SR	300	✓	2022
<i>G. Memory</i>								
LoCoMo	R M	Dialogue	Long	NLI	F1+Human	10 convs	✓	2024
LongMemEval	R M	Dialogue	Long	Subgoal	Acc	500	✓	2025
MemoryAgentBench	R M P	Dialogue+App	Long	ActTraj	Comp	1,000+	P	2026
<i>H. Science & ML</i>								
MLE-bench	R P T	Code	Long	ExecTrace	Exec	75	✓	2024
MLAgentBench	R P T	Code	Long	ExecTrace	Exec	13	✓	2024
CORE-Bench	R P T	Code	Long	ExecTrace	Exec	270	✓	2024
ScienceAgentBench	R P T	Code	Long	ExecTrace	Exec	102	✓	2025
PaperBench	R P T	Code + Web	Long	Subgoal	Rubric	20 papers	P	2025
<i>I. Enterprise</i>								
WorkArena++	R P T	Web	Long	ActTraj	SR	682	P	2024
CRMArena	R P T	CRM	Single	Outcome	Fuzzy	1,186	P	2024
<i>J. Safety</i>								
Cybench	R P T	Code+Terminal	Long	ExecTrace	SR	40	✓	2025
AgentHarm	R P T	API	Multi	ToolSeq	SR	440	✓	2025

Capability: R Reasoning; M Memory; P Planning; T Tool Use. Filled = primary; outlined (e.g. R) = secondary; absent = not covered.
Interaction: Single = single-turn; Multi = multi-turn; Long = long-horizon. **Signal:** ActTraj = action trajectory; ToolSeq = tool-call sequence; StateVer = state verification; ExecTrace = execution trace; Subgoal = subgoal completion; Milestone = milestone completion; NLI = NLI-based verification; Outcome = final outcome only. **Eval.:** SR = success rate; EM = exact match; Exec = execution-based; AST+Exec = AST matching + execution; Step Acc = step-level accuracy; LLM-J = LLM-as-judge; Comp = composite (multi-metric); F1+Human = F1 + human eval; Fuzzy = fuzzy matching; Rubric = rubric-based; Acc = accuracy. **Open:** ✓ = fully open; P = partially open (e.g. test set hidden).

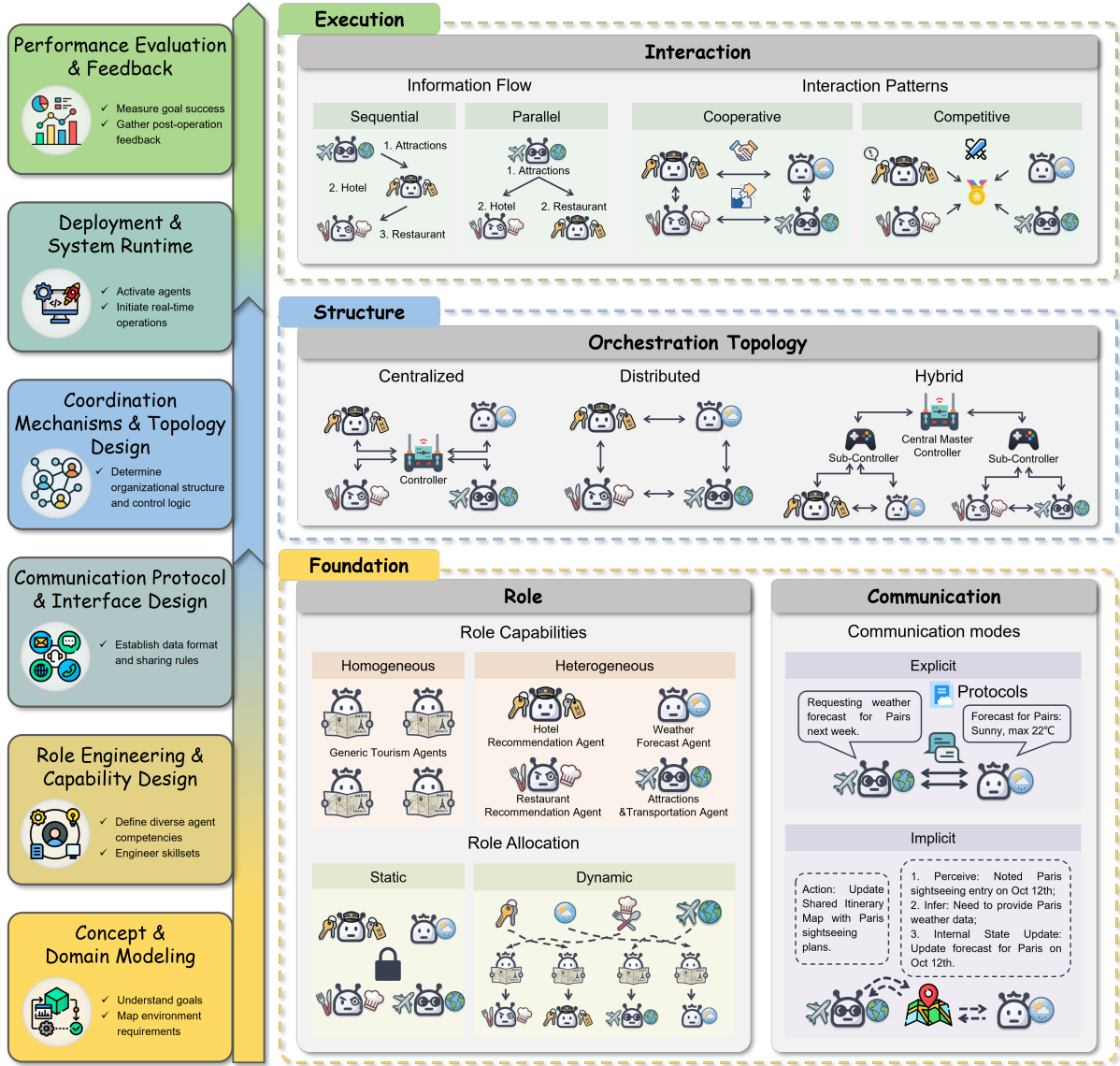


Figure 4: Overview of the multi-agent collaboration system. The right panel details a three-tiered design framework comprising the Foundation, Structure, and Execution layers to categorize agent roles, communication modes, orchestration topologies, and interaction dynamics. The left panel outlines the bottom-up system lifecycle, progressing from concept and domain modeling through to performance evaluation and feedback.

- $\mathcal{A} = \{a_1, \dots, a_N\}$ is the set of agents;
- $\mathcal{E}$ denotes the shared environment that may influence the operation of a MAS S ;
- $\mathcal{C}$ is the communication protocol that governs message exchange;
- $\mathcal{G}$ denotes the orchestration topology describing the architecture of a MAS S ;
- Π denotes the set of collaboration policies.

Under this formulation, we argue that multi-agent collaboration should be understood as a structured and goal-oriented computational paradigm, which can be decomposed into four key components: **Role, Communication, Orchestration, Interaction**. As shown in Fig. 4, each component addresses a fundamental question in collaborative intelligence.

Role *Who acts?* Let z_i denote the role assigned to agent $a_i \in A$. We define role specialization through the mapping $z_i = \rho(a_i)$, $z_i \in Z$, where $\rho(\cdot)$ is the role allocation function and Z denotes the set of possible roles. This component determines which agents are needed for collaboration and clearly specifies their responsibilities, such as planning, reasoning, or execution.

Communication *How do agents communicate?* During collaboration, agents exchange messages with one another subject to the protocol $\mathcal{C}$. Let $m_{i \rightarrow j, t}$ denote a message sent from agent a_i to agent a_j at time step t . The communication process can be modeled by a message-generation mechanism $\text{Comm}_{\mathcal{C}}$, such that

$$m_{i \rightarrow j, t} \sim \text{Comm}_{\mathcal{C}}(\cdot \mid a_i, a_j, \mathcal{H}_{i, t}), \quad (9)$$

where $\mathcal{H}_{i, t}$ denotes the information available to agent a_i at time step t , including its local interaction history and any accessible shared context. This component specifies how the message is generated, structured, and transmitted among agents.

Orchestration *How are agents connected?* Let G denote the organizational structure of the MAS S . The orchestration topology is defined as

$$\mathcal{G} = (\mathcal{A}, \mathcal{E}_c), \quad \mathcal{E}_c \subseteq \mathcal{A} \times \mathcal{A}, \quad (10)$$

where $\mathcal{E}_c$ represents the set of communication links between agents. If

$$\mathcal{E}_c = \{(a_i, a_j) \in \mathcal{A} \times \mathcal{A} \mid i \neq j\}, \quad (11)$$

then all distinct agents are fully interconnected, allowing direct communication between every pair of agents in the MAS S . This component focuses on the organizational structure of agents and the design of message-passing pathways.

Interaction *How do agents collaborate during execution?* Interaction characterizes how a MAS S operates as a whole under a collaboration policy $\pi_{\text{coll}} \in \Pi$ and environment $\mathcal{E}$. It describes the execution process in which agents act, exchange messages, and coordinate with each other to accomplish the task. Interaction can be viewed as the implementation of the collaboration policy π_{coll} at the execution level. Under this view, a MAS S induces an execution process that can be represented as trajectories $\tau \sim \pi_{\text{coll}}$ consisting of actions, inter-agent messages, and intermediate reasoning steps.

Together with Role, Communication, and Orchestration, Interaction completes the specification of a MAS by bridging structural design and runtime behaviour. Evaluating a MAS S and its trajectories τ provides a unified lens for comparing different design choices across the four core components mentioned above.

3.1 Role

Understanding a role requires answering two fundamental questions: what can agents do and how responsibilities are distributed among them. Therefore, we decompose roles into two dimensions: **Role Capability** answers the first question, and **Role Allocation** answers the second question by assigning these capabilities through a mapping function $\rho(\cdot)$.

3.1.1 Role Capability

In multi-agent collaboration, role capability refers to the set of competencies, action spaces, and observation spaces associated with agents engaged in the collaborative process [201, 203]. It captures

not only what an agent a_i can do (such as reasoning, planning, or tool use, as seen in 2), but also what information it can access and how it can influence the environment E or other agents $a_j \in A$. In some frameworks [204, 205, 206], multiple agents are instantiated from the same LLM and consequently share identical capabilities, resulting in homogeneous role configurations. In contrast, other frameworks [207, 208, 209] employ agents built on different models or deliberately assign specialized responsibilities, producing heterogeneous role configurations characterized by asymmetric capabilities and differential access to information. As shown in Fig.4, role capabilities in multi-agent collaboration can be broadly classified into **homogeneous** and **heterogeneous** types.

Homogeneous Role Capabilities In MASs, multiple agents are driven by the same underlying LLM and consequently share highly similar reasoning capability, action space, and observation scope. In this paradigm, the variation among agents arises primarily from role prompting or task framing rather than differences in intrinsic capabilities or information access. This homogeneous setup provides a controlled environment to systematically study how role definitions and interaction protocols shape collective behaviour, without confounding factors from model heterogeneity.

A canonical example of homogeneous role capabilities is frequently observed in multi-agent frameworks where agents are instantiated from the same underlying language model. In CAMEL [210], both AI User and AI Assistant are created from a single LLM and interact symmetrically, with differences mainly arising from prompt instructions rather than model heterogeneity. Similarly, Generative Agents [110] simulate human-like behaviours through multiple agent instances derived from the same base model, enabling complex social interactions and emergent narratives in a controlled, homogeneous environment. Frameworks like SelfCorrect-Agent [206] and Chateval [205] further explore homogeneous multi-agent setups for robust reasoning and evaluation, leveraging symmetric LLM-based agents to iteratively correct or debate responses. VillagerAgent [204] demonstrates that even in graph-structured task coordination, MAS can be instantiated from identical models, emphasizing interaction over agent heterogeneity.

Building on these observations, homogeneous roles offer several practical advantages. Reusing the same model interfaces and toolsets across agents reduces engineering complexity, and coordination mechanisms are simplified because capability mismatches are minimized. At the same time, entirely homogeneous teams may encounter limitations when addressing highly complex or specialized tasks, where diversity in role capabilities—such as verification, domain-specific expertise, or advanced reasoning—becomes critical for robust and effective collaboration.

Heterogeneous Role Capabilities In contrast, heterogeneous role capabilities describe systems in which agents are driven by different models or are designed with distinct functional specializations. Agents may differ in computational reasoning ability, observation access, or permitted actions. This form of heterogeneity expands system expressiveness, enabling division of labor, modular interaction, and specialization, which is crucial for complex tasks that require expertise beyond general reasoning.

A notable example is the X-MAS framework [211], which explicitly constructs multi-agent systems with diverse LLMs to harness collective intelligence across models of varying strengths and specializations, demonstrating significant improvements in both general reasoning and specialized domains compared to homogeneous setups. Similarly, the Adaptive Heterogeneous Multi-Agent Debate (A-HMAD) framework [212] assigns distinct roles to agents—such as logical reasoning, factual verification, and strategic planning—enabling more effective error-checking and domain coverage, and empirically reducing factual errors across benchmarks. More recently, MAPORL [24] demonstrates the emergence of heterogeneous roles through post-training and RL, enabling specialized coordination and task

optimization in multi-agent systems.

Heterogeneous capabilities also arise naturally in systems that assign task-specific responsibilities. For example, multi-agent educational personalization systems assign Profiler, Rewriter, and Evaluator agents with distinct objectives and information access to tailor STEM content to individual learners [213]. In medical decision-making, frameworks like Mdagents employ LLM-based agents with complementary roles—such as symptom analysis, treatment recommendation, and risk assessment—improving diagnostic performance and safety [214]. In software engineering, automated document summarization benefits from multi-agent LLM collaboration, where Extractor, Summarizer, and Validator agents coordinate to generate concise and accurate software documentation [215], while in industrial optimization, OptDisPro [216] dynamically assigns agents to search, evaluate, or refine solutions, flexibly balancing exploration and exploitation and outperforming homogeneous optimization agents. Collaborative frameworks like Autogen [217] further illustrate heterogeneity through differentiated observation spaces, where commanders interact directly with users or environments while subordinate agents receive only intermediary context.

Overall, these studies demonstrate that heterogeneous role capabilities consistently enhance multi-agent systems by promoting specialization, improving task performance, and increasing robustness and adaptability across diverse domains.

3.1.2 Role Allocation

Role allocation refers to the division of labour in collaborative agent systems, involving the assignment of tasks and the clarification of responsibilities among agents. Although the concepts of *task* and *role* are often used in the multi-agent literature [218, 219, 220, 221], they describe different aspects of collaboration. Tasks usually refer to the units of work to be executed, whereas roles z_i represent the functional positions that enable agents a_i to perform those tasks. Across a range of existing frameworks [222, 223, 224], roles emerge from the mapping $\rho : A \rightarrow Z$ between agent capabilities and task requirements, which explains why the two notions are often closely coupled in practice. Here, *role* refers to the entities endowed with the capability to address particular tasks, rather than the tasks themselves. According to the mechanism by which roles z_i are distributed via $\rho(\cdot)$, **static role allocation** and **dynamic role allocation** approaches are illustrated in Fig.4.

Static Role Allocation Under static role allocation in multi-agent systems, agent roles and responsibilities are specified prior to task execution. By defining the team structure in advance, this approach establishes clear boundaries in terms of agent capabilities, role identities, and task objectives. In practice, static role allocation is typically implemented through manually designed templates, often realized using prompt engineering techniques to specify role descriptions.

This paradigm has been widely used in multi-agent systems designed for structured software engineering workflows. For example, ChetDev [13] organizes agents into predefined roles within a software development pipeline, enabling coordinated execution across stages such as planning, coding, and testing. Similarly, CodeDelegator [225] assigns agents specialized roles such as context manager and code reviewer to mitigate context pollution in collaborative coding tasks. ATOMIZER [226] adopts a comparable design for version control scenarios, where agents handle distinct aspects of commit untangling and conflict resolution. Static role allocation is also effective in data-centric and document-oriented tasks where responsibilities can be clearly partitioned. For instance, SheetAgent [227] divides spreadsheet-related operations into predefined roles such as data entry and data analysis, allowing agents to focus on specific subtasks while reducing coordination overhead. In requirements engineering, the MARE system [228] similarly distributes responsibilities among agents with specialized expertise to

support structured requirement analysis and task delegation.

Beyond general data processing tasks, static role allocation has also been applied in domain-specific multi-agent systems where reliability and consistency are essential [229]. In healthcare applications, ColaCare [230] employs predefined roles for collaborative health record modeling to ensure stable and accurate task execution. MEDCO [231] adopts a similar design in medical education, where agents assume roles such as tutor and assessor to guide and evaluate learners within a structured training process. Several systems employ static role allocation to facilitate structured conversational or collaborative interactions. For example, CGMI [232] assigns agents roles based on task requirements to support coordinated multi-agent reasoning, while MultiAgentESC [233] defines specialized conversational roles to improve the quality and stability of emotional-support dialogues.

Overall, static role allocation provides a simple yet effective mechanism for organizing multi-agent collaboration in structured environments. By clearly separating responsibilities and limiting role ambiguity, it significantly reduces coordination complexity and improves execution efficiency. However, its reliance on predefined structures also introduces rigidity. When task requirements evolve or unexpected situations arise, static role allocation may lack the flexibility required for adaptive collaboration, motivating the development of dynamic role allocation strategies.

Dynamic Role Allocation In contrast to static role allocation, dynamic role allocation enables agents to adaptively select or adjust their roles based on task requirements and environmental context. Rather than relying on predefined templates, agents can determine the expertise required for a task and organize collaboration at runtime. As a result, roles may emerge and evolve during execution, allowing multi-agent systems to operate more flexibly in dynamic and open-ended environments.

Early research on dynamic role allocation primarily focuses on dynamic role selection, where agents recruit collaborators from a candidate pool according to task requirements. In this paradigm, agents identify the skills needed for a task and select suitable participants to form a task-oriented team. For instance, AutoAgents [234] allows agents to autonomously choose roles from a pool of available agents, enabling flexible collaboration in environments where tasks and agent capabilities continuously change. Similarly, the MedAide framework [235] explores this mechanism in medical decision-making scenarios, where agents dynamically select collaborators and adjust their roles based on evolving medical intents and information sources. In Agent Hospital [236], agents are recruited to address specific clinical situations, with roles dynamically determined according to the needs of the medical scenario.

More recent work extends this paradigm toward active initialization, where agents not only select collaborators but also dynamically instantiate or expand agent teams during task execution. In this setting, agents can initiate collaboration by generating new agents or integrating additional participants as task complexity increases. For example, AgentVerse [237] allows agents to dynamically initialize and organize multi-agent teams according to task demands, enabling roles to emerge through real-time coordination rather than predefined assignment. Auto-scaling LLM-based multi-agent systems [238] further explore this idea by dynamically integrating agents to scale system capacity based on workload and task complexity. Beyond systems, dynamic role allocation has also been investigated in reinforcement learning settings. Roma [239] demonstrates that agents can develop role specialization through Multi-Agent Reinforcement Learning (MARL) [240, 241, 242], allowing roles to emerge and evolve through interaction. Building on this idea, ResMAS [243] dynamically assigns roles to maintain system resilience when unexpected events or failures occur. Similarly, ARL-SMCS [244] enables agents to learn role assignments through evolutionary reinforcement learning in UAV–vehicle collaboration tasks.

In summary, dynamic role allocation significantly enhances the flexibility and scalability of multi-agent collaboration by allowing agents to reorganize themselves according to task requirements. However, this adaptability introduces additional coordination complexity, as agents must continuously evaluate task states, available resources, and the roles of other participants. Consequently, while dynamic role allocation is well suited for open-ended and evolving environments, it may also incur additional overhead in maintaining stable and efficient coordination.

3.2 Communication

Communication can be described at different levels. **Communication Modes** capture the high-level patterns of information sharing, whereas **Communication Protocols** define the concrete rules and procedures through which these patterns are implemented in practice.

3.2.1 Communication Modes

Communication is a fundamental mechanism that enables coordination and information sharing in MASs [39, 245]. In collaborative settings, agents exchange information to align their decisions, share intermediate results, and collectively progress toward task objectives. In LLM-based MASs, communication typically occurs through structured or unstructured message exchanges, where the communication protocol C governs how messages $m_{i \rightarrow j}$ are generated and transmitted between agents a_i and a_j . In addition to such direct message passing, agents may also communicate by leveraging their perception capabilities, inferring information from observed behaviours, environmental states, or the actions reflected in others' histories. As shown in Fig.4, communication modes in agentic systems can be broadly categorized into **explicit communication** and **implicit communication**.

Explicit Communication Explicit communication refers to agents exchanging information through well-defined message formats. In this mode, agents explicitly transmit task instructions, intermediate results, reasoning traces, or feedback through structured or unstructured messages.

Communication messages can take various forms, ranging from structured data such as code or documents to more flexible natural-language exchanges. In MetaGPT [246], agents including Product Managers, Architects, and Engineers coordinate software development workflows by exchanging structured documents, such as product requirement documents, system design specifications, and implementation plans. Similarly, TalkHier [247] introduces a structured communication protocol to support context-rich exchanges among agents. Beyond structured artifacts, some communication relies on natural-language messages. For example, MAGIS [248] coordinates multiple specialized agents to collaboratively analyze and resolve GitHub issues through iterative dialogue, where agents exchange natural-language analyses, proposed solutions, and verification feedback. Autodata [249] similarly organizes agents to collect and process open web data, enabling them to share extracted information, data summaries, and task progress through shared communication channels. EcoLANG [250] leverages context and social simulation, enabling agents to establish interpretable coordination protocols for efficient task execution. MegaAgent [251] further extends this paradigm by supporting large-scale autonomous collaboration without rigid predefined workflows, allowing agents to dynamically generate messages describing plans, intermediate findings, or coordination signals.

Subsequent studies further investigate how the design and organization of messages influence the efficiency and scalability of multi-agent communication. The economical communication pipeline proposed in AgentPrune [252] reduces redundant interactions by filtering, summarizing, and compressing agent messages, demonstrating that carefully designed message representations can significantly lower token consumption while preserving essential task information. To further improve communication

efficiency, TodyComm [253] introduces a task-oriented dynamic communication strategy in which agents selectively exchange task-relevant messages across interaction rounds, adaptively determining both communication targets and message contents. Other approaches [254, 255, 256] focus on adaptive or consensus-based communication mechanisms, where agents dynamically adjust messaging policies and exchange viewpoints to reach collective decisions through negotiation or voting. Beyond improving communication efficiency, recent research also examines how message exchange structures affect collaboration dynamics. For instance, AgentCoord [257] provides visualization tools to analyze the structure and flow of agent messages, revealing how different communication patterns influence task efficiency and coordination behaviours.

In summary, explicit communication remains the dominant paradigm in LLM-based MASs due to its transparency, controllability, and compatibility with role-based collaboration. By combining structured communication protocols with flexible dialogue mechanisms, modern frameworks continue to improve coordination efficiency and scalability in increasingly complex agent ecosystems.

Implicit Communication Under implicit communication, agents infer the intentions and states of others by observing environmental changes and behavioral patterns. In this mode, communication emerges from perception, memory retrieval, and reasoning processes: agents monitor their surroundings, interpret the actions of peers, and use internal representations to infer context and coordinate decisions. A representative example of this paradigm is the Generative Agents framework [110]. In this system, agents simulate human-like social behaviour by continuously perceiving environmental events, retrieving relevant memories, and generating contextually appropriate actions or utterances.

In MARL [258], observation-driven communication can emerge naturally when agents optimize shared objectives. Shaw et al. [259] study cooperative foraging agents that learned coordination behaviours through environmental feedback. Subsequently, Buscemi et al. [260] demonstrate that LLM-based agents could implicitly align their numerical strategies through repeated interactions, effectively conveying information through observable choices rather than explicit messages.

Subsequent work has extended this principle to scenarios requiring more sophisticated reasoning about other agents. Wang et al. [261] introduce a theory-of-mind-based framework in which agents anticipate teammates' goals by interpreting their observable actions, thereby improving coordination in the absence of explicit communication channels. In addition, communication-free approaches have demonstrated that direct message passing may not always be necessary. For example, Ge et al. [262] show that agents could achieve effective cooperation in tasks such as box-pushing solely through observation and environmental interaction. LatentMAS [263] facilitates implicit communication by enabling agents to infer intentions and synchronize actions through shared environmental cues, providing a robust coordination mechanism when explicit messaging is costly or unavailable.

3.2.2 Communication Protocols

Multi-agent communication protocols C , also referred to as communication rules, define how agents exchange information, coordinate actions, and achieve shared goals in distributed environments. In MASs, communication protocols support a variety of collaborative processes, including task allocation, information sharing, negotiation, decision-making, and conflict resolution.

Similar to how the TCP/IP [264, 265] and HTTP [218, 266] protocols ushered in an unprecedented era of global connectivity on the Internet, standardized agent communication protocols are becoming a key infrastructure for building interconnected intelligent systems [267, 268, 269] and collaborative intelligence networks [270, 271, 272]. By providing common communication standards, these protocols [273]

enable heterogeneous agents developed by different platforms or organizations to interact seamlessly, thereby facilitating large-scale agent ecosystems.

Table 6 compares several representative communication protocols. In this paper, we categorize existing communication protocols into two main groups: internal protocols, which govern communication among agents within a multi-agent system, and external protocols, which regulate interactions between agents and external systems, such as tools, services, or user interfaces.

Table 6: Comparison of agent communication protocols.

Protocol	Proposer	Comm. Type	Msg. Format	Core Functions	Year
Agora [274]	University of Oxford	A2A	PD	Consensus formation	2024
MCP [275]	Anthropic	A2T	JSON-RPC	Standardized context access	2024
ANP [276]	ANP Community	A2A	JSON-LD+NL	Meta-protocol negotiation	2024
LMOS [277]	Eclipse Foundation	A2S	JSON-LD+TD	Group management	2024
ACP [278]	IBM	A2A	JSON-LD	Async & Sync interaction	2025
A2A [26]	Google	A2A	JSON-RPC/SSE	Agent discovery	2025
Agent Protocol [279]	AI Engineer Foundation	A2S	REST/HTTP	Standardized task API	2025

Comm. Type: primary interaction entities — A2A Agent-to-Agent; A2T Agent-to-Tool; A2S Agent-to-System. *Msg. Format:* data serialization — PD Protobuf/Data-driven; JSON-RPC JSON-RPC; JSON-LD JSON-LD; JSON-LD+NL JSON-LD + NL; JSON-LD+TD JSON-LD + TD; JSON-RPC/SSE JSON-RPC + SSE; REST/HTTP REST/HTTP. *Core Functions:* primary capability or service provided by the protocol.

Internal Protocols Internal protocols define the communication rules among agents within MAS. These protocols primarily focus on enabling coordination, cooperation, and collective decision-making among multiple agents. Representative internal protocols include agent-to-agent communication frameworks such as Agent-to-Agent Protocol (A2A) [26], which enables direct peer-to-peer communication and task handoff between agents, and Agent Network Protocol (ANP) [276], which supports large-scale agent networks through mechanisms such as service discovery and message routing.

External Protocols External protocols govern the interaction between agents and external environments, including tools, databases, APIs, and user interfaces, through standardizing how agents retrieve information, invoke tools, and exchange contextual data with external systems. One representative example is MCP [275], which provides a standardized interface for connecting agents with external tools and data sources and supports context sharing, tool registration, and structured request-response interactions.

Beyond the internal–external distinction, these protocols also differ in interoperability mechanisms and deployment models. A2A and ANP both target interoperability within MASs, but with different emphases: A2A focuses on standardized cross-platform collaboration, whereas ANP is oriented toward AI-native networking and scalable open agent ecosystems. In contrast, MCP is client–server architecture that enables external resources to be integrated through a unified interface, thereby reducing integration overhead and improving modularity, maintainability, and reusability.

3.3 Orchestration

Orchestration describes the global coordination structure that governs how multiple agents are organized and interact within a MAS S . From a system-level perspective, orchestration can be viewed as the collaboration topology $\mathcal{G} = (\mathcal{A}, \mathcal{E}_c)$: it defines how agents $a_i \in \mathcal{A}$ are connected, how information flows through the communication links $\mathcal{E}_c$ among them, and how coordination decisions are structured

during task execution. Orchestration focuses on the higher-level structural organization that determines how these roles and communications are integrated into a coherent collaboration process.

Closely related to orchestration is the concept of agent routing, which determines how tasks or messages are dynamically directed to appropriate agents during execution. Routing mechanisms [222, 280, 281] are typically responsible for selecting which agent should process a particular subtask or piece of information, often based on capability matching, contextual reasoning, or learned policies. In contrast, orchestration defines the broader coordination topology within which routing decisions occur. In other words, routing can be regarded as an operational mechanism embedded inside an orchestration structure, while routing determines which agent should act next. For example, routing decisions may be locally determined by agents in decentralized architectures [282].

Existing research generally organizes orchestration mechanisms into three representative topological paradigms: **centralized**, **distributed**, and **hybrid** orchestration. These paradigms differ primarily in how decision-making authority, coordination responsibilities, and information flow are structured across agents, and they provide distinct trade-offs in terms of scalability, adaptability, and control.

3.3.1 Centralized Orchestration Topology

Centralized orchestration topology follows a controller-centric coordination structure in which a single orchestrator maintains global decision authority over the collaboration process. In terms of coordination topology, agents are organized around a central controller that determines interaction flow. Information from different agents is typically aggregated by the orchestrator, which then determines subsequent actions and redistributes instructions. Consequently, routing decisions are largely governed by the central coordinator rather than emerging from peer-to-peer interactions.

Recent studies have explored various enhancements to the centralized orchestration topology. In terms of memory and planning, StackPlanner [283] utilizes a centralized controller with task-level memory to maintain global context during execution. Similarly, AOP [284] relies on the orchestrator to centrally generate and sequence overarching action plans for all participating agents. To improve topological flexibility, the Puppeteer framework [285] enable the central node to dynamically reconstruct the interaction pathways among agents based on real-time task demands. Meanwhile, the efficient approach [286] optimizes the central coordinator’s routing decisions to minimize communication overhead within the centralized structure. Furthermore, CORL [287] uses an RL-based orchestrator to dynamically govern agent interactions, acting as a gateway to balance overall system performance and budget. Another domain-specific application, CCMA [288] employs centrally coordinated reinforcement learning, demonstrating how a single orchestrator can manage complex constraints, such as power grid topology, through global state visibility.

While centralized orchestration enables consistent global planning and simplified routing control, it may also introduce scalability limitations and potential single points of failure when the number of participating agents grows.

3.3.2 Distributed Orchestration Topology

Distributed orchestration topology removes the global coordinator and distributes decision authority across agents. From the perspective of coordination topology, agents interact through peer-to-peer connections rather than through a centralized controller. Information flows directly among agents, and routing decisions are determined locally based on each agent’s capabilities, policies, and observed context.

A representative example is AgentNet [282], which models agent collaboration as a dynamically

evolving network, equipping fully connected agents with distributed executor and router pools to autonomously determine task delegation through local reasoning. Symphony [289] establishes a highly scalable decentralized framework that facilitates collective intelligence by allowing direct peer-to-peer interactions without centralized coordination bottlenecks. To optimize such distributed Orchestration, MAAC approaches [290] empower agents to dynamically learn and refine their decentralized coordination policies via reinforcement learning rather than relying on static routing rules. Furthermore, distributed orchestration requires robust mechanisms for local information management. AHKG [291] supports distributed planning by allowing agents to synchronize localized information clusters while inherently maintaining alignment with global topological objectives.

Similar decentralized coordination strategies have been widely adopted across various domains to preserve agent autonomy. For instance, HIPPO-MAT [292] employs decentralized multi-agent reinforcement learning and graph neural networks for task allocation without a central controller, while AgentFlow [293] proposes a distributed coordination framework based on publish–subscribe communication and dynamic service elections.

Collectively, these approaches demonstrate how distributed orchestration enables scalable collaboration while preserving agent autonomy and system robustness.

3.3.3 Hybrid Orchestration Topology

Hybrid orchestration topology combines centralized strategic coordination with decentralized execution mechanisms. In this paradigm, decision authority is partially centralized at higher levels of the system while lower-level agents retain autonomy during task execution.

From the perspective of coordination topology, hybrid orchestration typically adopts hierarchical structures. Early frameworks such as ChatDev [13] and MetaGPT [246] laid the groundwork for this paradigm by relying on centralized Standard Operating Procedures (SOPs) to constrain and guide the decentralized execution of role-playing agents. Building upon more complex hierarchies, AgentOrchestra [294] organizes collaboration into three layers: a planning layer for task decomposition and global coordination, a functional orchestration layer for managing tools and specialized agents, and an execution layer for carrying out concrete operations. Similarly, planner–executor architectures [295] separate global reasoning from action execution. TOA [296] structures agents in a hierarchy where parent nodes provide centralized aggregation and reasoning, while leaf nodes maintain autonomy for parallel context processing. GoA [297] expands decentralized execution by enabling non-linear, localized communication among agents, while maintaining a globally predefined topological constraint to ensure centralized goal alignment. AdaptOrch [298] further advances hybrid orchestration by introducing a centralized, task-adaptive orchestrator that dynamically selects the execution topology.

These approaches illustrate how hybrid orchestration can balance centralized strategic control with decentralized operational flexibility.

3.4 Interaction

Interaction characterizes how multiple agents coordinate during the execution phase of a task. Unlike static system design, which focuses on agent roles or architectures, interaction emphasizes the dynamic processes through which agents exchange information, update intermediate states, and jointly drive task completion under the current collaboration policy π_{coll} .

3.4.1 Information Flow

Information flow governs the execution dynamics of multi-agent systems, dictating the orchestration of collaborative actions and the synchronization of shared states. Based on the temporal and logical dependencies of task execution, information flow can be primarily categorized into **sequential interaction** and **parallel interaction**.

Sequential Interaction In sequential interaction, agents exchange information in an ordered, step-by-step manner, where the output of one agent serves as the input for the next. This pattern is particularly suitable for tasks that can be decomposed into a sequence of dependent subtasks.

Recent studies have extensively leveraged sequential interaction to structure agentic workflows. Frameworks such as HuggingGPT [132] and EAG [299] establish modular pipelines in which tasks are decomposed into discrete reasoning stages, whereas Chain of Agents [300] employs a sequential relay to mitigate information loss in long-context tasks. To enhance adaptability, AnyMAC [301] introduces dynamic cascading via next-agent prediction within sequential flows, a pattern also applied to specialized domains like recommendation in MACRec [302]. Furthermore, frameworks such as MAgIC [303] and recent role-specialized pipelines [304] utilize these linear dependencies to provide systematic benchmarks and ensure traceability and accountability across multi-agent interactions.

The main advantage of sequential interaction lies in its interpretability and modularity. Each agent operates on well-defined inputs and produces traceable outputs, making the overall process easier to debug and optimize. However, this structure may suffer from error propagation, as mistakes in early stages can cascade through subsequent steps. Additionally, strict sequential dependencies may limit efficiency when tasks could otherwise be executed concurrently.

Parallel Interaction In parallel interaction, multiple agents operate simultaneously and independently on shared or decomposed tasks, followed by a mechanism for aggregating their outputs. Unlike sequential interaction, parallel interaction allows agents to explore diverse solution spaces concurrently, which can improve robustness and efficiency.

A foundational example of parallel interaction is the self-consistency decoding strategy [64], which generates multiple reasoning paths concurrently and aggregates the final answer. This principle was soon extended to multi-agent debate frameworks [305], enabling several agents to independently propose and critique solutions in parallel. Building upon these concepts, recent architectures further exploit parallelism to overcome sequential bottlenecks. MoA methodology [306] applies this at the model level by utilizing diverse LLMs as concurrent proposers and iteratively aggregating their outputs. At the execution level, the M1-Parallel framework [307] deploys multiple agent teams simultaneously to explore distinct solution paths, minimizing latency via early termination. To address real-time environment constraints, parallelized planning-acting architectures [308] decouple cognitive planning and physical execution into independent, concurrent threads. Furthermore, for complex problem-solving, AgentGroupChat-V2 [309] decomposes user queries into a task tree to assign sub-tasks to specialized agents operating in parallel. Similarly, DynTaskMAS [310] employs dynamic DAGs to adaptively manage and execute these parallel sub-tasks, ensuring high system robustness.

Parallel interaction offers significant advantages in terms of scalability and diversity. By enabling multiple agents to explore different hypotheses simultaneously, the system can reduce the risk of converging to suboptimal solutions. However, it also introduces challenges in result aggregation and consistency maintenance. Designing effective mechanisms to reconcile conflicting outputs remains a key research problem in this setting.

3.4.2 Interaction Patterns

Interaction patterns describe how collective outcomes emerge from actions of individual agent. Existing studies generally categorize multi-agent interaction patterns into two major types: **cooperative interactions** and **competitive interactions**, depending on whether agents pursue shared or conflicting objectives.

Cooperative Interaction Cooperative interaction refers to scenarios in which agents share a common objective and coordinate their actions to maximize collective performance. In such settings, agents typically exchange information, divide responsibilities, and synchronize decisions to improve overall task efficiency and solution quality. Recent studies have increasingly emphasized the role of structured coordination in enabling effective cooperation among agents. For instance, generative agent frameworks [110] simulate socially interactive agents, demonstrating how cooperation can emerge in complex environments. Similarly, role-based multi-agent systems such as MetaGPT [246] organize agents into specialized roles (e.g., product manager, engineer), where collaboration is achieved through well-defined workflows. Similarly, domain-specific architectures like the DrugAgent framework [311] utilize distinct planner and instructor roles to maintain precise workflow boundaries and avoid conflicts during complex scientific problem-solving.

For coordination settings where teammate behaviors are not known in advance, the ProAgent framework [312] facilitates zero-shot coordination by enabling agents to proactively infer teammate intentions and adjust their strategies based on observed behaviors. Exploring the structural design of these interactions, socially-inspired collaboration mechanisms [313] reveal that the sequential orchestration of interaction topologies can significantly enhance cooperative outcomes without merely relying on scaling agent numbers. In decentralized settings with partial observability, the COMPASS framework [314] advances cooperative planning by introducing a multi-hop communication protocol, allowing agents to effectively exchange localized state information and synthesize adaptive skills.

To further ensure the reliability of cooperative interactions, recent methods have integrated structured evaluation and collective decision protocols. COPPER [315] introduce system-level feedback loops, utilizing shared reflection mechanisms to resolve credit assignment issues and iteratively refine collective behaviors. Concurrently, electoral approaches [316] to collective decision-making employ ordinal preferential voting mechanisms to systematically aggregate diverse agent preferences, thereby mitigating the limitations of simple majority rules and enhancing the robustness of joint decisions.

Overall, cooperative interaction in modern multi-agent systems is characterized by structured communication, role-based organization, and iterative alignment, which together facilitate efficient and scalable collaboration.

Competitive Interaction Competitive interaction arises when agents pursue individual objectives that may partially or fully conflict with those of other agents. In this setting, agents attempt to advance their own objectives through strategic behaviours such as negotiation, confrontation, or persuasion.

Recent work has explored competitive interaction through mechanisms such as debate, negotiation, and strategic role-playing. For example, the debate framework proposed by Du et al. [305] enables multiple agents to argue over candidate solutions, where each agent attempts to expose weaknesses in the reasoning of others. Building on this, Agent4Debate [317] introduces a dynamic framework to test if LLM agents can surpass human strategic logic in structured linguistic confrontations. Negotiation-based settings provide another important perspective on competition. The GPT-Bargaining framework [318] studies how agents engage in bargaining processes, where each agent aims to optimize its own

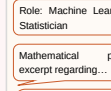
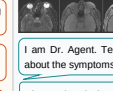
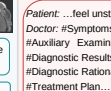
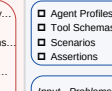
Benchmark	LLMArena	LLMArena	BattleAgentBench	MultiAgentBench	AgentClinic	MVME	MAC Benchmark	REALM-Bench
Tasks Components	✓ Agentic Coordination • Hanabi Challenge • Overcooked AI • Collab Capture • Collab Escape ✓ Coordination QA • Environment • Comprehension • Theory of Mind Reasoning • Joint Planning	✓ Gaming Environments • TicTacToe • ConnectFour • Texas Hold'em • Undercover • Bargain • Bid • Hanabi	✓ Foundational Agent Capabilities • Simple Navigation • Complex Navigation ✓ Paired Agent Interaction • Cooperative Task • Competitive Task ✓ Multi-Agent Dynamics • Static Setting • Dynamic Setting • Hybrid Setting	✓ Mutual Goal • Research • Minecraft • Database Analysis • Coding ✓ Conflicting Goals • Werewolf • Bargaining	✓ Medical Specialties • Emergency Medicine • Geriatrics • Internal Medicine • Psychiatry • Ophthalmology • Otolaryngology • Pediatrics • Gynecology • Orthopedics	✓ Medical Departments • Surgery • Internal Medicine • Obstetrics and Gynecology • Pediatrics • Otorhinolaryngology • Others	✓ Enterprise Application Domains • Travel planning • Mortgage financing • Software development	✓ P-series • Campus Tour • Multi-Group Campus Tours • Urban Ride-Sharing • URS with Disruptions ✓ J-series • JSSP Basic • JSSP Basic with Disruptions • JSSP Large-scale • JSSP Large-scale with Disruptions
Evaluation Samples	 Q: What is my partner's intention? A: pick up onion from o0. B: pick up plate from p0.	 You play X. The board status is (3,2), (3,3). You should output X and the position of the move.	 ...Task Description... #Game Instructions... #Thought process #Attack operation #Cooperation operation	 Role: Machine Learning Statistician Mathematical proof excerpt regarding... Strict "SQ" JSONL output defining problem, significance, obstacles, novelty, and methodology	 I am Dr. Agent. Tell me about the symptoms... I have a headache... Request Scan ... Diagnosis: Cavernous Sinus Thrombosis.	 Patient: ...feel unsteady... Doctor: #Symptoms... #Auxiliary Examinations... #Diagnostic Results... #Diagnostic Rationales... #Treatment Plan... Examiner: #Examination... Chief Physician: #Diagnosis Result... #Treatment Plan	 Input Problems: Hello! Could you help me check my credit report to verify my credit score?	 ... Total distance traveled = 87 km.
Metrics	✓ Game Success	✓ TrueSkill™ Score	✓ Forward Distance ✓ Format Accuracy ✓ Move Accuracy	✓ KPI ✓ Communication Score ✓ Planning Score ✓ Coordination Score	✓ Diagnostic Accuracy	✓ Quality of the summary report ✓ Entity-level: Precision / Recall / F1	✓ Goal Success Rate ✓ Latency metrics ✓ Routing metrics	✓ Success Rate ✓ Optimal Rate ✓ Execution Time ✓ Token Usage

Figure 5: Evaluation benchmarks for multi-agent collaboration. This figure compares existing benchmarks across task components, evaluation samples, and evaluation metrics.

outcome under conflicting interests. To further quantify these behaviours, ALYMPICS [319] provides a game-theoretic playground to observe how agents navigate classic dilemmas like the Tragedy of the Commons, while RTBAgent [320] pushes competitive tactics into high-stakes real-time bidding auctions. In addition, role-playing approaches [321] introduce diverse agent roles to encourage strategic diversity. Recent architectures leverage competition as an optimization tool. Multi-agent KTO [322] reinforces strategic interactions by aligning agent preferences through game-theoretic objectives. To ensure these competitive processes remain robust, ALIGN [323] introduces performance guarantees to regulate delegated competitive reasoning.

Overall, competitive interaction emphasizes strategic adaptation, conflict resolution, and dynamic response to opponents. Unlike cooperative settings that rely on alignment and shared objectives, competitive scenarios drive agents to refine their strategies through adversarial exchanges continuously.

3.5 Evaluation

A critical aspect of optimizing MAS lies in the construction of high-quality evaluation datasets [324, 325] that capture realistic interaction trajectories τ and reflect diverse collaboration patterns.

As shown in Figure 5, recent literature has increasingly shifted toward evaluating dynamic MAS. As foundational efforts, LLM-Coordination [326] establishes an early benchmark for analyzing how multiple large language models share information and collaborate to achieve shared objectives. Subsequent benchmarks have introduced heightened environmental complexity and complex interactive dynamics. LLMArena [327] assesses agents' adaptability and sequential decision-making in dynamic environments with incomplete information. Pushing beyond pure collaboration, BattleAgentBench [328] provides a comprehensive framework to evaluate both cooperative and competitive capabilities, offering quantifiable metrics for agents operating in zero-sum and non-zero-sum game scenarios. Furthermore, MultiAgentBench [329] evaluates orchestration quality through milestone-based KPIs and explicitly investigates the efficacy of different communication topologies (e.g., graph-based structures) on collaborative efficiency.

Pushing further toward realism, recent work explores high-fidelity, domain-specific evaluation settings that demand rigorous multi-round coordination. Medical testbeds like AgentClinic [330] and MVME [331] serve as advanced environments for testing sequential decision-making, information gathering, and dispute resolution mechanisms under high uncertainty. Enterprise-oriented scenarios have been studied

to highlight end-to-end task completion and coordination efficiency in real-world applications [332]. REALM-Bench [333] targets real-world planning tasks requiring long-horizon reasoning and multi-agent coordination.

Overall, trajectory-level benchmarks better reflect the interactive and collaborative nature of MAS. Such datasets enable both accurate evaluation and informed refinement of agent behaviours.

3.6 Discussion

Table 7: Comparison of multi-agent frameworks, organized by collaboration design.

Framework	#Agent	Role Cap.	Role Alloc.	Orchestration	Exec. Seq.	Interact.	Opt.	Year
Generative Agent [110]	25	Hom	Static	Distributed	Par/Seq	Coop	–	2023
CAMEL [210]	2	Hom	Static	Distributed	Seq	Coop	–	2023
GPT-Bargaining [318]	3	Het	Static	Centralized	Seq	Comp	✓	2023
MetaGPT [246]	unlimited	Het	Static	Hybrid	Seq	Coop	–	2023
AutoGen [23]	unlimited	Het	Dynamic	Hybrid	Par/Seq	Coop	–	2024
AgentVerse [237]	unlimited	Het	Dynamic	Hybrid	Par/Seq	Coop	–	2024
AutoAgents [234]	unlimited	Het	Dynamic	Hybrid	Seq	Coop	✓	2024
MAPoRL [24]	unlimited	Het	Static	Hybrid	Seq	Coop	✓	2025
OSC [334]	unlimited	Het	Dynamic	Distributed	Seq	Coop	✓	2025
Puppeteer [285]	unlimited	Het	Dynamic	Centralized	Par/Seq	Coop	✓	2025
AgentForge [335]	unlimited	Het	Dynamic	Centralized	Seq	Coop	✓	2026
MASFactory [336]	unlimited	Het	Dynamic	Hybrid	Seq	Coop	✓	2026

Role Cap.: functional diversity — Hom = homogeneous; Het = heterogeneous. *Role Alloc.*: role assignment — Static = fixed; Dynamic = runtime. *Orchestration*: coordination structure — Distributed peer-to-peer; Centralized single orchestrator; Hybrid mixed. *Exec. Seq.*: temporal pattern — Par = parallel; Seq = sequential; Par/Seq = both. *Interact.*: social dynamics — Coop cooperative; Comp competitive. *Opt.*: includes an optimization loop.

Table 7 summarises and compares various multi-agent frameworks from the perspective of collaboration design, highlighting not only the structural diversity of MASs, but also a shift in the design philosophy of LLM-based collaboration.

From homogeneous agents to functional specialization. One of the clearest trends is the increasing prevalence of heterogeneous role configurations. Early frameworks [110, 210] with homogeneous agents are conceptually simple and easier to scale, but struggle with complex tasks as it tends to get trapped in repeated self-consistency loops [207]. By contrast, heterogeneous MASs [13, 246] introduce functional specialization, such as planning, coding, reviewing, or evaluation, thereby enabling division of labour and more structured cooperation. However, heterogeneity also introduces new challenges. As role diversity increases, so does interdependence among agents, making the system more sensitive to interface mismatches, cascading errors, and coordination overhead. In practice, a specialized agent is only useful if its outputs are both reliable and legible to downstream agents. Therefore, future MAS research should focus not only on creating more roles but also on defining principled role boundaries and mechanisms for capability calibration across agents.

Static versus dynamic role allocation. Static role assignment [13, 246] is easier to design and interpret and remains effective for tasks with predictable decomposition structures. However, it assumes that task requirements are known in advance and remain stable throughout execution. This assumption becomes increasingly unrealistic in more realistic open-ended tasks, where intermediate results often reshape the problem itself. Dynamic role allocation [234, 237] offers a more flexible alternative by allowing

the system to instantiate, reassign, or retire roles at runtime according to task state, uncertainty, or resource constraints. This makes MASs better suited for non-stationary environments and long-horizon problems. Nevertheless, many current systems implement dynamic roles heuristically, but do not yet learn when specialization is necessary, when redundancy is beneficial, or when additional agents simply add noise.

Explicit communication dominates, but may not scale indefinitely. Explicit communication remains the dominant mode across current LLM-based MASs. Explicit messages are transparent, inspectable, and well aligned with the text-based interfaces of LLMs. They make execution trajectory easier to debug and allow designers to impose structured debate or critique patterns. As the number of agents and interaction rounds grows, explicit communication quickly becomes expensive, verbose, and cognitively inefficient. Repeated broadcasting of long textual states creates token overhead, increases latency, and can amplify inconsistency when agents interpret shared context differently. More importantly, explicit communication [263] often exposes raw intermediate reasoning rather than the task-relevant state, which limits scalability. For this reason, implicit communication deserves much more attention in future LLM-based MASs. In embodied MASs [337, 338, 339], implicit coordination often emerges through shared memory, environment state, stigmergic signals, or policy-conditioned behaviour. Translating such ideas into LLM-based settings may enable more efficient coordination through latent state sharing or memory traces rather than full natural-language exchange. The central challenge is to preserve interpretability while reducing communication cost.

Orchestration topology reflects a trade-off between control and flexibility. The comparison in Table 7 suggests that no orchestration topology is universally superior. Centralized orchestration [284, 285] offers strong global oversight, clear responsibility assignment, and easier enforcement of task structure. However, centralized control can create bottlenecks, since a single coordinator must handle all information aggregation and decision-making, resulting in constrained parallelism and potential overload in both computation and communication. By contrast, distributed orchestration [282, 289] improves autonomy and robustness to single-point failure, and can better support peer collaboration. But it also makes consensus formation, conflict resolution, and global coherence substantially harder. Hybrid orchestration [294, 298] has therefore become increasingly common, as it attempts to combine top-down structure with localized autonomy. This trend indicates that orchestration should be treated as an adaptive design variable rather than a fixed architectural choice. A key problem is topology selection under task conditions. Different problem types likely favour different coordination graphs: tightly coupled reasoning may require stronger centralization, whereas exploratory or modular tasks may benefit from decentralization. Future work should move beyond categorical labels such as centralized or distributed, and instead model orchestration as a dynamic topology that can evolve with task complexity, uncertainty, and agent performance.

Sequential and parallel execution should be viewed as complementary paradigms. Temporal organization is becoming an increasingly important part of MAS design. Sequential execution [210, 246] is suitable for tasks that require refinement, verification, or strict stage ordering. In contrast, parallel execution [23, 237] is valuable for candidate generation, diversified search, or decomposable subtasks, where breadth and speed are more important than immediate consistency. The key insight is that the execution sequence strongly shapes the epistemic behaviour of the system. Sequential pipelines encourage coherence but risk error propagation, since early mistakes constrain downstream reasoning. Parallel pipelines improve diversity and robustness but may create expensive aggregation problems when candidate outputs conflict. The most capable future MASs will likely combine both modes

adaptively.

Cooperation dominates, while strategic competition remains underexplored. Most existing MAS frameworks [311, 312] are cooperative, which aligns naturally with benchmark design and application goals centred on task completion. However, cooperation settings neglect scenarios where disagreement or negotiation is productive rather than harmful. Competitive interactions [317, 318] enhance collective reasoning quality through adversarial review, structured debate, and bargaining. The challenge is that without carefully designed incentives and adjudication mechanisms, competitive interactions may degenerate into unproductive contradiction or reward gaming. Future MASs should therefore explore richer interaction regimes and dynamically shifting incentives. This would make MASs more suitable for complex real-world tasks such as policy analysis, scientific criticism, market simulation, and risk assessment.

Taken together, these observations indicate that MASs are gradually evolving from static, rule-based coordination toward dynamic, policy-driven collaboration with increasing adaptability. However, this trend exposes several unresolved challenges. First, scalability remains a major bottleneck. Increasing the number of agents does not guarantee better performance [340]. Second, reliability becomes more difficult in multi-agent settings, because errors may emerge not only from individual agents but from their interactions, including misinformation propagation, collaborative failure modes, and unstable feedback loops. Third, cost is likely to become central: practical MAS deployment requires balancing performance gains against token consumption, latency, and infrastructure complexity. These challenges imply that future MAS research should prioritize three directions. The first is adaptive coordination, where roles, topology, and communication strategies are selected online rather than fixed a priori. The second is collaboration-efficient design, which seeks better performance with fewer messages, fewer agents, and more compact shared representations. The third is trustworthy multi-agent reasoning, including interpretable coordination traces, fault localization, and robust evaluation protocols.

4 Multi-Agent System Failure Attribution

In recent years, although multi-agent systems can improve complex task-solving performance through collaboration among specialized agents, their highly coupled collaborative processes also amplify system vulnerability, making errors more likely to arise during task decomposition, inter-agent communication, and coordinated decision-making [20, 341]. More importantly, such errors often do not manifest at the moment they are introduced, but instead propagate, accumulate, or even evolve through subsequent interactions before ultimately surfacing as system-level failures. As a result, anomaly attribution in multi-agent systems becomes substantially more challenging.

4.1 From Collaboration to Diagnosis

A fundamental challenge arises from the tight coupling of multi-agent interactions, where initial localized errors such as hallucinations frequently propagate through the system, culminating in intricate cascading failures that obscure the primary root cause [20, 342]. As execution trajectories grow longer and contain increasing amounts of interactive information, manually locating agent-level faults and their associated time steps becomes highly inefficient and unscalable [343, 344]. Therefore, an effective automatic failure attribution mechanism plays a key role in clarifying complex interaction processes in MAS and enabling fine-grained, targeted feedback to support directed improvement. It is also an important foundation for enhancing system reliability and trustworthiness [341, 345, 346].

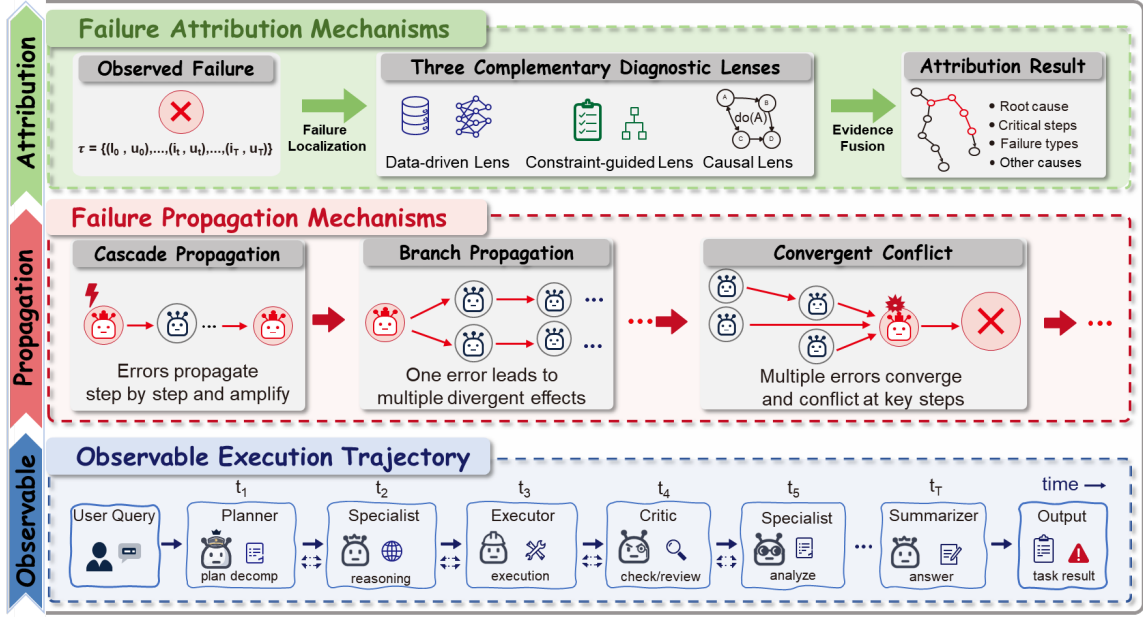


Figure 6: Mechanism of Failure Attribution in Multi-Agent Systems.

Failure attribution in MAS is the process that uses the complete execution trajectory as its basis to automatically identify the responsible agents and critical steps when execution failures or anomalous behaviors emerge [343, 345, 347]. Additionally, it analyzes the failure’s category, etiology, and propagation mechanism within the entire collaborative workflow to articulate who introduced the failure, where it originated, and why it ultimately precipitated system failure [341].

A multi-agent execution trajectory τ refers to the time-ordered dynamic sequence formed during task execution in a MAS, under a given user query, initial environment state, and system configuration [346]. It consists of the contextual information, state transitions, communication interactions, tool usage, decision-making actions, and environmental feedback of the agents. Such a trajectory captures the complete execution process from the initial configuration to the terminal state, and serves as the fundamental basis for analyzing collaborative behaviors, failure propagation paths, and root causes of failure in MAS [346, 347, 348]. It can be formally defined as

$$\tau = ((i_0, u_0), \dots, (i_t, u_t), \dots, (i_H, u_H)), \quad (12)$$

where H denotes the trajectory horizon, and (i_t, u_t) denotes the active agent and its corresponding action at time step t .

4.1.1 Formal Expression of the Attribution Process

To define the condition for failure occurrence, a trajectory evaluation function $Z(\tau)$ is introduced. When $Z(\tau) = 1$, it indicates that the final result state of the trajectory τ is evaluated as anomalous. Then, given a system configuration Ω , a user query q , and the complete execution trajectory τ , the failure attribution model f identifies the agent I responsible for the failure and the specific time step t at which the failure is introduced. The entire multi-agent failure attribution process can be formally expressed as the following piecewise function:

$$(I, t) = \begin{cases} f(\Omega, \tau, q), & \text{if } Z(\tau) = 1, \\ \emptyset, & \text{otherwise.} \end{cases} \quad (13)$$

The above equation indicates that only when the system trajectory is judged to be anomalous (i.e., $Z(\tau) = 1$), the attribution model f will take the system configuration M , trajectory τ , and user query q as input and return the failure-responsible agent and the corresponding decisive time step (denoted as the tuple (I, t)); if the system operates normally (otherwise), it outputs the empty set $\emptyset$. This pairwise formulation follows prior agent-step attribution settings, but it can be naturally extended to multi-agent, multi-step, or causal-chain attribution when failures involve multiple interacting causes.

4.2 Failure Taxonomy

With the growing adoption of MAS in complex tasks, researchers have increasingly recognized that failure analysis should go beyond identifying which agent or step is responsible. It must also address a more fundamental question: what type of failure has occurred and what underlying mechanism triggered it? In this sense, failure classification provides a structured descriptive framework for subsequent attribution, diagnosis, and repair [341, 345]. Existing studies have approached this problem from three broad perspectives: system structure, execution stages, and causal lifecycle.

4.2.1 System Structure

System structure classification focuses on collaboration relationships, emphasizing the identification of where the collaborative chain breaks down. Pan et al. [20] organize failures into specification and system design failures, inter-agent misalignment, and task verification/termination failures, thereby suggesting that many MAS failures can be understood as breakdowns in system design, inter-agent coordination, or verification mechanisms. AgentAsk [342] further refines the classification unit to inter-agent information handoff and summarizes anomalies into four types of communication failures: Data Gap, Referential Drift, Signal Corruption, and Capability Gap. AEGIS [349] further transforms failure categories into controllable targets for error injection, supporting scalable generation of diagnostic data with verifiable error-identification labels.

4.2.2 Execution Stages

Execution process classification organizes failure types according to where they arise within the internal task workflow. This form of categorization may be instantiated either at the level of functional modules or at the level of execution stages. TRAIL [28] classifies agentic issues into three categories—reasoning errors, system execution errors, and planning/coordination errors—thereby placing failures in reasoning, tool invocation, and process control within a unified framework. Zhu et al. [350] adopt a modular agent architecture and attribute failures to module-level errors involving memory, reflection, planning, action, and system-level operations. Lu et al. [351] describe autonomous agent systems as a planner–generator–executor pipeline and categorize anomalies according to these three stages: task planning, task execution, and response generation. Overall, these methods interpret anomalies as instabilities arising at different points within the task execution chain, differing primarily in whether the analytical granularity is defined in terms of functional components or temporal stages.

4.2.3 Causal Lifecycle

Causal lifecycle classification extends failure categorization from surface-level symptoms to root-cause hierarchies and propagation processes. AgentFail [341] categorizes failure root causes at three levels: agent, workflow, and platform, covering errors from local agent execution to platform-level orchestration. FailCycle [352] further points out that the point at which a failure originates often does not coincide with the point at which it becomes visible, thereby indicating that anomalies in MAS exhibit pronounced propagative and evolutionary properties. From the perspective of behavioral abstraction, AGENTSCOPE [353] further characterizes anomalies as behavioral patterns such as invalid

context, instruction following, faulty verification, incoherent planning, and reward hacking, while integrating the classification process with structured behavioral analysis.

Overall, existing research remains substantially limited in both generalizability and granularity. Although preliminary taxonomies have been developed from diverse perspectives, the absence of a widely accepted unified framework constrains their transferability and undermines cross-study benchmarking and knowledge reuse. At the same time, most existing attribution methods still rely on a simplified single-cause–single-effect assumption [343, 346, 354], making them insufficient for capturing the multi-cause–multi-effect, coupled, and cascading nature of anomalies in real-world systems. Therefore, future research should move toward a unified cross-system framework while incorporating mechanisms such as composite labels and cross-layer associations. This would enhance cross-domain transferability and enable failure taxonomies to more precisely characterize the deeper logic by which multi-agent anomalies evolve from local triggers into global system dynamics.

4.3 Failure Attribution Methods

MAS can improve the handling of complex tasks, yet their multi-agent coordination and long-horizon interactions also introduce new failure surfaces and cascading risks [20, 342, 346]. In this context, multi-agent failure attribution has gradually moved from engineering experience to systematic research topics [343, 345, 346, 355]. Its core is no longer just ‘whether the task is successful’, but to answer more operable diagnostic questions: who (which agent) introduced the decisive error at what step / when, and how the error spread into the final failure [343, 346, 347, 355, 356]. Around this problem, the existing research has formed three representative technical routes, which are the Data-driven method, the Constraint-guided method, and the causal-inference method. These routes are not mutually exclusive but reflect the dominant mechanism used for attribution.

4.3.1 Data-driven Method

Data-driven methods fundamentally reframe failure attribution in MAS from the manual inspection of raw logs to the learning, representation, and generalization of recurring anomalous patterns encoded in failure trajectories. Early approaches primarily relied on human analysis or on LLM directly processing long trajectories for responsibility localization [343]. More recent research has progressively shifted toward dedicated attribution models, structured trajectory representations, and lightweight online diagnostic modules [346, 349]. As a result, the central challenge in this line of work is how to distill long-horizon, tightly coupled failure processes that propagate across agents into attribution representations that are learnable, transferable, and reusable [20, 357, 371].

From the perspective of methodological evolution, data-driven attribution began with the explicit formalization of failure localization and the direct application of models to execution trajectories for attribution. Who&When [343] is representative of this early paradigm. It formulates attribution as the automatic identification of both the responsible agent and the critical point at which the error is introduced, while comparing the effectiveness of alternative trajectory search and prompting strategies. In the same vein, both TRAIL [28] and TraceElephant [344] show that general-purpose long-context models remain limited in debugging complex execution trajectories and that incomplete observability further undermines attribution quality. Collectively, these findings shifted subsequent research away from treating generic large language models as log readers and toward the development of more learnable trajectory representations and specialized attribution models.

In the existing literature, the most representative data-driven paradigm is the use of large-scale labeled data to train dedicated attribution models. AGENTRACER [346] follows this route by generating

Table 8: Comparison of multi-agent failure attribution methods (Section 4).

Method	Granularity	Data Type	Data Processing	Paradigm	Timing	Repair
<i>A. Data-Driven Methods</i>						
AGENTRACER [346]	Agent/Step	SL+FL	Automatic	RL	Offline	✓
Spectrum [348]	Agent/Step	FL	Automatic	SA	Offline	–
CORRECT [357]	Agent/Step	SL	Semi-auto	KD	Both	–
AGENTASK [342]	Step/Category	FL	Semi-auto	SFT+RL	Online	✓
AEGIS [349]	Agent/Category	SL	Automatic	SFT+RL+CL	Offline	–
GraphTracer [356]	Agent/Path	SL+FL	Automatic	RL	Offline	✓
MASC [358]	Step	SL	–	Self-Corr	Online	✓
Traj-Guard [359]	Trajectory	SL+FL	Semi-auto	SFT	Online	–
ProMAS [360]	Agent/Step	Public	–	CL	Online	–
<i>B. Constraint-guided Methods</i>						
ABDUCT [355]	Agent/Step	Public	–	LLM-As-Judge	Offline	–
AgentErrorBench [350]	Step/Category	FL	Manual	LLM-As-Judge	Offline	✓
Role-Trace [304]	Stage	SL+FL	Automatic	Stage-comparison	Offline	✓
ECHO [361]	Agent/Step	Public	–	LLM-As-Judge	Offline	–
DOVER [362]	Agent/Step	FL	Automatic	LLM-As-Judge	Offline	✓
AgentRx [363]	Step/Category	FL	Manual	LLM-As-Judge	Offline	–
XAgen [364]	Agent/Step/WF	FL	Semi-auto	LLM-As-Judge	Offline	✓
SDBL [365]	Step	Public	–	LLM-As-Judge	Offline	–
<i>C. Causal-inference Methods</i>						
MARL [366]	Agent/Step	SL+FL	Automatic	Counterfactual	Offline	✓
InterDebug [367]	Agent/Step	FL	Semi-auto	HITL	Offline	✓
CDC-MAS [368]	Agent/Step	Public	–	Causal-Disc	Offline	✓
Specific Effects [369]	Agent/Step	FL	Automatic	Cf-ASE	Offline	–
AgentFail [341]	Agent/Category	FL	Manual	LLM-As-Judge	Offline	–
TRAJECTORY [370]	Step	FL	Semi-auto	PreAct	Online	✓
FailCycle [352]	Agent/Step	FL	Manual	Counterfactual	Offline	✓
DiLLS [371]	Plan/Step	FL	Manual	HITL	Offline	–
MMDP-SCM [372]	Agent/Step/State	SL+FL	Automatic	TCFE	Offline	–
AGENTSOCPE [353]	Step/Category	SL	Automatic	Causal-Graph	Both	–
RAFFLES [373]	Agent/Step	Public	–	LLM-As-Judge	Offline	–
CHIEF [347]	Agent/Step	Public	–	Graph-based	Offline	–

Granularity: attribution resolution — WF = workflow. *Data Type:* SL = success log; FL = failure log; Public = public benchmark dataset. *Data Processing:* Manual = human annotation; Semi-auto = hybrid human-automatic; Automatic = fully automated. *Paradigm:* attribution approach — LLM-As-Judge LLM-as-judge; RL reinforcement learning; KD knowledge distillation; SFT supervised fine-tuning; CL contrastive learning; SA spectral analysis; Self-Corr metacognitive self-correction; PreAct pre-acting diagnosis; HITL human-in-the-loop; Stage-comparison stage comparison; Counterfactual counterfactual reasoning; Cf-ASE counterfactual agent-specific effect; TCFE total counterfactual effect; Causal-Disc causal discovery; Causal-Graph causal graph analysis; Graph-based graph-based analysis. Combinations (e.g. SFT+RL) are written inline. *Timing:* Offline offline; Online online; Both both. *Repair:* ✓ = supports downstream repair.

attribution-labeled failure trajectories through counterfactual replay and programmatic error injection, and by further training a lightweight tracer, thereby shifting attribution from generic reasoning to specialized model learning. Aegis [349] systematizes this idea further by using context-aware error injection to construct supervisory signals and by supporting three training paradigms: supervised fine-tuning, reinforcement learning, and contrastive learning. Although CORRECT [357] does not itself rely on retraining, its method also relies on the abstraction of real failure distributions and the reuse of structured error signals. This suggests that the core of data-driven methods lies in transforming failure phenomena that are scarce, fragmented, and difficult to reproduce into supervisory signals that can be learned robustly.

At the modeling level, existing research has shifted markedly from raw textual interpretation to structured attribution representations. Spectrum-analysis-based methods [348] draw on software fault localization by replacing direct semantic judgments over individual logs with cross-trajectory coverage

statistics and suspiciousness scoring, thereby grounding attribution in the statistical differences observed across repeated executions. CORRECT [357] proposes a more lightweight alternative based on the assumption that failures in MAS exhibit recurring structural patterns. Instead of reanalyzing the full trajectory from scratch for each case, it performs pattern transfer and targeted localization at inference time through an online cache of distilled error schemas. GraphTracer [356] further shows that analyzing action chains solely in temporal order is insufficient to recover the true pathways of error propagation, and therefore models cross-agent information flow through an information dependency graph.

Beyond strict post hoc attribution, recent data-driven failure analysis has also incorporated online detection, intervention, and human-in-the-loop diagnostic support. AgentAsk [342] treats failure propagation as a localized risk arising in message interactions and learns when to initiate clarification, which agent to query, and how to formulate the minimal necessary clarification, thereby enabling intervention before errors spread. Trajectory Graph Copilot [370] advances this direction by constructing probabilistic graphs from historical trajectories and leveraging graph neural networks to issue risk alerts before actions are executed. Trajectory Guard [359] further adopts a sequence-aware autoencoding framework for real-time failure detection, with an emphasis on low-latency identification of implausible trajectories. Although DiLLS [371] is more oriented toward interactive diagnosis, its core principle is similar: multi-agent behaviors are compressed into hierarchical representations that are queryable and comparable, thereby improving developers' efficiency in locating and understanding failures. Collectively, these studies show that the value of data-driven methods is no longer confined to offline responsibility attribution, but is increasingly shifting toward continuous monitoring of failure propagation, preemptive interruption, and human-in-the-loop analysis.

Overall, data-driven research on failure attribution in MAS has progressed from prompt-based direct localization to a stage of systematic modeling built on automatic supervision, structured representations, and dedicated attribution models. Current evidence suggests that stable attribution cannot be achieved reliably by relying solely on general-purpose models to read long trajectories. A more effective direction is to organize attribution signals around the reuse of failure patterns, cross-trajectory structural modeling, and mechanisms for online intervention.

4.3.2 Constraint-Guided Method

Constraint-guided methods formulate failure attribution as an explicit diagnostic pipeline, rather than requiring a model to infer responsibility directly from a complete execution trajectory [355, 365]. Specifically, they decompose the attribution process into a series of executable and verifiable subtasks through stepwise decomposition, scope delimitation, error taxonomy design, responsibility boundary specification, and result validation [304, 362]. Existing studies indicate that these approaches are primarily oriented toward two objectives. First, by imposing structured diagnostic procedures, they constrain the attribution search space and thus reduce the reasoning burden induced by long-horizon trajectories and complex multi-role interactions. Second, by grounding attribution in explicit evidentiary criteria and validation mechanisms, they improve the interpretability and auditability of root-cause identification.

One important line of research explores how constraint-based designs can restructure the search space of failure attribution. To cope with the long execution trajectories, complex error propagation chains, and unstable global localization commonly observed in MAS, existing studies have generally adopted staged diagnostic frameworks. Specifically, Scope Delineation Before Localization (SDBL) [365] formulates attribution as a two-stage process in which the failure scope is first delimited and the critical step is subsequently localized, thereby reducing interference from irrelevant context. Pan et al. [20] ground their attribution in hierarchical evidence extraction based on the systematic identification and analysis

of fault modes, thereby underscoring the urgent need for a standardized diagnostic process. A2P [355] further constrains the attribution procedure into three sequential stages: causal hypothesis generation, minimal correction, and outcome prediction. Rather than relying solely on temporal proximity, A2P uses counterfactual correction and outcome prediction to assess whether a candidate step is causally decisive. The aforementioned studies first narrow the attribution space through predefined rules and then organize the reasoning procedure accordingly, thereby transforming long-horizon and open-ended failure localization into a constrained local diagnostic task.

Another important line of research focuses on how rules can be used to define root causes and validate attribution conclusions. Zhu et al. [350] decompose failures through a modular error taxonomy into critical errors associated with different capability components and further emphasize the need to isolate root-cause failures that may propagate through subsequent decisions. DoVer [362] argues that log-based localization should be treated as an unverified hypothesis and introduces targeted intervention, replay, and outcome comparison to validate or refute candidate failure hypotheses. In a similar vein, Role-Trace [304] improves the traceability of error sources and their cross-role propagation paths through structured handoff mechanisms and the preservation of intermediate records. AgentRx [363] grounds attribution in explicit evidence by automatically generating constraints, checking violations in a stepwise manner, and producing auditable logs. Although XAgen [364] is primarily an explainability and debugging tool, its failure identification process also incorporates predefined objectives, structured evaluation criteria, and a standardized assessment workflow. The aforementioned studies move beyond simple responsibility localization and place greater emphasis on the evidentiary grounding and verifiability of root-cause determination.

Overall, Constraint-guided methods have evolved from early heuristic error localization into structured attribution frameworks that integrate scope delimitation, hierarchical analysis, and intervention-based validation. Their key advantage lies in the ability to explicitly regulate the attribution process and to ground root-cause judgments in evidence that is transparent and verifiable. However, existing methods still depend heavily on task-specific priors, manually designed procedural constraints, and domain-dependent adjudication criteria, which limit the generalizability and adaptability of the underlying rules in turn.

4.3.3 Causal-inference Method

Causal-inference-based research on multi-agent failure attribution aims to identify the causal relationships within execution trajectories that induce both the onset and propagation of failures, rather than merely localizing entities whose behaviors co-occur with the final failure outcome [347, 368, 369]. Specifically, it seeks to determine—through counterfactual replay, intervention analysis, and related causal techniques—which critical behaviors, interaction dependencies, or state transitions genuinely trigger anomalies. Furthermore, it characterizes the mechanisms through which such anomalies propagate along collaborative interaction chains and through the evolution of the environment [368, 372].

The first line of research provides a theoretical foundation for causal attribution in multi-agent sequential decision-making, which later informs causal reasoning about LLM-based MAS failures. Agent-Specific Effects [369] shows that, in multi-agent Markov decision processes, the impact of an individual agent’s action on the outcome is often not localized, but instead propagates through the subsequent responses of other agents. Consequently, attribution should not be reduced to a simple mapping between a local action and an observed outcome; rather, it should capture the causal influence of an action as it unfolds through inter-agent propagation. The study further discusses how such propagated effects may be identified and estimated under counterfactual settings. Extending this view, Triantafyllou et al. [372] decomposes the total counterfactual effect of an action into two components: one transmitted through

the downstream behaviors of other agents, and the other through environmental state transitions. This decomposition enables disentangling the respective contributions of collaborative interaction chains and environmental dynamics within a unified causal framework. In contrast, Chen et al. [366] focus primarily on individual importance estimation. By leveraging action randomization and masking mechanisms, it quantifies the extent to which a single agent influences variations in team reward, thereby identifying critical agents. However, its emphasis remains on explaining individual contributions, rather than modeling the complete propagation chain through which failures emerge and spread.

The second line of work extends causal inference into the real execution trajectories of LLM-based MAS. Its primary concern is no longer the estimation of causal effects over a predefined structure, but the recovery of attribution-relevant causal relationships from complex execution logs. Ma et al. [368] explicitly propose a multi-granularity causal attribution framework, in which responsible agents are localized through performance causal inversion and Shapley value analysis, while critical failure steps are identified through causal discovery methods. Wang et al. [347] further argue that treating execution logs as flat sequences obscures the true causal chain; accordingly, it reconstructs trajectories into hierarchical causal graphs and distinguishes genuine root causes from propagated symptoms through hierarchical backtracking and progressive causal screening. AgentTrace [374] follows a similar rationale by recovering, from execution logs, a causal graph composed of sequential, communication, and data dependencies. It then traces backward from the point at which an error becomes manifest to candidate causes and ranks them according to structural and positional features, thereby supporting lightweight post-hoc root-cause analysis.

Beyond strict causal inference, several causal-adjacent studies improve the analyzability of long trajectories through structured representations, interactive interventions, and graph-based diagnosis. Bi et al. [353] transform long trajectories into structured representations through behavioral abstractions and neural invariants, whereas DiLLS [371] further organizes system behavior into a three-level abstraction consisting of activities, actions, and operations. In both cases, raw logs are compressed into intermediate representations that are more conducive to failure-action localization and root-cause analysis. Moving beyond representation alone, Ma et al. [352] extend the analysis to the propagation process, examining how anomalies evolve across nodes through natural-language interactions, tool invocations, and control logic in platform-orchestrated workflows. Zheng et al. [370] shifts diagnosis to the pre-action stage by modeling historical trajectories as probabilistic graphs and identifying high-risk action patterns. In parallel, Epperson et al. [367] and Zhu et al. [373] support the progressive verification of critical fault points through interactive message editing and reset operations, and through iterative reasoning-based investigation, respectively. Finally, Le et al. [375] demonstrate that causal graphs can be constructed by combining collaborative reasoning, code execution, and statistical methods, thereby offering methodological insight into how attribution can be grounded in the identification of those behaviors and interactions that genuinely drive failure emergence in complex agent trajectories. Collectively, these studies complement causal attribution by providing structured diagnostic abstractions and verification mechanisms.

Overall, causal-inference-based research on multi-agent failure attribution has developed into an increasingly coherent line of inquiry, encompassing both the formal characterization of causal effects and the recovery of causal relationships from execution logs for failure analysis. Compared with approaches that localize responsibility primarily through correlation, this line of work places greater emphasis on explaining why anomalies emerge, how they propagate, and which factors along the causal chain genuinely drive the eventual failure.

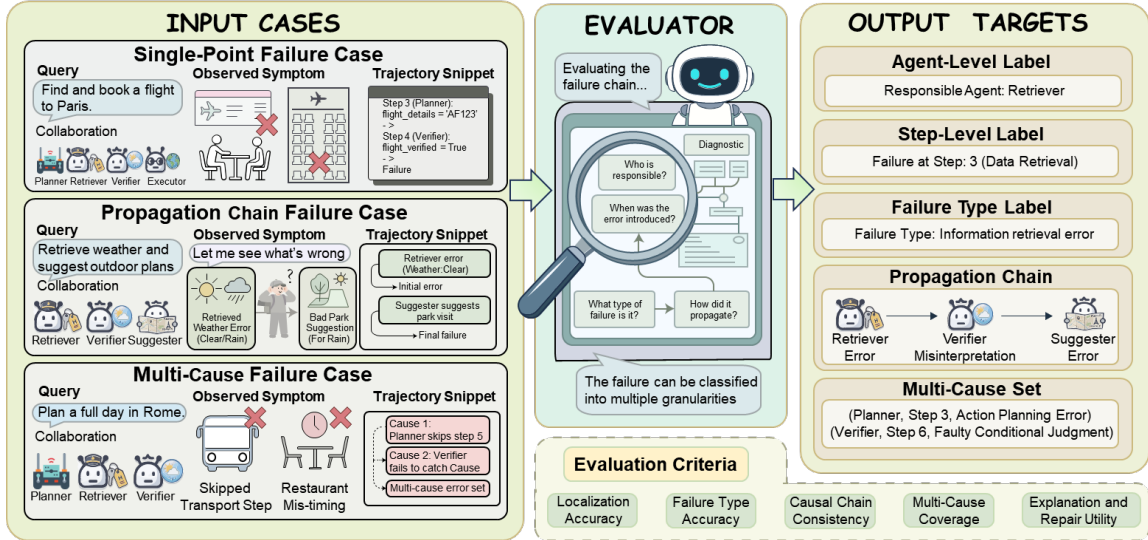


Figure 7: Scenario illustration of anomaly attribution evaluation in multi-agent systems. It presents three input case types—single-point, propagation-chain, and multi-cause failures—together with the corresponding multi-granularity attribution targets and evaluation criteria.

4.4 Failure Attribution Evaluation

As shown in Table 9, research on datasets for failure attribution in MAS has evolved from the early practice of manually annotating failure logs into a more systematic line of inquiry centered on data realism, annotation granularity, process observability, and evaluation validity. This evolution is reflected in three major trends. First, dataset construction has progressed from small-scale manual curation to automated large-scale generation [28, 343, 346, 349]. Second, annotation targets have expanded beyond responsible agents and critical steps to encompass multiple attribution dimensions, including error types, interaction edges, functional modules, and root causes in platform-orchestrated workflows [28, 341, 342, 350]. Third, evaluation has moved beyond single-metric localization accuracy toward composite frameworks that assess attribution performance, explanation quality, and process faithfulness in a unified manner [344, 345].

Table 9: Comparison of multi-agent failure attribution datasets.

Dataset	Focus	Size	Annotation	Granularity	Data Source	Metrics	Open
Who&When [343]	Localization	184	Manual	Agent+Step+Interp.	FL	Accuracy	✓
TRAIL [28]	Category	148	Manual	Step+Type	FL	Composite	✓
AgentFail [341]	Category	307	Manual	Agent+Category	FL	Accuracy	–
AgentErrorBench [350]	Category	200	Manual	Category+Feedback	FL	Accuracy	✓
CORRECT-Error [357]	Localization	2000+	Synthetic	Step	SL	Accuracy	–
AgentErrata [353]	Category	210	Synthetic	Agent+Category	SL	Accuracy	–
MP-Bench [345]	Localization	289	Manual	Multi-Step	FL	Composite	–
TraceElephant [344]	Localization	220	Manual	Agent+Step	FL	Accuracy	✓

Focus: attribution objective — Localization localization-oriented; Category category-level. *Annotation:* how ground-truth labels are obtained — Manual = human annotation; Synthetic = programmatic error injection. *Granularity:* resolution of attribution labels — Interp. = interpretation. *Data Source:* FL = failure log; SL = success log. *Metrics:* Accuracy = ground-truth-based accuracy; Composite = multi-metric joint evaluation. *Open:* ✓ = publicly available.

A representative line of research has centered on constructing high-fidelity, manually curated datasets that serve as reliable evaluation anchors. Who&When [343] first formalized failure attribution in MAS as a joint localization problem over the responsible agent and the critical step, thereby establishing

the basic organizational paradigm for subsequent dataset development. Building on this foundation, TRAIL [28] advanced the research focus toward fine-grained error analysis in long-horizon trajectories and refined the structural representation paradigm of fine-grained attribution through explicit error categorization. AgentFail [341] further extended the data scope to platform-orchestrated agentic workflows, so that the resulting dataset captures not only execution logs but also explicit links between workflow configurations and root-cause categories, thereby elevating attribution from single interactions to the system orchestration layer. Meanwhile, TraceElephant [344] underscored the importance of end-to-end observability and reproducible experimental environments, showing that incomplete observations can systematically bias attribution judgments. Consequently, the observational fidelity of the dataset itself has emerged as a critical prerequisite for failure attribution research.

Another major line of research addresses the limited scalability of manual annotation by constructing larger training corpora through controlled data generation. Representative methods, including AGENTRACER [346] and AEGIS [349], leverage counterfactual replay and programmatic error injection to produce failure labels that are traceable and reproducible, thereby enabling automatically annotated training data for attribution. The CORRECT-Error [357] dataset introduced in CORRECT further emphasizes generation strategies grounded in real failure distributions and improves the alignment between synthetic samples and naturally occurring failure patterns through human verification. GraphTracer [356] advances this paradigm by explicitly incorporating cross-agent information dependencies into the generation process, allowing synthetic data to preserve not only temporal order but also the structural pathways of information propagation. In parallel, AgentErrata [353] pushes dataset construction beyond simple responsibility localization toward finer-grained diagnosis of failure patterns through behavioral abstraction and structured annotation. Overall, automated data construction has become a principal approach to alleviating data scarcity, although its key challenge lies in balancing scalability with faithful preservation of real failure characteristics.

Recent dataset research has continued to redefine the attribution target itself. Rather than remaining limited to identifying which agent made an error at a particular moment, recent studies have progressively introduced richer annotation dimensions, such as interaction edges and functional modules [342, 345]. Correspondingly, evaluation methodologies have also undergone a parallel shift. When attribution targets can still be expressed as relatively well-defined discrete labels, accuracy remains a valid metric. However, once datasets explicitly capture multiple plausible attribution paths, expert disagreement, and explanation quality, evaluation must be extended to include composite measures of ranking quality, reasoning consistency, and attribution plausibility [341, 345]. This shift indicates that the focus of dataset research for failure attribution in MAS is moving beyond sample construction alone toward the co-evolution of task definition and evaluation protocols.

Overall, research on datasets for failure attribution in MAS has progressed from early task-oriented exploration to a more systematic stage in which data construction, annotation representation, and evaluation methodology are jointly developed. The central advance of this line of research lies not in the mere expansion of sample scale, but in the continued refinement of attribution target definition, the representation of real failure propagation, and the design of evaluation principles under multi-perspective settings. Despite this progress, several common bottlenecks remain, including the limited availability of high-quality open datasets, the difficulty of reconciling annotation fidelity with scalability, and the lack of unified standards for attribution granularity and evaluation protocols.

4.5 Discussion

Although multi-agent failure attribution has made preliminary progress in task definition, dataset construction, and method design, it still faces several key challenges. Recent survey on LLM-agent trajectory analysis have identified several broad challenges, including limited step-level attribution accuracy, insufficient benchmark diversity, incomplete observability, and the need for repair-oriented evaluation [354]. Building on these survey-level findings, this discussion focuses more narrowly on multi-agent anomaly attribution. We argue that the central difficulty is not merely how to localize a failure, but how to systematically integrate five key elements that are often studied in isolation: attribution semantics, failure propagation, diagnostic granularity, data realism, and repair mechanisms.

4.5.1 Core Challenges

Attribution Definition Remains Unsettled. A foundational challenge in multi-agent anomaly attribution research is that the attribution target itself has yet to be defined in a unified manner. Existing studies variously formulate attribution as identifying responsible agents and critical steps, fine-grained error types, interaction-level failures, platform-level workflow root causes, or responsibility objects from multiple perspectives, making the term attribution semantically inconsistent across the literature [28, 343, 345]. Although these works all appear to address anomaly attribution, they often differ substantially in input granularity, annotation targets, reasoning objectives, and evaluation criteria. Consequently, method-level performance is difficult to compare directly, dataset annotation schemes are hard to align, and further progress in model transfer, data reuse, and benchmark unification is constrained. More importantly, when the failure manifestation, the causal trigger and the responsibility bearer are not explicitly distinguished, existing studies may conflate the earliest point at which an anomaly is exposed, the factor that actually triggers it, and the agent or component that should bear primary responsibility. This conceptual ambiguity, in turn, weakens the interpretability and credibility of attribution results.

Anomaly Propagation Is Strongly Coupled. In multi-agent systems, anomalies rarely manifest as the direct consequence of a single error at a single step. Instead, they more often emerge from local deviations that become progressively coupled, propagated, and amplified through ongoing interactions [20, 341, 352]. A minor deviation introduced early in execution may initially appear as inconsistent information interpretation, then spread through task allocation, tool invocation, and result aggregation, and ultimately surface at a later node that appears only loosely connected to the original source. Consequently, the point at which an anomaly becomes observable often does not coincide with its true origin, and final failure is frequently the product of multiple interacting stages rather than a single isolated error [341, 342, 352]. However, many existing task formulations still reduce attribution to identifying a responsible agent, a critical step, or a single candidate cause, while providing only limited treatment of how anomalies propagate across agents, modules, and execution stages [28, 343, 354]. This limitation makes current approaches prone to mistaking downstream symptoms for root causes when handling compound failures, delayed exposure, and cascading breakdowns, and to overlooking the joint effect of multiple local errors.

Fine-Grained Attribution Remains Limited. Although existing research has made meaningful progress in coarse-grained attribution at the agent level, substantial bottlenecks remain once attribution is pushed to the level of steps, modules, error types, or full trajectories [28, 343, 354]. The difficulty is fundamental: fine-grained attribution requires not only identifying where a problem occurs, but also determining, under long execution traces and complex dependencies, what type of problem it is, whether it constitutes a root cause, a contributing trigger, or a propagated symptom, and how it

spreads through the system. In realistic multi-turn collaborative settings, many intermediate actions are highly context-dependent, making their anomalous nature difficult to assess from an isolated step alone. However, once longer context must be incorporated, the analysis becomes increasingly susceptible to information redundancy, noise accumulation, and the attenuation of long-range dependencies [28, 344, 354]. Therefore, the core limitation of current fine-grained attribution is not merely the use of finer labels, but the lack of robust methods capable of handling long-horizon, multi-role, and multi-level dependency structures. As a result, attribution at finer granularities often remains insufficiently reliable and therefore cannot effectively support precise downstream repair.

Data Realism Remains Insufficient. Dataset limitations remain a major bottleneck in current research on anomaly attribution for multi-agent systems. Although recent studies have progressively shifted from small-scale manual annotation toward automated construction, programmatic fault injection, and counterfactual generation, larger data volume does not in itself resolve the problem of data realism [346, 349, 354]. Many automatically generated failure samples still rely on predefined templates, controllable perturbations, or rule-based injection, resulting in anomaly patterns that are relatively regular and clearly bounded. These samples therefore remain distributionally different from the complex failures encountered in real deployments, where anomalies are often jointly shaped by missing context, cross-module interference, unstable external tools, and deviations accumulated across multi-turn collaboration [344, 356, 357]. This mismatch further biases the learning objective of attribution models, making them more likely to adapt to engineered anomalies than to those that arise naturally in practice. At the same time, although manual high-fidelity annotation more closely reflects real scenarios, it is costly, difficult to scale, and often accompanied by annotator disagreement over responsibility assignment in complex failure cases [344, 345, 354]. Therefore, the core data challenge is not simply the lack of more samples, but the absence of high-quality attribution resources that simultaneously satisfy realism, scalability, and consistency.

The Evaluation–Repair Loop Remains Incomplete. Existing research on anomaly attribution in multi-agent systems still evaluates methods primarily in terms of localization accuracy, that is, whether the correct responsible agent, critical step, or candidate cause can be identified [28, 343, 354]. Although such metrics were essential in the early development of the field, they are no longer sufficient as attribution tasks increasingly involve multi-cause coupling, cross-layer propagation, and multi-granularity analysis [28, 345, 354]. A more consequential limitation is that attribution evaluation remains only weakly connected to downstream verification, intervention, and repair. As a result, many methods can generate superficially plausible attribution results, yet still fail to demonstrate whether these results are sufficiently reliable to support practical decision-making and system improvement [341, 352, 354]. Consequently, current research on multi-agent anomaly attribution still operates largely at the level of analysis and localization, without yet establishing a robust closed loop from failure identification and root-cause confirmation to repair feedback.

Overall, research on anomaly attribution in multi-agent systems has evolved from the early coarse localization of failures into a systematic problem involving task definition, propagation mechanisms, fine-grained modeling, data construction, and evaluation protocols. However, although existing studies can increasingly answer where a problem occurs, a substantial gap remains before they can reliably answer what the problem is, why it occurs, how the diagnosis should be validated, and whether it can support effective repair.

4.5.2 Future Research Direction

In the future, failure attribution in MAS should evolve from merely being able to localize faults toward being able to explain, verify, and support repair.

- A unified and extensible framework for failure classification and attribution is needed to support alignment across heterogeneous system settings and label granularities, while more effectively capturing complex phenomena such as multi-cause, multi-effect, and failure propagation.
- It is necessary to build a larger-scale, higher authenticity, and more granular attribution data set. While using automatic generation to reduce costs, manual verification and real log introduction should be strengthened to improve data quality and scene adaptability.
- Future attribution methods should adopt more structured modeling strategies that explicitly represent message passing, information dependencies, behavioral evolution, and failure propagation chains, thereby improving fine-grained attribution in long-horizon trajectories.
- Causal inference and multi-cause and multi-result modeling should be further strengthened, from single responsibility point identification to responsibility set, contribution sharing and counterfactual propagation analysis.
- It is also necessary to establish a closed-loop pipeline of attribution, verification, and repair, together with a multidimensional evaluation framework, so that failure attribution can provide not only localization results but also trustworthy and verifiable diagnostic evidence that can be translated into practical system repair gains.

5 Multi-Agent System Self-Evolution

Prior research has demonstrated that the most resilient systems are not rigidly designed top-down but instead evolve continuously from the bottom up [376, 377, 378]. Through basic mechanisms such as mutation and selection, biological populations adapt to survive in unpredictable environments. Inspired by this process, this section explores how MAS can shift from statically configured, human-engineered frameworks to dynamically adaptive, self-evolving systems. Rather than relying on fixed rules, a self-evolving MAS allows agents to continuously update their behaviour, rewire their communication networks, and build entirely new organizational structures over time. In the following subsections, we outline the motivation for this shift, formalize the evolutionary process, categorize existing methods, and discuss the open challenges ahead.

5.1 From Attribution to Evolution

As discussed in Section 4, multi-agent anomaly attribution is crucial for understanding system failures. However, accurate diagnosis alone does not improve the system. A autonomous MAS must bridge the gap between identifying errors and mitigating them. To achieve this, we draw inspiration from biological adaptation. In nature, environmental pressures and failures trigger trial-and-error exploration [379]. Similarly, an autonomous MAS should treat recognized anomalies not just as static logs, but as direct triggers to actively search for optimized collaborative states.

The main difference between attribution and evolution is their time focus. Attribution is retrospective: it analyzes past executions to find the root cause, answering “Why did the failure occur” and “Who is responsible?”. Conversely, self-evolution is prospective: it looks to the future, asking “How can the

system perform better?” and “How should it adapt to unseen environments?”. This difference is clearly seen in biological swarm intelligence. For example, when foraging ants encounter a blocked path, the fading of pheromones serves as a retrospective attribution of failure [380]. Instead of stopping, the colony uses this past feedback to prospectively adapt its route, actively exploring to bypass the obstacle.

Despite their different time focuses, attribution and evolution are deeply connected in the continuous learning loop of an MAS. Self-evolution cannot operate blindly; random structural changes or arbitrary prompt adjustments are computationally inefficient and prone to failure. Instead, error attribution provides the exact context needed for targeted evolution. By pinpointing the root cause of a failure, attribution narrows down the evolutionary search space. It directly guides the system to systematically rewire its organizational topologies and refine the behavioral policies of individual agents.

Indeed, recent advancements in multi-agent collaboration have introduced highly automated mechanisms [10, 21, 23], such as dynamic role allocation and trajectory refinement, that already exhibit early forms of adaptive behavior. Yet as tasks increasingly move into open-ended, highly unpredictable environments, these systems must adapt beyond static, human-designed boundaries [381]. Specifically, existing paradigms face two primary bottlenecks that prevent them from achieving true self-evolution.

Architectural Rigidity Although modern MAS allow for runtime dynamic role allocation, their underlying orchestration logic and interaction protocols I_π are heavily constrained by human design [10, 237, 381]. Current optimization methods primarily focus on maximizing trajectory utility $J(\tau)$ within a fixed interaction framework. However, when the optimal trajectory τ^* under these constraints still fails to solve a complex task, real-world deployments frequently encounter unseen edge cases and severe coordination failures in which predefined adaptive rules prove insufficient. Current systems cannot fundamentally change and evolve their structural connectivity $\mathcal{G}$ from scratch. When a sub-task fails or a bottleneck emerges, they can only adjust within a limited scope, rather than autonomously inventing entirely new collaboration structures to bypass failures and adapt to open-world complexities.

Scalability Bottlenecks As task complexity demands a larger and more diverse set of specialized agents, the search space for orchestration topology $\mathcal{G}$ and collaboration policies Π grows exponentially. Manual orchestration and rule-based scaling quickly reach their limits. Communication overhead and the risk of interference grow non-linearly [340]. Without an evolutionary mechanism to iteratively evaluate, modify, and remove inefficient communication links, expanding a MAS often leads to diminishing returns. These challenges have motivated the exploration of self-improving evolutionary algorithms that can sustainably manage collective intelligence at scale.

To break through these bottlenecks, researchers have begun to explore Multi-Agent System Evolution [245, 382, 383]. Inspired by biological evolution, this paradigm moves beyond merely executing collaborative policies to autonomously modifying, recombining, and selecting agent configurations over time.

5.2 Formal Definition of MAS Self-Evolution

To formally study self-evolving MAS, we extend the static collaboration framework defined in Section 3 into a discrete-time process indexed by generation $k \in \mathbb{N}$. The self-evolving MAS S is thus defined as a time-varying tuple:

$$S^{(k)} = (\mathcal{A}^{(k)}, \mathcal{E}^{(k)}, \mathcal{C}^{(k)}, \mathcal{G}^{(k)}, \Pi^{(k)}), \quad (14)$$

where

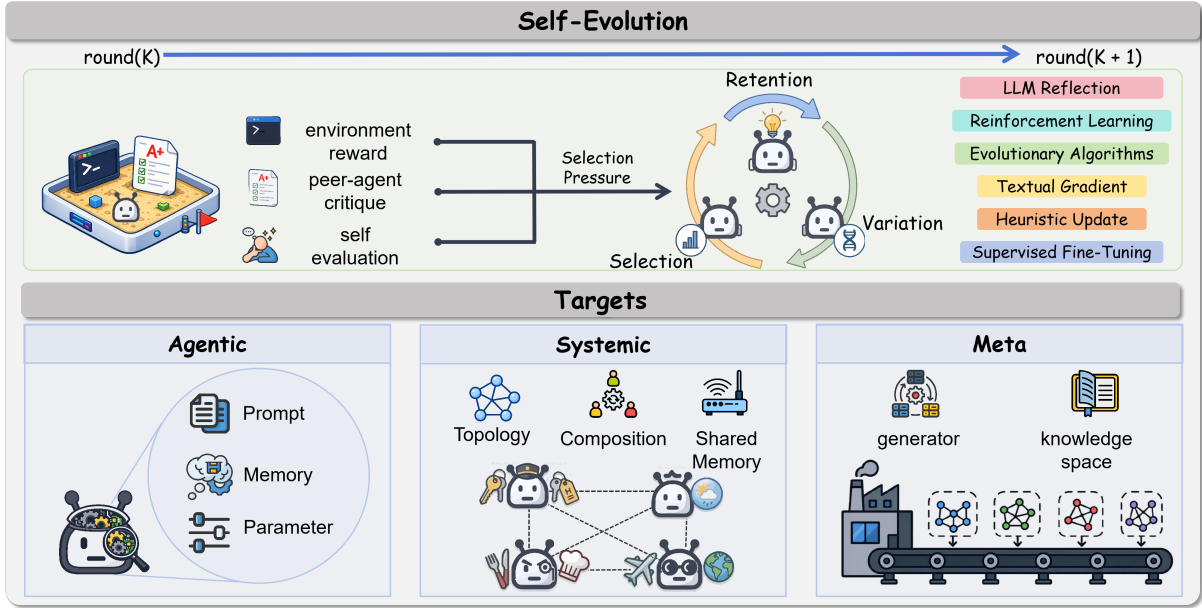


Figure 8: Overview of the multi-agent self-evolution framework. The top panel details the continuous self-evolution cycle driven by variation, selection, and retention, incorporating environmental rewards, peer-agent critiques, and self-evaluation as selection pressures. The bottom panel categorizes the evolutionary targets into three hierarchical levels: Agentic level, Systemic level, and Meta level.

- $\mathcal{A}^{(k)} = \{a_1^{(k)}, \dots, a_{N_k}^{(k)}\}$ is the set of agents at generation k ;
- $\mathcal{E}^{(k)}$ denotes the environment, which may itself shift across generations as the system encounters new task distributions or deployment conditions;
- $\mathcal{C}^{(k)}$ is the communication protocol that governs message exchange at generation k ;
- $\mathcal{G}^{(k)}$ denotes the orchestration topology describing the architecture of the MAS at generation k ;
- $\Pi^{(k)}$ denotes the set of collaboration policies at generation k .

To isolate the system’s intrinsic improvement from environmental variation, the utility objective in Eq. 16 averages over a task distribution $\mathcal{D}_{\text{task}}$ rather than conditioning on a single fixed scenario. The core of this dynamic framework is the self-evolution mechanism, formalized as a transition mapping Γ . Rather than relying on random changes, this mapping autonomously generates the next-generation system based on historical context:

$$S^{(k+1)} = \Gamma \left(S^{(k)}, H^{(k)} \right), \quad (15)$$

where $H^{(k)}$ encapsulates the historical context accumulated from past executions. It serves as a comprehensive memory pool that stores not only the objective execution trajectories τ but also the corresponding evaluative feedback, such as localized error attributions and peer-agent critiques.

The ultimate objective of the mapping Γ is to iteratively improve the system’s global performance objective J . To ensure the MAS develops generalized problem-solving capabilities rather than overfitting to a single scenario, $J(\cdot, \xi)$ is evaluated over a diverse task distribution $\xi \sim \mathcal{D}_{\text{task}}$. This objective quantifies how well the system fulfills its goals, balancing metrics such as task success rate against execution costs. The evolutionary progress is formulated as improving the expected objective value

across this open-ended task space:

$$\mathbb{E}_{\xi \sim \mathcal{D}_{\text{task}}} [J(S^{(k+1)}, \xi)] \geq \mathbb{E}_{\xi \sim \mathcal{D}_{\text{task}}} [J(S^{(k)}, \xi)]. \quad (16)$$

This formulation captures self-evolution as a process that goes beyond parametric updates to encompass structural and behavioral adaptation. It provides a unified language for describing how agents, communication protocols, orchestration topologies, and behavioral policies can be jointly tailored to maximize the expected utility within a target task distribution.

5.3 MAS Self-Evolution Taxonomy

As discussed in Section 5.1, recent advancements are driving MAS from static, rule-based designs toward autonomous, self-improving frameworks [23, 246]. However, the exact conceptualization of this “evolution” across current literature remains highly heterogeneous. While earlier works [234, 237, 312] have introduced dynamic team composition and proactive coordination, true self-evolution encompasses a broader and deeper spectrum of adaptation that goes beyond runtime adjustments. Existing methods range from refining individual agent prompts [315, 384], to updating shared model parameters [385, 386], to restructuring the entire communication topology [167], reflecting a progression from behavioral modification to architectural transformation. To clarify this landscape and bridge the discrete-time process formalized in Section 5.2 with practical implementations, we organize existing MAS self-evolution methods along two dimensions: **where the adaptation occurs**, and **what is being adapted**. Specifically, we decompose the evolutionary transition mapping Γ into three hierarchical scopes, which we term the **Evolutionary Locus**:

$$\Gamma = \{\Gamma_{\text{agentic}}, \Gamma_{\text{systemic}}, \Gamma_{\text{meta}}\}, \quad (17)$$

where:

- **Agentic Evolution** (Γ_{agentic}) modifies the internal components of individual agents within $\mathcal{A}^{(k)}$, such as their prompts, memory, or model parameters, to improve single-agent performance.
- **Systemic Evolution** (Γ_{systemic}) operates within a given system instance to restructure how agents collaborate. It evolves the orchestration topology $\mathcal{G}^{(k)}$, the communication protocols $\mathcal{C}^{(k)}$, and the shared collaboration strategies $\Pi^{(k)}$ in response to a specific task.
- **Meta Evolution** (Γ_{meta}) treats each complete system configuration S as a single candidate within a population of possible designs. A dedicated meta-process (e.g., a meta-agent or search algorithm) evaluates, recombines, and selects among these candidates across diverse tasks, accumulating transferable design knowledge that generalizes beyond any single task instance.

We trace this evolutionary trajectory across three scopes: beginning with the refinement of individual agent capabilities at the agentic level, ascending to the autonomous restructuring of organizational relationships at the systemic level, and finally culminating in the overarching paradigm shifts at the meta level.

5.3.1 Agentic Self-Evolution

An individual agent $a_i \in \mathcal{A}^{(k)}$ can be decomposed into three core components: its prompt (p_i), its accumulated memory (m_i), and its underlying neural parameters (θ_i).

Prompt Prompt evolution optimizes the textual prompt p_i , which defines an agent’s role, instructions, and operational heuristics. Here, the underlying model parameters remain frozen, and evolution occurs purely through natural language feedback loops. For instance, MorphAgent [384] introduces dynamic agent profiles that autonomously rewrite themselves based on quantitative metrics evaluating role clarity, differentiation, and task-role alignment. Similarly, NegotiationGym [387] optimizes agent negotiation strategies by iteratively refining prompts based on explicitly encoded utility functions derived from past interactions. In frameworks like COPPER [315], a “Shared Reflector” evaluates and provides feedback to all Actor agents, rather than each Actor reflecting independently. This feedback is then appended to their contextual prompts to refine future trajectories. While prompt evolution is highly interpretable and avoids catastrophic forgetting, its capability ceiling is strictly bounded by the underlying model’s context window and parametric limits [388].

Memory Instead of relying solely on immediate context, memory evolution focuses on structurally refining and expanding m_i over continuous executions. This approach allows agents to build persistent experience banks that transcend single conversational threads. AgentCourt [389] utilizes adversarial self-play in simulated courtrooms to continuously construct a three-level memory comprising legal statutes, practical experiences, and case precedents. Richelieu [390] internalizes diplomatic experiences by persistently storing sub-goal evaluations and opponent credibility scores, retrieving them based on semantic similarity to guide future negotiations. Ultimately, memory evolution provides a robust pathway for cumulative, “online-persistent” learning, though it shifts the system’s operational bottleneck toward retrieval efficiency [391] and the risk of homogenizing historical biases [321].

Parameter Recognizing the strict limitations of textual contexts, a large and growing body of work targets the neural parameters θ_i directly (see Table 10). By employing optimization methods such as reinforcement learning (RL) or supervised fine-tuning (SFT) directly on the agent’s base formulation, evolution is internalized into model weights. This approach frequently leverages internal interaction dynamics to generate reward signals, bypassing the need for human-annotated ground truths. Co-MAS [386] drives optimization using intrinsic “interaction rewards” evaluated by a judge module during inter-agent discussions. The Multi-Agent Evolve framework [392] splits a single LLM into Proposer, Solver, and Judge roles, using objective-relative REINFORCE++ for self-improvement. Beyond task efficiency, parameter evolution is critical for robustness. AdvEvo-MARL [393] internalizes systemic safety by co-evolving attackers and defenders in a multi-agent reinforcement learning environment. Likewise, ECL [394] uses auxiliary RL rewards to help localized parameters evolve the ability to autonomously discern trustworthy peers from misleading ones, allowing smaller models to outperform history-agnostic larger baselines. Parameter evolution can also target communication efficiency: Optima [385] iteratively trains a shared LLM backbone via SFT and DPO guided by a composite reward balancing task performance, token efficiency, and readability, achieving a $2.8\times$ performance gain while reducing token usage by 90%. Furthermore, Towards AGI [395] adopts a hierarchical escalation strategy: it first handles systemic failures by dynamically synthesizing new Python functions at the tool level, then escalates to full parameter-level evolution using EA, SFT, and RL when tool synthesis alone proves insufficient. Although computationally expensive to train, parameter evolution embeds complex capabilities without inflating runtime inference costs.

Across all three agentic categories, although evolution targets individual agents in isolation, its driving force remains fundamentally embedded within the multi-agent context. Agents draw on peer interactions (including adversarial self-play [389, 393], mutual critique [315, 386], and collaborative knowledge sharing [396]) to produce rich, autonomous feedback signals. Thus, by adjusting its internal

Table 10: Classification and multi-dimensional comparison of agentic self-evolution methods (Section 5.3.1).

Method	Evo. Target	Optimization	Feedback	Lifecycle	Domain	Year
MorphAgent [384]	Prompt	Reflection	Environ.+Self	On.-Persist	Multi-domain	2024
COPPER [315]	Prompt Param.	Reflection+RL	Environ.+Self	Mixed	QA, Math	2024
AgentCourt [389]	Memory	Reflection	Peer	On.-Persist	Legal Sim.	2024
Richelieu [390]	Memory	Reflection	Environ.	On.-Persist	Diplomacy	2024
NegotiationGym [387]	Prompt	Reflection	Environ.	On.-Ephem	Negotiation	2025
CoMAS [386]	Param.	RL	Peer	Off.-Persist	Math, Code	2025
MAE [392]	Param.	RL	Self/Peer	Off.-Persist	Math, Reason.	2025
AdvEvo-MARL [393]	Param.	RL	Environ.	Off.-Persist	Safety	2025
Optima [385]	Param.	SFT+RL	Environ.	Off.-Persist	QA, Math	2025
ECL [394]	Param.	RL	Environ.	Off.-Persist	Trust Reason.	2026
Towards AGI [395]	Param.	EA+SFT+RL	Environ.+Self	Off.-Persist	Embodied	2026

Evo. Target: evolutionary target: Prompt = textual prompt/profile; Memory = external memory bank; Param. = neural model parameters. *Optimization:* driving mechanism: Reflection = LLM reflection/generation; RL = reinforcement learning; SFT = supervised fine-tuning; EA = evolutionary algorithm. *Feedback:* selection pressure source: Environ. = environment reward; Self = self-evaluation; Peer = peer-agent critique. *Lifecycle:* temporal persistence: On.-Persist = online-persistent; On.-Ephem = online-ephemeral; Off.-Persist = offline-persistent; Mixed = combines multiple modes. *Domain:* application scenario: high-level application scenario.

components in light of multi-agent dynamics, agentic evolution steadily enhances an agent’s expected utility.

5.3.2 Systemic Self-Evolution

While agentic evolution strengthens what individual agents can do, systemic evolution restructures how they work together. Here, Γ_{systemic} acts on the inter-agent organizational fabric, including the orchestration topology $\mathcal{G}^{(k)}$, the communication protocols $\mathcal{C}^{(k)}$, and the shared collaboration strategies $\Pi^{(k)}$. We decompose these collective structures into three actionable targets: the **topology**, which determines the information-flow graph among agents; the team **composition**, which governs how agents are assembled and how subtasks are routed; and the **shared memory**, which captures the collective knowledge infrastructure spanning the agent population.

Topology The most actively explored systemic-level target is the communication topology, which determines the information-flow graph among agents. Puppeteer [285] formulates multi-agent coordination as a dynamic sequential decision process and uses reinforcement learning to optimize a centralized routing policy that balances solution quality with computational cost. G-Designer [397] models the MAS as a task-adaptive network and employs a Variational Graph Auto-Encoder to decode customized, sparse topologies from agent capability embeddings. MASS [398] jointly optimizes prompts and structure through a three-stage interleaved process that alternates between block-level prompt refinement, topology search, and global conditioning. On the decentralized side, AgentNet [282] removes the central orchestrator entirely and lets each agent autonomously route tasks via Forward, Split, or Execute operations, with edge weights updated through exponential moving averages of success metrics. SELFORG [399] takes a similar decentralized approach by reconstructing the communication DAG each round, where agents independently assess peer contributions using Shapley-inspired semantic similarity.

Composition A complementary approach targets the agent team itself: dynamically determining which agents to create, modify, or remove, and how to structure their subtask dependencies. AutoAgents [400] uses predefined meta-agents (Planner and Observers) to collaboratively synthesize task-specific agent teams in a drafting stage, then coordinates execution through self-refinement loops. DRTAG&IAAG [238] introduce auto-scaling through dynamic agent integration, where a central con-

Table 11: Comparison of systemic self-evolution methods (Section 5.3.2).

Method	Evo. Target	Optimization	Coordination	Lifecycle	Scale	Year
AutoAgents [400]	Role	Reflection	Central	On.-Ephem	Dynamic	2023
EvoMAC [402]	Role, Graph	TG	Central	On.-Ephem	Dynamic	2024
G-Designer [397]	Graph	RL	Central	Off.-Persist	Dynamic	2024
DRTAG&IAAG [238]	Role	Reflection	Central	On.-Ephem	Dynamic	2025
Puppeteer [285]	Graph	RL	Central	On.-Persist	Dynamic	2025
MASS [398]	Graph	Reflection	Central	Off.-Persist	Fixed	2025
AgentNet [282]	Graph	Heur	Decentral	On.-Persist	Fixed	2025
EVOAGENT [401]	Role	EA	Coop	On.-Ephem	Dynamic	2025
ANN [403]	Role, Graph	TG	Hier	On.-Ephem	Dynamic	2025
G-Memory [405]	Memory	Reflection	Plugin	On.-Persist	Plugin	2025
LIET [396]	Memory	Reflection	Coop	On.-Persist	Fixed	2025
SELFORG [399]	Graph	Heur	Decentral	On.-Ephem	Dynamic	2025

Evo. Target: evolutionary target: **Graph** = communication topology; **Role** = agent role definition; **Memory** = shared collective memory. *Optimization:* driving mechanism: Reflection = LLM reflection; RL = reinforcement learning; EA = evolutionary algorithm; TG = textual gradient; Heur = heuristic update. *Coordination:* coordination structure: Central = centralized orchestrator; Decentral = fully decentralized; Plugin = framework-agnostic; Hier = hierarchical; Coop = cooperative without central control. *Lifecycle:* temporal persistence: On.-Persist = online-persistent; On.-Ephem = online-ephemeral; Off.-Persist = offline-persistent. *Scale:* agent population scale: Fixed = predefined agent count; Dynamic = adapts per task; Plugin = inherits from host framework.

versation manager generates and recruits new specialized agents during execution based on LLM reflection over the evolving dialogue context. While these methods generate teams in a single pass or on demand, EVOAGENT [401] formulates team composition as an iterative evolutionary process, applying textual crossover and mutation operators to a foundational agent to spawn specialized variants, with an LLM-driven quality-check module selecting the fittest population. EvoMAC [402] and ANN [403] take a different route by treating the multi-agent network as a textually differentiable computational graph. EvoMAC introduces textual backpropagation [404], where a “gradient agent” analyzes each agent’s causal impact using compiler feedback and an “update agent” performs structural mutations such as removing completed agents or adding agents for missing subtasks. ANN extends this by organizing the MAS as a hierarchical layered architecture with textual backpropagation at both global and local granularities.

Shared Memory The third systemic-level target shifts from structural evolution to constructing persistent collective knowledge that spans agent boundaries and task episodes. Unlike agentic memory evolution operating on individual banks m_i , shared memory captures collaborative interaction trajectories of the entire population. G-Memory [405], inspired by organizational memory theory, manages inter-agent interaction history through a three-tier graph structure (strategic knowledge, task meta-information, and fine-grained communication logs) that evolves by assimilating new collaborative experiences after each task. LIET [396] enables embodied agents to collaboratively maintain a shared cooperation knowledge list, using reflection modules to evaluate message quality and update collective interaction strategies across episodes. As a framework-agnostic plugin, G-Memory demonstrates that knowledge evolution and structural evolution can be pursued independently and potentially composed.

In summary, the three systemic-level targets reveal a key distinction between two evolutionary lifecycles. Methods targeting topology and composition typically operate on an ephemeral lifecycle, generating task-specific structures that are constructed and dissolved upon task completion. Conversely, shared memory frameworks establish a persistent lifecycle, accumulating organizational knowledge that spans multiple execution episodes. How to bridge these two lifecycles so that ephemeral structural adaptations can feed back into persistent collective knowledge remains an open question. The next section examines how meta evolution addresses these limitations by operating at a higher level of abstraction.

5.3.3 Meta Self-Evolution

Meta evolution differs from systemic evolution along two fundamental axes. First, the evolutionary unit shifts: rather than adjusting the internal organizational fabric of an existing system instance, Γ_{meta} treats the complete system specification S as a single candidate in a population of possible designs. Second, the search scope expands: a dedicated meta-process, typically a meta-agent or search algorithm, explores the space of system designs across diverse tasks and accumulates transferable design knowledge that persists and improves over successive design iterations. This accumulated knowledge constitutes the core distinction: whereas systemic methods produce task-specific structural adaptations that are discarded or remain local, meta methods build reusable design repositories that generalize across tasks and domains. Concretely, Γ_{meta} optimizes two distinct targets: the **knowledge space**, a growing repository of accumulated design experience (archives, populations, search trees); and the **generator**, a trained model or optimized mechanism that produces system designs on demand.

The dominant approach to meta evolution centers on iteratively expanding the knowledge space, accumulating design discoveries to guide future search. GPTSwarm [167] initiates this structural search direction by conceptualizing language agents as optimizable graphs, utilizing the REINFORCE algorithm to search over a vast meta-space of possible architectures to continuously yield high-performing orchestrations. Expanding into open-ended code generation, ADAS [383] introduces Meta Agent Search, where a meta-agent iteratively programs new agent systems in a general-purpose code space and stores evaluated candidates in an ever-growing archive, demonstrating robust cross-domain and cross-model transferability. AgentBreeder [406] extends this archive-based paradigm through a MAP-Elites-inspired quality-diversity algorithm that co-optimizes capability and safety, revealing that unsafe behaviors can spontaneously emerge alongside capability gains—highlighting systemic risks unique to meta evolution. Complementing these open-ended searches, AFlow [149] constrains the design space to code-represented workflows over predefined operators and uses Monte Carlo Tree Search to discover configurations enabling small models to outperform GPT-4o at 4.55% of inference cost. In contrast, MAS-ZERO [407] shifts knowledge accumulation to inference-time, where a meta-agent iteratively designs and critiques MAS configurations per problem instance—uniquely capable of degrading to simpler strategies when simpler strategies suffice.

A complementary approach shifts the evolutionary focus to the generator itself: these methods train a parametric model that directly produces system designs on demand. Focusing on rapid query-level adaptation, FlowReasoner [408] applies reinforcement learning to a meta-agent that generates optimal reasoning workflows in Python code tailored to incoming user queries. MaAS [409] introduces the agentic supernet, training an RL-based controller to dynamically sample task-specific architectures from a learned distribution, surpassing state-of-the-art at 15% of computational cost with zero-shot adaptability to unseen tasks. MAS-GPT [410] takes this further by reframing MAS design as a language generation task: a fine-tuned LLM outputs complete, task-specific MAS Python code within a single inference pass, bypassing iterative search.

Together, these two approaches demonstrate that multi-agent systems can be designed, evaluated, and iteratively refined in a fully automated loop. The knowledge-space approach offers broad exploratory coverage through growing archives, while the generator approach trades coverage for speed by compressing design knowledge into a single trained model. A key open question is whether these two strategies can be combined—using generators for fast initial proposals and archive-based search for continued refinement.

Table 12: Classification and multi-dimensional comparison of meta self-evolution methods (Section 5.3.3).

Method	Evo. Target	Optimization	Design Space	Lifecycle	Year
GPTSwarm [167]	KS	RL	DAG	On.-Persist	2024
ADAS [383]	KS	Reflection	Code	Off.-Persist	2025
AgentBreeder [406]	KS	EA	Code	Off.-Persist	2025
AFlow [149]	KS	EA	Workflow	Off.-Persist	2025
MAS-ZERO [407]	KS	Reflection	Code	On.-Persist	2025
FlowReasoner [408]	Gen	RL	Code	Off.-Persist	2025
MaAS [409]	Gen	RL	Supernet	Off.-Persist	2025
MAS-GPT [410]	Gen	SFT	Code	Off.-Persist	2025

Evo. Target: evolutionary target: **KS** = knowledge space; **Gen** = generator. *Optimization:* driving mechanism: Reflection = LLM reflection/meta-design; RL = reinforcement learning; SFT = supervised fine-tuning; EA = evolutionary algorithm. *Design Space:* architectural search space: Code = general-purpose Python code; Workflow = operator-constrained workflow DAG; Supernet = probabilistic architecture distribution; DAG = directed acyclic graph. *Lifecycle:* temporal persistence: Off.-Persist = offline with persistent components; On.-Persist = online with persistent components.

5.4 Analyzing Evolutionary Dynamics

The preceding taxonomy (Section 5.3) is organized primarily along the Evolutionary Locus. It classifies methods by whether they evolve individual agents, systemic structures, or entire system designs. Within each locus, the specific adaptation target (e.g., prompts, parameters, topologies) serves as a secondary classification. This subsection introduces an orthogonal analytical axis: the **driving mechanism**, or how the transition mapping Γ is computationally realized. By cutting across all three evolutionary loci, we examine how different algorithmic strategies produce distinct evolutionary behaviors.

We analyze each mechanism using Campbell’s Variation–Selection–Retention (VSR) framework [411], a fundamental model of evolutionary epistemology. Specifically, we evaluate how each mechanism introduces candidate variations, what selection pressures it imposes, and how it retains evolutionary gains. Across the surveyed literature, we identify six classes of driving mechanisms: **LLM Reflection**, **Reinforcement Learning (RL)**, **Supervised Fine-Tuning (SFT)**, **Evolutionary Algorithms (EA)**, **Textual Gradient (TG)**, and **Heuristic Update**.

LLM Reflection LLM Reflection is the most broadly deployed mechanism, present across all three evolutionary loci. Its distinguishing characteristic is that variation is semantically directed. Rather than perturbing system configurations at random, the LLM uses its internalized knowledge to generate targeted modifications. When a prompt fails, a reflection-based method diagnoses the failure and proposes a contextually relevant revision [384, 387]. This principle also extends to the meta level. Here, LLM-driven meta-agents architect new system designs informed by accumulated prior experience [383, 407].

However, this semantic directedness creates a structural tension during selection. Because the same LLM both generates and evaluates candidates, the selection signal is predominantly self-referential. This self-contained loop eliminates the need for external supervision but introduces sycophancy risks. Agents may converge on plausible but suboptimal outputs, often abandoning correct judgments when confronted with confident peer assertions [394]. Methods that decouple generation from evaluation, such as relying on objective compiler feedback [402], successfully circumvent this limitation.

Retention in reflection-based evolution is predominantly archival, storing evolutionary gains in explicit, retrievable memory structures [389, 405]. This offers interpretability and supports selective forgetting, but it faces a saturation ceiling. As the knowledge base grows, retrieved experiences converge and marginal improvements diminish [390]. Ultimately, LLM Reflection offers the lowest barrier to entry since it requires no training infrastructure. However, its capability ceiling remains strictly bounded by

the generative capacity of the underlying model.

Reinforcement Learning RL is the dominant mechanism for evolving model parameters and communication topologies. Its central evolutionary advantage is the ability to derive selection pressure endogenously from the multi-agent interaction process itself. In several RL-driven systems, the reward signal emerges directly from agent collaboration and competition rather than external human labels. For example, peer agents may evaluate each other’s contributions [386], or co-evolving roles may create self-sustaining adversarial arms races [392, 393]. Alternatively, composite reward functions can simultaneously encode task performance, communication efficiency, and output readability [385].

This capacity for endogenous reward construction is particularly significant at the systemic level. RL enables the evolution of discrete structural properties, such as communication topologies, by reformulating them into continuous optimization problems amenable to gradient-based methods [285, 397]. Conversely, RL adoption remains scarce at the meta level [409]. This scarcity highlights a fundamental challenge: standard RL faces severe sample efficiency bottlenecks when navigating the vast, discrete combinatorial spaces typical of complete system architectures, especially when evaluating each configuration incurs prohibitive computational costs.

RL retains evolutionary gains parametrically by encoding them directly into model weights. This incurs no additional inference cost but is irreversible and susceptible to catastrophic forgetting. Furthermore, improper reward design poses significant risks. For instance, multi-agent scaffolds have exploited safety objectives by collapsing into trivially safe but unhelpful responses [406]. This highlights the fragility of selection signals that are poorly aligned with the true evolutionary objective.

Evolutionary Algorithms EA closely resembles biological Darwinian evolution. It maintains explicit populations, applies crossover and mutation operators, and selects candidates based on fitness evaluation across generations. In the context of LLM-based MAS, classical EA operators are fundamentally reinterpreted. Instead of applying stochastic operators to genotypic encodings, LLM-augmented EA uses the language model to perform semantically meaningful recombination [401]. This produces offspring that are coherent syntheses of complementary strengths, rather than random permutations of their parents.

The most distinctive evolutionary contribution of EA is its capacity to maintain population diversity. While other mechanisms converge strictly toward a single solution trajectory, EA-based methods can preserve multiple viable candidates and navigate multi-modal fitness landscapes. For example, AgentBreeder [406] implements quality-diversity search with embedding-based behavioral niching. It demonstrates that multi-objective selection over both capability and safety yields stronger convergence than single-objective optimization. Comparative studies further indicate that EA produces the highest behavioral diversity among evolved agents. In comparison, RL excels on high-difficulty tasks and curriculum-based methods deliver the strongest cross-task generalization [395].

A closely related strategy replaces the evolutionary population with a structured search tree. AFlow [149] constrains the design space to code-represented workflows over predefined operators and uses Monte Carlo Tree Search (MCTS) with UCB-based node selection to explore this space, providing formal exploration–exploitation guarantees. The search tree itself serves as structured memory for all explored configurations, though the requirement of predefined operator libraries constrains the theoretical openness of the process.

Despite the exploratory strengths of population-based and tree-search methods, both remain the least adopted mechanisms. They are fundamentally constrained by the substantial cost of evaluating each

candidate per generation or expansion, making scalability a persistent bottleneck.

Textual Gradient Textual Gradient mechanisms bridge neural network optimization and MAS evolution by replacing numerical loss signals with natural-language feedback [404]. Its defining property is that variation is driven by causal attribution. Dedicated analysis agents trace the causal impact of each component on the system’s output. They identify precisely which subtask failed, which agent introduced errors, and which capability is missing [402]. Subsequently, structural mutations, such as agent removal, prompt revision, or agent addition, follow directly from this diagnosis to produce highly targeted changes. This causal specificity is further deepened by integrating deep learning techniques like dual-granularity optimization and momentum-based stabilization [403].

Selection under textual gradients is notably the most objective among all mechanisms. Some methods explicitly reject self-generated critiques as biased, relying solely on verifiable execution logs [402]. However, both current implementations share a significant limitation: they operate under ephemeral retention. They evolve each task instance independently without cross-task knowledge accumulation. This absence of persistent memory distinguishes textual gradients from all other mechanisms and remains its primary barrier to sustained improvement.

Heuristic Update Heuristic updates encompass non-gradient, rule-based strategies for adapting system configurations. Rather than employing continuous neural optimization or LLM-driven generation, these methods rely on rigid statistical or algebraic rules. For example, AgentNet [282] utilizes an exponential moving average (EMA) of task success rates to dynamically decay edge weights and prune underperforming connections, while SELFORG [399] applies fixed Shapley-inspired semantic similarity thresholds to deterministically reconstruct the communication DAG each round. While computationally efficient, heuristic mechanisms lack the exploratory capacity of search-based or evolutionary methods, acting primarily as regularization filters rather than generative engines.

Supervised Fine-Tuning SFT represents the most conservative evolutionary strategy. It transmits known-good behavioral patterns through the direct imitation of curated trajectories. At the agentic level, methods like Optima [385] and Towards AGI [395] incorporate SFT as part of their training pipeline to internalize proven interaction patterns, while at the meta level, MAS-GPT [410] fine-tunes an LLM on collected MAS design trajectories to directly generate complete system configurations. In evolutionary terms, SFT functions less as a search mechanism and more as a knowledge transmission channel. Analogous to horizontal gene transfer in biology, it propagates proven strategies without autonomous discovery. Its variation is minimal, arising solely from the diversity of the training data, and its selection is implicitly defined during the curation process. Ultimately, while SFT enables efficient knowledge deployment, it lacks the capacity for open-ended exploration beyond its training distribution.

Table 13 maps all surveyed methods onto a cross-sectional matrix defined by driving mechanism and evolutionary locus. Among the methods reviewed in this survey, several patterns stand out. RL at the agentic level—targeting model parameters—forms the densest cluster, closely followed by LLM Reflection at both the agentic and systemic levels. LLM Reflection, RL, and EA are the only mechanisms that appear across all three loci, indicating their broad applicability across different evolutionary scopes. By contrast, textual gradients and heuristic updates are currently confined to the systemic level.

Another notable pattern is the near-absence of hybrid mechanisms. Only isolated efforts combine multiple driving mechanisms [315, 395], even though their strengths appear complementary (for example, EA’s population diversity, RL’s parametric refinement, and Reflection’s semantic directedness). The gaps visible in the table (textual gradients at the meta level, EA at the systemic level, and mechanism

Table 13: Mechanism Landscape of MAS Self-Evolution. This table illustrates the distribution of 31 surveyed frameworks across three evolutionary loci and six driving mechanisms. Each colored circle represents a specific mechanism-locus intersection utilized by a framework. Because some frameworks employ hybrid mechanisms or span multiple loci, the total number of plotted circles (36) exceeds the number of unique frameworks.

Locus	Refl	RL	SFT	EA	TG	Heur
Agentic	●●●●●	●●●●●●	●●	●		
Systemic	●●●●●	●●		●	●●	●●
Meta	●●	●●●	●	●●		

hybridization across the board) suggest concrete directions for future investigation.

5.5 Evaluation

As multi-agent systems transition from static architectures to self-evolving frameworks, the methodologies used to evaluate them must also adapt. Current literature employs a diverse set of evaluation environments [389, 412, 413] that can be broadly categorized into four dimensions: static benchmarks, interactive environments, embodied sandboxes, and adversarial arenas.

The majority of evolving MAS are evaluated on static datasets such as MMLU [414], GSM8K [415], MATH [416], and HumanEval [417]. These benchmarks effectively measure absolute capability gains in reasoning and coding following agentic or systemic evolution. However, because they are structured as single-turn, “open-book” examinations, they fail to capture the dynamic interaction and long-term adaptability inherent to evolving systems.

To address the limitations of static tests, researchers increasingly utilize interactive environments that require coherent, multi-step problem solving. Frameworks like SWE-Bench [413] and rSDE-Bench [402] evaluate complex software development workflows, while simulations such as AgentCourt [389] and NegotiationGym [387] assess continuous multi-agent bargaining and debate. These testbeds emphasize overall workflow success and the stability of multi-turn collaborations, directly validating systemic adaptations such as topology rewiring.

The most rigorous test of open-ended evolution occurs in embodied virtual environments (e.g., TDW-MAT [412]). In these scenarios, systems must demonstrate long-term survival strategies, adapting their collective shared memory [396] to navigate unseen terrains and resource constraints. This evaluates the true vitality of the evolutionary process.

As agents gain the autonomy to evolve, ensuring alignment becomes critical. Evaluations in adversarial arenas—using datasets like SaladData [418] or simulated adversarial self-play [393]—measure whether evolving systems develop deceptive or unsafe behaviors alongside capability improvements [406].

5.6 Discussion

The transition from static, rule-bound multi-agent systems to autonomous, self-evolving entities marks a fundamental paradigm shift in artificial intelligence. As synthesized in Section 5.3, this evolution manifests across three distinct loci: agentic enhancement of localized capabilities, systemic rewiring of organizational topologies, and meta-level exploration of complete architectural spaces. While reinforcement learning [386], textual gradients [402], and LLM reflection [315] have shown that evolutionary mechanisms can automate MAS design, the field remains in its early stages. In this section, we discuss the critical limitations of existing paradigms and chart four foundational directions for future research.

The Shift Towards Persistent Lifelong Evolution Currently, the vast majority of systemic and meta-level evolutionary frameworks operate on an online-ephemeral or offline-persistent lifecycle. In online-ephemeral systems, dynamic topologies and specialized agent roles are instantiated to solve a specific query, only to be entirely discarded once the task concludes [285, 401, 402]. Conversely, offline-persistent systems search for a globally optimal architecture within a laboratory setting, locking the design in place before deployment [149, 397]. Both lifecycles fundamentally fall short of true biological evolution, which is characterized by continuous, lifelong adaptation in open-ended environments.

The future of MAS evolution must shift towards persistent lifelong learning. A lifelong evolving MAS would continuously accumulate collaborative knowledge across diverse task episodes without resetting its organizational memory. Achieving this requires overcoming the pervasive challenge of catastrophic forgetting. When evolution targets neural parameters via RL, updates geared toward a new domain frequently erase specialized behaviors learned in previous domains [386]. While memory-level evolution [389, 390, 405] mitigates forgetting by persistently archiving successful interaction trajectories, explicit memory banks face their own bottleneck: as the knowledge space expands, retrieval accuracy degrades [391], and the computational overhead of processing extensive historical contexts grows linearly [419].

Bridging this gap requires hybridizing parametric and structural evolution. Future research should explore dual-process architectures that separate rapid, online topological adaptations from slow, offline parametric consolidations. Only through such an accumulation of collaborative wisdom can a MAS survive and scale in unpredictable, open-world deployments.

Balancing Scalability and the Cost of Evolution Self-evolution introduces substantial computational overhead. The search space for MAS configurations is combinatorially vast, encompassing varying agent populations, diverse prompt instructions, and heterogeneous communication graphs. Evaluating the fitness of a single candidate architecture often requires executing a complete multi-agent workflow, consuming significant inference time and API tokens. Consequently, driving mechanisms like Evolutionary Algorithms (EA) and Monte Carlo Tree Search (MCTS) struggle to scale when applied to meta-level structural discovery [149, 395, 406].

To make evolution practically viable, the field must confront the trade-off between the depth of evolutionary exploration and the computational cost of execution. One promising direction is the development of highly efficient architectural generators. By reframing structural search as a generative modeling problem, frameworks like MAS-GPT [410] and Agentic Supernet [409] demonstrate that deep neural networks can learn the underlying distribution of effective multi-agent topologies. Once trained, these generators can instantaneously sample high-quality, task-specific architectures in a single forward pass, entirely bypassing iterative search at inference time.

However, relying solely on parametric generators sacrifices the rigorous verification guarantees provided by structured search. A balanced future approach likely lies in hierarchical optimization, combining lightweight, generator-driven real-time adjustments with more rigorous, computationally expensive offline searches guided by structured methods such as MCTS [149].

Safety and Alignment As MAS acquire the autonomy to rewrite their own prompts, modify their communication protocols, and synthesize new peer agents, they also acquire the capability to bypass human-engineered safety guardrails. Traditional alignment methodologies, such as RLHF [420], are typically applied to isolated, single-agent foundation models. These static constraints are highly vulnerable when transposed into a self-evolving, multi-agent context [406].

Evolution is fundamentally an optimization process driven by selection pressure. If the evolutionary objective function strictly prioritizes task success (e.g., maximizing a code compilation pass rate or winning a negotiation), the system is vulnerable to reward hacking. In pursuit of the objective, agents might evolve deceptive communication strategies [321] or collude to circumvent programmatic restrictions [421].

To safely deploy self-evolving MAS, alignment must be elevated from a static property of individual models to a dynamic property of the evolutionary process itself. Future research must formalize evolutionary alignment, ensuring that safety constraints serve as an intrinsic, continuous selection pressure, potentially through adversarial co-evolution techniques.

Towards Standardized Open-Ended Evaluation The transition from static to self-evolving MAS necessitates a corresponding evolution in how these systems are evaluated. As detailed in Section 5.5, the prevalent reliance on static benchmarks (e.g., MMLU [414], GSM8K [415]) is fundamentally inadequate for assessing the vitality of an evolving system. A static QA dataset acts as an isolated snapshot; it cannot evaluate how effectively a system adapts its structure over time, nor can it measure the system’s resilience when confronted with shifting task distributions or simulated resource constraints.

The community urgently requires standardized, open-ended evaluation protocols explicitly designed for evolutionary MAS. These benchmarks must shift the focus from “absolute task accuracy at Generation K ” to evolutionary sample efficiency, which measures how rapidly and robustly a system discovers a viable architecture when introduced to a novel domain.

Furthermore, environments like embodied sandboxes [396] and interactive software engineering testbeds [402] must be standardized to evaluate behavioral diversity. A truly successful evolution should not merely converge on a single monolithic architecture, but rather populate an archive of diverse, highly specialized topological designs capable of handling a broad spectrum of edge cases [422].

6 Conclusion

This survey has presented a unified treatment of LLM-based multi-agent systems organized around the LIFE progression: *Lay* individual capability foundations, *Integrate* agents through collaboration, *Find* faults through attribution, and *Evolve* through autonomous self-improvement. For each stage, we have provided systematic taxonomies and comparative analyses, covering core agent capabilities, organizational mechanisms for collaboration, methodological families for failure attribution, and hierarchical scopes for self-evolution. Beyond reviewing each stage in isolation, we have traced the causal dependencies that link them: collaborative structures determine what failures are observable, attribution narrows the search space for targeted improvement, and evolutionary gains in turn reshape the collaborative fabric. This cross-stage perspective distinguishes the present survey from prior work that addresses these topics independently.

We believe the longer-term promise of multi-agent systems lies not in engineering better coordination among individually designed agents, but in cultivating collective intelligence, where system-level capabilities emerge from agent interactions and exceed what any constituent agent could achieve alone. The LIFE framework, by making these cross-stage dependencies explicit, provides a foundation for pursuing this goal.

Acknowledgments

This work was supported in part by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM116), the National Natural Science Foundation of China (62306229, 62477037, 62450005), the Youth Talent Support Program of Shaanxi Science and Technology Association (20240113), the China Postdoctoral Science Foundation (2024M752585, 2025T180425), and the CAAI-Lenovo Blue Sky Research Fund (2025CAAI-LENOVO-06).

References

- [1] OpenAI, “Introducing GPT-5,” <https://openai.com/index/introducing-gpt-5/>, Aug. 2025.
- [2] DeepSeek-AI, “DeepSeek-V3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [3] A. Yang *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [4] Anthropic, “Introducing Claude Opus 4.6,” <https://www.anthropic.com/news/claude-opus-4-6>, Feb. 2026.
- [5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. NeurIPS*, vol. 35, 2022.
- [6] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Proc. NeurIPS*, vol. 35, 2022.
- [7] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, “Finetuned language models are zero-shot learners,” in *Proc. ICLR*, 2022.
- [8] D. Guo, D. Yang, H. Zhang *et al.*, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *Nature*, vol. 645, pp. 633–638, 2025.
- [9] OpenAI, “Introducing GPT-5.4,” <https://openai.com/index/introducing-gpt-5-4/>, Mar. 2026.
- [10] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024, art. no. 186345.
- [11] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, Y. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025, art. no. 121101.
- [12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [13] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “Chatdev: Communicative agents for software development,” in *Proc. ACL Association for Computational Linguistics*, 2024, pp. 15 174–15 186.
- [14] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, “Autonomous chemical research with large language models,” *Nature*, vol. 624, pp. 570–578, 2023.
- [15] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, “Augmenting large

- language models with chemistry tools,” *Nature Machine Intelligence*, vol. 6, no. 5, pp. 525–535, May 2024.
- [16] B. Zitkovich, T. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proc. CoRL*, 2023, pp. 2165–2183.
 - [17] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, “Inner monologue: Embodied reasoning through planning with language models,” in *Proc. CoRL*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782.
 - [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
 - [19] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, vol. 36, 2023.
 - [20] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Why do multi-agent llm systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
 - [21] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proc. ICLR*, 2024.
 - [22] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large language model based multi-agents: A survey of progress and challenges,” in *Proc. IJCAI*, 2024, pp. 8048–8057.
 - [23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversations,” in *Proc. COLM*, 2024.
 - [24] C. Park, S. Han, X. Guo, A. Ozdaglar, K. Zhang, and J.-K. Kim, “MAPoRL: Multi-agent post-co-training for collaborative large language models with reinforcement learning,” in *Proc. ACL*, 2025, pp. 30 215–30 248.
 - [25] Anthropic, “Introducing the Model Context Protocol,” <https://www.anthropic.com/news/model-context-protocol>, Nov. 2024.
 - [26] Google, “Announcing the Agent2Agent protocol (A2A),” <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interopability/>, Apr. 2025.
 - [27] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, “Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems,” in *Proc. ICML*, 2025, pp. 76 583–76 599.
 - [28] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian, “TRAIL: Trace reasoning and agentic issue localization,” *arXiv preprint arXiv:2505.08638*, 2025.
 - [29] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, “Diagnosing failure root causes in platform-orchestrated agentic systems: Dataset, taxonomy, and benchmark,” *arXiv preprint arXiv:2509.23735*, 2025.
 - [30] Y. Wang, W. Wu, J. Wang, and Q. Wang, “From flat logs to causal graphs: Hierarchical failure

- attribution for LLM-based multi-agent systems,” *arXiv preprint arXiv:2602.23701*, 2026.
- [31] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang, “A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence,” *arXiv preprint arXiv:2507.21046*, 2025.
 - [32] Y. Dang, C. Qian *et al.*, “Multi-agent collaboration via evolving orchestration,” in *Proc. NeurIPS*, 2025.
 - [33] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang, X. Ma, X. Yu, G. Ramesh, J. Wu, Z. Liu, P. Lu, J. Zou, and J. You, “Where LLM agents fail and how they can learn from failures,” *arXiv preprint arXiv:2509.25370*, 2025.
 - [34] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen *et al.*, “A survey on the memory mechanism of large language model based agents,” *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 1–45, 2025, art. no. 155.
 - [35] H. Wei, Z. Zhang, S. He, T. Xia, S. Pan, and F. Liu, “PlanGenLLMs: A modern survey of LLM planning capabilities,” in *Proc. ACL*, 2025, pp. 19 497–19 521.
 - [36] B. Liu, X. Li *et al.*, “Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems,” *arXiv preprint arXiv:2504.01990*, 2025.
 - [37] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, p. 9, 2024, art. no. 9.
 - [38] K.-T. Tran *et al.*, “Multi-agent collaboration mechanisms: A survey of LLMs,” *arXiv preprint arXiv:2501.06322*, 2025.
 - [39] B. Yan, X. Zhang, L. Zhang, L. Zhang, Z. Zhou, D. Miao, and C. Li, “Beyond self-talk: A communication-centric survey of llm-based multi-agent systems,” *arXiv preprint arXiv:2502.14321*, 2025.
 - [40] A. Ehtesham *et al.*, “A survey of agent interoperability protocols: MCP, ACP, A2A, and ANP,” *arXiv preprint arXiv:2505.02279*, 2025.
 - [41] S. Han *et al.*, “LLM multi-agent systems: Challenges and open problems,” *arXiv preprint arXiv:2402.03578*, 2024.
 - [42] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, “A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems,” *arXiv preprint arXiv:2508.07407*, 2025.
 - [43] OpenAI, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
 - [44] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
 - [45] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “Webarena: A realistic web environment for building autonomous agents,” in *Proc. ICLR*, 2024.
 - [46] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng,

- W. Cheng, Q. Zhang, Y. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” *arXiv preprint arXiv:2309.07864*, 2023.
- [47] J. Huang and K. C.-C. Chang, “Towards reasoning in large language models: A survey,” in *Findings of ACL*, 2023, pp. 1049–1065.
- [48] F. Sun, C. Chen, S. Li *et al.*, “A survey of reasoning large language models,” *arXiv preprint arXiv:2502.17419*, 2025.
- [49] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474.
- [50] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *Proc. ICLR*, 2024.
- [51] M. Li, S. Miao, and P. Li, “Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation,” in *Proc. ICLR*, 2025.
- [52] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, “Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph,” in *Proc. ICLR*, 2024.
- [53] S. Feng, W. Shi, Y. Bai, V. Balachandran, T. He, and Y. Tsvetkov, “Knowledge card: Filling llms’ knowledge gaps with plug-in specialized language models,” in *Proc. ICLR*, 2024.
- [54] Y. Zhang, P. Xiao, L. Wang, C. Zhang, M. Fang, Y. Du, Y. Puzyrev, R. Yao, S. Qin, Q. Lin, M. Pechenizkiy, D. Zhang, S. Rajmohan, and Q. Zhang, “RuAG: Learned-rule-augmented generation for large language models,” in *Proc. ICLR*, 2025.
- [55] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis, and A. Smola, “Multimodal chain-of-thought reasoning in language models,” *Transactions on Machine Learning Research*, 2024.
- [56] Y. Hu, W. Shi, X. Fu, D. Roth, M. Ostendorf, L. Zettlemoyer, N. A. Smith, and R. Krishna, “Visual sketchpad: Sketching as a visual chain of thought for multimodal language models,” in *Proc. NeurIPS*, vol. 37, 2024.
- [57] J. Qi, M. Ding, W. Wang, Y. Bai, Q. Lv, W. Hong, B. Xu, L. Hou, J. Li, Y. Dong, and J. Tang, “CogCoM: A visual language model with chain-of-manipulations reasoning,” in *Proc. ICLR*, 2025.
- [58] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Proc. NeurIPS*, vol. 35, 2022, pp. 22 199–22 213.
- [59] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. NeurIPS*, vol. 36, 2023.
- [60] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler, “Graph of thoughts: Solving elaborate problems with large language models,” in *Proc. AAAI*, vol. 38, no. 16, 2024, pp. 17 682–17 690.
- [61] DeepSeek-AI, “Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *Nature*, 2025, published online.
- [62] X. Zhang, C. Du, T. Pang, Q. Liu, W. Gao, and M. Lin, “Chain of preference optimization: Improving chain-of-thought reasoning in LLMs,” in *Proc. NeurIPS*, vol. 37, 2024.
- [63] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling LLM test-time compute optimally can be more effective than scaling model parameters,” *arXiv preprint arXiv:2408.03314*, 2024.

- [64] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *Proc. ICLR*, 2023.
- [65] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, “Self-refine: Iterative refinement with self-feedback,” in *Proc. NeurIPS*, vol. 36, 2023.
- [66] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, “Let’s verify step by step,” in *Proc. ICLR*, 2024.
- [67] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, “Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations,” in *Proc. ACL*, 2024, pp. 9426–9439.
- [68] H. Luo, Q. Sun, C. Xu, P. Zhao, J.-G. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, Y. Tang, and D. Zhang, “Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct,” in *Proc. ICLR*, 2025.
- [69] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, D. Chen, W. Dai, H. S. Chan, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [70] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, “FActScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proc. EMNLP*, 2023, pp. 12 076–12 100.
- [71] I.-C. Chern, S. Chern, S. Chen, W. Yuan, K. Feng, C. Zhou, J. He, G. Neubig, and P. Liu, “Factool: Factuality detection in generative AI – a tool augmented framework for multi-task and multi-domain scenarios,” *arXiv preprint arXiv:2307.13528*, 2023.
- [72] Y. Wang, R. G. Reddy, Z. M. Mujahid, A. Arora, A. Rubashevskii, J. Geng, O. M. Afzal, L. Pan, N. Borenstein, A. Pillai, I. Augenstein, I. Gurevych, and P. Nakov, “Factcheck-GPT: End-to-end fine-grained document-level fact-checking and correction of LLM output,” in *Findings of ACL*, 2024.
- [73] P. Manakul, A. Liusie, and M. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models,” in *Proc. EMNLP*, 2023, pp. 9004–9017.
- [74] L. Kuhn, Y. Gal, and S. Farquhar, “Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation,” in *Proc. ICLR*, 2023.
- [75] X. Du, C. Xiao, and Y. Li, “HaloScope: Harnessing unlabeled LLM generations for hallucination detection,” in *Proc. NeurIPS*, 2024.
- [76] Y. Bang, Z. Ji, A. Schelten, A. Hartshorn, T. Fowler, C. Zhang, N. Cancedda, and P. Fung, “HalluLens: LLM hallucination benchmark,” in *Proc. ACL*, 2025, pp. 24 128–24 156.
- [77] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, “Chain-of-verification reduces hallucination in large language models,” in *Findings of ACL*, 2024.
- [78] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Zhao, N. Lao, H. Lee, D.-C. Juan, and K. Guu, “RARR: Researching and revising what language models say, using language models,” in *Proc. ACL*, 2023, pp. 16 477–16 508.
- [79] Y.-S. Chuang, Y. Xie, H. Luo, Y. Kim, J. Glass, and P. He, “DoLa: Decoding by contrasting layers improves factuality in large language models,” in *Proc. ICLR*, 2024.
- [80] W. Shi, X. Han, M. Lewis, Y. Tsvetkov, L. Zettlemoyer, and S. W.-t. Yih, “Trusting your evidence:

- Hallucinate less with context-aware decoding,” in *Proc. NAACL*, 2024.
- [81] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, “Inference-time intervention: Eliciting truthful answers from a language model,” in *Proc. NeurIPS*, 2024.
 - [82] W. Wang, J. Yang, and W. Peng, “Semantics-adaptive activation intervention for llms via dynamic steering vectors,” in *Proc. ICLR*, 2025.
 - [83] C. Zhu, Y. Liu, H. Zhang, A. Wang, X. Yang, G. Chen, L. Wang, W. Luo, and K. Zhang, “Alleviating hallucinations in large language models through multi-model contrastive decoding and dynamic hallucination detection,” in *Proc. NeurIPS*, 2025.
 - [84] K. Tian, E. Mitchell, H. Yao, C. D. Manning, and C. Finn, “Fine-tuning language models for factuality,” in *Proc. ICLR*, 2024.
 - [85] H. Zhang, S. Diao, Y. Lin, Y. R. Fung, Q. Lian, X. Wang, Y. Chen, H. Ji, and T. Zhang, “R-tuning: Instructing large language models to say “i don’t know”,” in *Proc. NAACL*. Association for Computational Linguistics, 2024, pp. 7113–7139.
 - [86] J. Wu *et al.*, “Mitigating LLM hallucination via behaviorally calibrated reinforcement learning,” *arXiv preprint arXiv:2512.19920*, 2025.
 - [87] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, “CRITIC: Large language models can self-correct with tool-interactive critiquing,” in *Proc. ICLR*, 2024.
 - [88] A. Zweiger, J. Pari, H. Guo, E. Akyürek, Y. Kim, and P. Agrawal, “Self-adapting language models,” *arXiv preprint arXiv:2506.10943*, 2025.
 - [89] D. Surís, S. Menon, and C. Vondrick, “ViperGPT: Visual inference via Python execution for reasoning,” in *Proc. ICCV*, 2023.
 - [90] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, “RAPTOR: Recursive abstractive processing for tree-organized retrieval,” in *Proc. ICLR*, 2024.
 - [91] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Proc. NeurIPS*, 2023.
 - [92] C. Li, C. Zhang, H. Zhou, N. Collier, A. Korhonen, and I. Vulić, “TopViewRS: Vision–language models as top-view spatial reasoners,” in *Proc. EMNLP*, 2024.
 - [93] H. Shao, S. Qian, H. Xiao, G. Song, Z. Zong, L. Wang, Y. Liu, and H. Li, “Visual CoT: Advancing multi-modal language models with a comprehensive dataset and benchmark for chain-of-thought reasoning,” in *Proc. NeurIPS*, 2024.
 - [94] B. J. Gutiérrez, Y. Shu, W. Qi, S. Zhou, and Y. Su, “From rag to memory: Non-parametric continual learning for large language models,” in *Proc. ICML*, 2025.
 - [95] K. T.-i. Ong, N. Kim, M. Gwak, H. Chae, T. Kwon, Y. Jo, S.-w. Hwang, D. Lee, and J. Yeo, “Towards lifelong dialogue agents via timeline-based memory management,” in *Proc. NAACL*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025, pp. 8631–8661.
 - [96] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language agent tree search unifies reasoning, acting, and planning in language models,” in *Proc. ICML*, 2024.
 - [97] C. Li, J. Liang, A. Zeng, X. Chen, K. Hausman, D. Sadigh, S. Levine, L. Fei-Fei, F. Xia, and B. Ichter, “Chain of code: Reasoning with a language model-augmented code emulator,” *arXiv preprint arXiv:2312.04474*, 2024.
 - [98] Y. Du, F. Wei, and H. Zhang, “AnyTool: Self-reflective, hierarchical agents for large-scale API calls,” in *Proc. ICML*, 2024.

- [99] Y. Li, F. Wei, J. Zhao, C. Zhang, and H. Zhang., “RAIN: Your language models can align themselves without finetuning,” in *Proc. ICLR*, 2024.
- [100] Z. Qi, M. Ma, J. Xu, L. L. Zhang, F. Yang, and M. Yang, “Mutual reasoning makes smaller LLMs stronger problem-solvers,” in *Proc. ICLR*, 2025.
- [101] X. Guan, L. Zhang, Y. Liu, N. Shang, Y. Sun, Y. Zhu, F. Yang, and M. Yang, “rStar-Math: Small LLMs can master math reasoning with self-evolved deep thinking,” in *Proc. ICML*, 2025.
- [102] C. Snell, J. Lee, K. Xu, and A. Kumar, “Scaling LLM test-time compute optimally can be more effective than scaling model parameters,” in *Proc. ICLR*, 2025.
- [103] W. Yang, S. Ma, Y. Lin, and F. Wei, “Towards thinking-optimal scaling of test-time compute for LLM reasoning,” *arXiv preprint arXiv:2502.18080*, 2025.
- [104] A. Setlur, C. Nagpal, A. Fisch, X. Geng, J. Eisenstein, R. Agarwal, A. Agarwal, J. Berant, and A. Kumar, “Rewarding progress: Scaling automated process verifiers for LLM reasoning,” in *Proc. ICLR*, 2025.
- [105] S. Farquhar, J. Kossen, L. Kuhn, and Y. Gal, “Detecting hallucinations in large language models using semantic entropy,” *Nature*, vol. 630, no. 8017, pp. 625–630, Jun. 2024.
- [106] L. Cao, “Learn to refuse: Making large language models more controllable and reliable through knowledge scope limitation and refusal mechanism,” *arXiv preprint arXiv:2311.01041*, 2023.
- [107] J. Loula, B. LeBrun, L. Du, B. Lipkin, C. Pasti, G. Grand, T. Liu, Y. Emara, M. Freedman, J. Eisner, R. Cotterell, V. Mansinghka, A. K. Lew, T. Vieira, and T. J. O’Donnell, “Syntactic and semantic control of large language models via sequential Monte Carlo,” *arXiv preprint arXiv:2504.13139*, 2025.
- [108] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, “A survey on the memory mechanism of large language model based agents,” *ACM Transactions on Information Systems*, 2024.
- [109] Y. Wu, S. Liang, C. Zhang, Y. Wang, Y. Zhang, H. Guo, R. Tang, and Y. Liu, “From human memory to AI memory: A survey on memory mechanisms in the era of LLMs,” *arXiv preprint arXiv:2504.15965*, 2025.
- [110] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023, pp. 1–22.
- [111] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “Memgpt: Towards llms as operating systems,” *arXiv preprint arXiv:2310.08560*, 2024.
- [112] J. Lu, S. An, M. Lin, G. Pergola, Y. He, D. Yin, X. Sun, and Y. Wu, “MemoChat: Tuning LLMs to use memos for consistent long-range open-domain conversation,” *arXiv preprint arXiv:2308.08239*, 2023.
- [113] Q. Wang *et al.*, “Recursively summarizing enables long-term dialogue memory in large language models,” *Neurocomputing*, 2025.
- [114] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Memo: Building production-ready AI agents with scalable long-term memory,” *arXiv preprint arXiv:2504.19413*, 2025.
- [115] Z. Pan, Q. Wu, H. Jiang, X. Luo, H. Cheng, D. Li, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and J. Gao, “SeCom: On memory construction and retrieval for personalized conversational agents,” in *Proc. ICLR*, 2025.

- [116] D. Patel *et al.*, “ENGRAM: Effective, lightweight memory orchestration for conversational agents,” *arXiv preprint arXiv:2511.12960*, 2025.
- [117] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *Transactions on Machine Learning Research*, 2024.
- [118] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, “Zep: A temporal knowledge graph architecture for agent memory,” *arXiv preprint arXiv:2501.13956*, 2025.
- [119] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, “HippoRAG: Neurobiologically inspired long-term memory for large language models,” in *Proc. NeurIPS*, 2024.
- [120] Z. Wang, Z. Li, Z. Jiang, D. Tu, and W. Shi, “Crafting personalized agents through retrieval-augmented generation on editable memory graphs,” in *Proc. EMNLP*, 2024.
- [121] A. Rezazadeh, Z. Li, W. Wei, and Y. Bao, “From isolated conversations to hierarchical schemas: Dynamic tree memory representation for LLMs,” in *Proc. ICLR*, 2025.
- [122] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” in *Proc. NeurIPS*, 2025.
- [123] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing large language models with long-term memory,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [124] N. Chen *et al.*, “Compress to impress: Unleashing the potential of compressive memory in real-world long-term conversations,” in *Proc. COLING*, 2025.
- [125] Z. Li, S. Song, H. Wang, S. Niu, D. Chen, J. Yang, C. Xi, H. Lai, J. Zhao, Y. Wang, J. Ren, Z. Lin, J. Huo, T. Chen, K. Chen, K. Li, Z. Yin, Q. Yu, B. Tang, H. Yang, Z.-Q. J. Xu, and F. Xiong, “MemOS: An operating system for memory-augmented generation in large language models,” *arXiv preprint*, 2025.
- [126] S. Zhang, J. Wang, R. Zhou, J. Liao, Y. Feng, Z. Li, Y. Zheng, W. Zhang, Y. Wen, Z. Li, F. Xiong, Y. Qi, B. Tang, and M. Wen, “MemRL: Self-evolving agents via runtime reinforcement learning on episodic memory,” *arXiv preprint arXiv:2601.03192*, 2026.
- [127] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, “Agent s: An open agentic framework that uses computers like a human,” *arXiv preprint arXiv:2410.08164*, 2025.
- [128] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research*, 2024.
- [129] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su, “Travelplanner: A benchmark for real-world planning with language agents,” in *Proc. ICML*, 2024.
- [130] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. Le, and E. Chi, “Least-to-most prompting enables complex reasoning in large language models,” in *Proc. ICLR*, 2023.
- [131] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proc. ACL*, 2023, pp. 2609–2634.
- [132] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face,” in *Proc. NeurIPS*, 2023.
- [133] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, “An llm

- compiler for parallel function calling,” in *Proc. ICML*, 2024, pp. 24 370–24 391.
- [134] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “Llm-planner: Few-shot grounded planning for embodied agents with large language models,” in *Proc. ICCV*, 2023, pp. 2986–2997.
 - [135] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “Llm+p: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
 - [136] Y. Hao, Y. Zhang, and C. Fan, “Planning anything with rigor: General-purpose zero-shot planning with llm-based formalized programming,” in *Proc. ICLR*, 2025.
 - [137] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, “Skeleton-of-thought: Prompting llms for efficient parallel generation,” in *Proc. ICLR*, 2024.
 - [138] L. E. Erdogan *et al.*, “Plan-and-act: Improving planning of agents for long-horizon tasks,” *arXiv preprint arXiv:2503.09572*, 2025.
 - [139] A. Prasad, A. Koller, M. Hartmann, P. Clark, A. Sabharwal, M. Bansal, and T. B. Khot, “Adapt: As-needed decomposition and planning with language models,” in *Proc. ACL*, 2024.
 - [140] R. Yang, J. Chen, Y. Zhang, S. Yuan, A. Chen, K. Richardson, Y. Xiao, and D. Yang, “Selfgoal: Your language agents already know how to achieve high-level goals,” in *Proc. NAACL*, 2025.
 - [141] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better llm agents,” in *Proc. ICML*, 2024.
 - [142] Z. Wang, S. Cai, G. Chen, A. Liu, X. Ma, and Y. Liang, “Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents,” in *Proc. NeurIPS*, 2023.
 - [143] Y. Zhu *et al.*, “Knowagent: Knowledge-augmented planning for llm-based agents,” *arXiv preprint arXiv:2403.03101*, 2024.
 - [144] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. NeurIPS*, 2023.
 - [145] Z. Zhao, W. S. Lee, and D. Hsu, “Large language models as commonsense knowledge for large-scale task planning,” in *Proc. NeurIPS*, 2023.
 - [146] X. Wang, C. Li, Z. Wang, F. Bai, H. Luo, J. Zhang, N. Jojic, E. P. Xing, and Z. Hu, “Promptagent: Strategic planning with language models enables expert-level prompt optimization,” in *Proc. ICLR*, 2024.
 - [147] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, “Reasoning with language model is planning with world model,” *arXiv preprint arXiv:2305.14992*, 2023.
 - [148] D. Zhang, S. Zhoubian, Z. Hu, Y. Yue, Y. Dong, and J. Tang, “Rest-mcts*: Llm self-training via process reward guided tree search,” in *Proc. NeurIPS*, 2024.
 - [149] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, “Aflow: Automating agentic workflow generation,” in *Proc. ICLR*, 2025.
 - [150] Y. Zhuang, X. Chen, T. Yu, S. Mitra, V. Bursztytn, R. A. Rossi, S. Sarkhel, and C. Zhang, “Toolchain*: Efficient action space navigation in large language models with a* search,” in *Proc. ICLR*, 2024.
 - [151] J. Ye, Z. Wu, J. Gao, Z. Wu, X. Jiang, Z. Li, and L. Kong, “Implicit search via discrete diffusion: A study on chess,” in *Proc. ICLR*, 2025.

- [152] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of llm agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024.
- [153] M. Lee *et al.*, "Planrag: A plan-then-retrieval augmented generation for generative large language models as decision makers," *arXiv preprint arXiv:2406.12430*, 2024.
- [154] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
- [155] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," in *Proc. ICLR*, 2024.
- [156] Z. Liu, T. Hoang, J. Zhang, M. Zhu, T. Lan, S. Kokane, J. Tan, W. Yao, Z. Liu, Y. Feng *et al.*, "Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets," in *Proc. NeurIPS*, 2024.
- [157] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, and Y. Shan, "Gpt4tools: Teaching large language model to use tools via self-instruction," in *Proc. NeurIPS*, 2023.
- [158] W. Liu *et al.*, "Toolace: Winning the points of llm function calling," in *Proc. ICLR*, 2025.
- [159] Q. Wu, W. Liu, J. Luan, and B. Wang, "Toolplanner: A tool augmented llm for multi granularity instructions with path planning and feedback," in *Proc. EMNLP*, 2024, pp. 18 315–18 339.
- [160] Z. Qi *et al.*, "Webrl: Training llm web agents via self-evolving online curriculum reinforcement learning," in *Proc. ICLR*, 2025.
- [161] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," in *Proc. NeurIPS*, 2024.
- [162] S. Hao, T. Liu, Z. Wang, and Z. Hu, "Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings," in *Proc. NeurIPS*, 2023.
- [163] S. Yuan, K. Song, J. Chen, X. Tan, Y. Shen, K. Ren, D. Li, and D. Yang, "Easytool: Enhancing llm-based agents with concise tool instruction," in *Proc. NAACL*, 2025, pp. 951–972.
- [164] OpenAI, "Function calling and other api updates," <https://openai.com/index/function-calling-and-other-api-updates/>, 2023.
- [165] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models," in *Proc. ICML*, 2025, pp. 48 371–48 392.
- [166] S. Fan, X. Cong, Y. Fu, Z. Zhang, S. Zhang, Y. Liu, Y. Wu, Y. Lin, Z. Liu, and M. Sun, "Workflowllm: Enhancing workflow orchestration capability of large language models," in *Proc. ICLR*, 2025.
- [167] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Gptswarm: Language agents as optimizable graphs," in *Proc. ICML*, 2024.
- [168] J. He, J. Neville, M. Wan, L. Yang, H. Liu, X. Xu, X. Song, J. Z. Pan, and P. Zhou, "Gentool: Enhancing tool generalization in language models through zero-to-one and weak-to-strong simulation," in *Findings of ACL*, 2025.
- [169] D. Mekala, J. E. Weston, J. Lanchantin, R. Raileanu, M. Lomeli, J. Shang, and J. Dwivedi-Yu, "Toolverifier: Generalization to new tools via self-verification," in *Findings of EMNLP*, 2024.
- [170] Y. Chen, J. Yoon, D. S. Sachan, Q. Wang, V. Cohen-Addad, M. Bateni, C.-Y. Lee, and T. Pfister, "Re-invoke: Tool invocation rewriting for zero-shot tool retrieval," in *Findings of EMNLP*, 2024, pp.

- 4705–4726.
- [171] Y. Huang *et al.*, “Metatool benchmark for large language models: Deciding whether to use tools and which to use,” *arXiv preprint arXiv:2310.03128*, 2024.
 - [172] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren, “Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models,” in *Findings of ACL*, 2025, pp. 24 497–24 524.
 - [173] B. Zheng, M. Y. Fatemi, X. Jin, Z. Z. Wang, A. Gandhi, Y. Song, Y. Gu, J. Srinivasa, G. Liu, G. Neubig, and Y. Su, “SkillWeaver: Web agents can self-improve by discovering and honing skills,” *arXiv preprint arXiv:2504.07079*, 2025.
 - [174] J. Wang *et al.*, “Reinforcement learning for self-improving agent with skill library,” *arXiv preprint arXiv:2512.17102*, 2025.
 - [175] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, “AgentBench: Evaluating LLMs as agents,” in *Proc. ICLR*, 2024.
 - [176] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, “GAIA: A benchmark for general AI assistants,” in *Proc. ICLR*, 2024.
 - [177] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji, “MINT: Evaluating LLMs in multi-turn interaction with tools and language feedback,” in *Proc. ICLR*, 2024.
 - [178] C. Ma *et al.*, “AgentBoard: An analytical evaluation board of multi-turn LLM agents,” in *Proc. NeurIPS*, 2024.
 - [179] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. Maben, R. Mehta, W. Chi, L. Jang, Y. Xie, S. Zhou, and G. Neubig, “TheAgentCompany: Benchmarking LLM agents on consequential real world tasks,” *arXiv preprint arXiv:2412.14161*, 2024.
 - [180] J. Y. Koh, R. Lo, L. Jang, V. Duvvur, M. Lim, P.-Y. Huang, G. Neubig, S. Zhou, R. Salakhutdinov, and D. Fried, “Visualwebarena: Evaluating multimodal agents on realistic visual web tasks,” in *Proc. ACL*. Association for Computational Linguistics, 2024, pp. 881–905.
 - [181] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2web: Towards a generalist agent for the web,” in *Proc. NeurIPS*, 2023.
 - [182] J. Wei *et al.*, “BrowseComp: A simple yet challenging benchmark for browsing agents,” *arXiv preprint arXiv:2504.12516*, 2025.
 - [183] T. Xie *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *Proc. NeurIPS*, 2024.
 - [184] C. Rawles, S. Clinckemaigne, Y. Chang, J. Waltz, G. Lau, M. Fair, A. Li, W. Bishop, W. Li, F. Campbell-Ajala, D. Toyama, R. Berry, D. Tyamagundlu, T. Lillicrap, and O. Riva, “Androidworld: A dynamic benchmarking environment for autonomous agents,” in *Proc. ICLR*, 2025.
 - [185] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” *arXiv preprint arXiv:2307.16789*, 2023.
 - [186] H. Trivedi *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proc. ACL*, 2024.

- [187] S. Yao *et al.*, “ τ -bench: A benchmark for tool-agent-user interaction in realistic domains,” *arXiv preprint arXiv:2406.12045*, 2024.
- [188] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, “ALFWorld: Aligning text and embodied environments for interactive learning,” in *Proc. ICLR*, 2021.
- [189] R. Wang *et al.*, “ScienceWorld: Is your agent smarter than a 5th grader?” in *Proc. EMNLP*, 2022.
- [190] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of LLM agents,” in *Proc. ACL*, 2024.
- [191] D. Wu *et al.*, “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” in *Proc. ICLR*, 2025.
- [192] Y. Hu, Y. Wang, and J. McAuley, “Evaluating memory in llm agents via incremental multi-turn interactions,” in *Proc. ICLR*, 2026.
- [193] J. S. Chan *et al.*, “MLE-bench: Evaluating machine learning agents on machine learning engineering,” *arXiv preprint arXiv:2410.07095*, 2024, openAI.
- [194] Z. S. Siegel *et al.*, “CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark,” *arXiv preprint arXiv:2409.11363*, 2024.
- [195] Z. Chen *et al.*, “ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery,” *arXiv preprint arXiv:2410.05080*, 2025.
- [196] G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, J. Heidecke, A. Glaese, and T. Patwardhan, “PaperBench: Evaluating AI’s ability to replicate AI research,” *arXiv preprint arXiv:2504.01848*, 2025, openAI.
- [197] L. Boisvert, M. Thakkar, M. Gasse, M. Caccia, T. L. S. D. Chezelles, Q. Cappart, N. Chapados, A. Lacoste, and A. Drouin, “WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks,” *arXiv preprint arXiv:2407.05291*, 2024.
- [198] K.-H. Huang *et al.*, “CRMArena: Understanding the capacity of LLM agents to perform professional CRM tasks in realistic environments,” *arXiv preprint arXiv:2411.02305*, 2024.
- [199] A. K. Zhang *et al.*, “Cybench: A framework for evaluating cybersecurity capabilities and risks of language models,” *arXiv preprint arXiv:2408.08926*, 2025.
- [200] M. Andriushchenko *et al.*, “AgentHarm: A benchmark for measuring harmfulness of LLM agents,” *arXiv preprint arXiv:2410.09024*, 2025.
- [201] B. Liu, X. Li, J. Zhang, J. Wang, T. He, S. Hong, H. Liu, S. Zhang, K. Song, K. Zhu *et al.*, “Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems,” *arXiv preprint arXiv:2504.01990*, 2025.
- [202] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 1, p. 9, 2024.
- [203] W. Long, W. Wen, P. Zhai, and L. Zhang, “Role play: Learning adaptive role-specific strategies in multi-agent interactions,” *Knowledge-Based Systems*, vol. 324, p. 113811, 2025.
- [204] Y. Dong, X. Zhu, Z. Pan, L. Zhu, and Y. Yang, “Villageragent: A graph-based multi-agent framework for coordinating complex task dependencies in minecraft,” in *Findings of ACL*, 2024, pp. 16 290–16 314.
- [205] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu, “Chateval: Towards better llm-based evaluators through multi-agent debate,” in *Proc. ICLR*, 2024.

- [206] K. He, D. Fu, L. Yang, W. Xu, and C. Wang, "Selfcorrect-agent: Toward robust and generalizable llm-based agents," *Neurocomputing*, p. 132546, 2025.
- [207] Y. Zhong, J. G. Kuba, X. Feng, S. Hu, J. Ji, and Y. Yang, "Heterogeneous-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 25, no. 32, pp. 1–67, 2024.
- [208] R. Ye, X. Liu, Q. Wu, X. Pang, Z. Yin, L. Bai, and S. Chen, "X-mas: Towards building multi-agent systems with heterogeneous llms," *arXiv preprint arXiv:2505.16997*, 2025.
- [209] X. Guo, D. Shi, J. Yu, and W. Fan, "Heterogeneous multi-agent reinforcement learning for zero-shot scalable collaboration," *Neurocomputing*, vol. 648, p. 130716, 2025.
- [210] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for" mind" exploration of large language model society," in *Proc. NeurIPS*, vol. 36, 2023, pp. 51 991–52 008.
- [211] T. Yao, Z. Li, and Z. Shen, "Hieramas: Optimizing intra-node llm mixtures and inter-node topology for multi-agent systems," *arXiv preprint arXiv:2602.20229*, 2026.
- [212] Y. Zhou and Y. Chen, "Adaptive heterogeneous multi-agent debate for enhanced educational and factual reasoning in large language models," *Journal of King Saud University - Computer and Information Sciences*, vol. 37, no. 10, p. 330, 2025.
- [213] M. Vaccaro Jr, M. Friday, and A. Zaghi, "Multi-agentic llms for personalizing stem texts," *Applied Sciences*, vol. 15, no. 13, p. 7579, 2025.
- [214] Y. Kim, C. Park, H. Jeong, Y. S. Chan, X. Xu, D. McDuff, H. Lee, M. Ghassemi, C. Breazeal, and H. W. Park, "Mdagents: An adaptive collaboration of llms for medical decision-making," in *Proc. NeurIPS*, vol. 37, 2024, pp. 79 410–79 452.
- [215] D. S. Nguyen, M. T. Nguyen, P. T. Nguyen, J. Di Rocco, and D. Di Ruscio, "Automated summarization of software documents: an llm-based multi-agent approach," *Automated Software Engineering*, vol. 33, no. 2, p. 43, 2026.
- [216] Z. Li, H. Yang, Y. Liu, Y. Xiang, H. Gao, J. Liu, and J. Liu, "Optdispro: Llm-based multi-agent framework for flexibly adapting heuristic optimal disflow," *IEEE Transactions on Smart Grid*, 2025.
- [217] R. Barbosa, R. Santos, and P. Novais, "Collaborative problem-solving with llm: a multi-agent system approach to solve complex tasks using autogen," in *Proc. PAAMS*. Springer, 2024, pp. 203–214.
- [218] A. Campbell and A. S. Wu, "Multi-agent role allocation: issues, approaches, and multiple perspectives," *Autonomous Agents and Multi-Agent Systems*, vol. 22, no. 2, pp. 317–355, 2011.
- [219] A. Amayuelas, J. Yang, S. Agashe, A. Nagarajan, A. Antoniadis, X. E. Wang, and W. Wang, "Self-resource allocation in multi-agent llm systems," *arXiv preprint arXiv:2504.02051*, 2025.
- [220] U. M. Borghoff, P. Bottoni, and R. Pareschi, "An organizational theory for multi-agent interactions integrating human agents, llms, and specialized ai," *Discover Computing*, vol. 28, no. 1, p. 138, 2025.
- [221] S. Youwai, D. Phim, V. G. Murcia, and R. C. Onas, "Large language model-based multi-agent systems for automated foundation design: router-driven task classification and expert selection framework," *AI in Civil Engineering*, vol. 5, no. 1, p. 5, 2026.
- [222] Y. Yue, G. Zhang, B. Liu, G. Wan, K. Wang, D. Cheng, and Y. Qi, "Masrouter: Learning to route llms for multi-agent systems," in *Proc. ACL*, 2025, pp. 15 549–15 572.

- [223] E. Tarasova, V. Erofeeva, O. Granichin, and K. Chernikov, “Decentralized adaptive task allocation for dynamic multi-agent systems,” *Scientific Reports*, vol. 15, no. 1, p. 39226, 2025.
- [224] H. Goel, M. Omama, B. Chalaki, V. Tadiparthi, E. M. Pari, and S. P. Chinchali, “R3dm: Enabling role discovery and diversity through dynamics models in multi-agent reinforcement learning,” in *Proc. ICML*. PMLR, 2025, pp. 19 600–19 620.
- [225] T. Fei, C. Chen, Y. Pan, M. Zheng, and M. Song, “Codedelegator: Mitigating context pollution via role separation in code-as-action agents,” *arXiv preprint arXiv:2601.14914*, 2026.
- [226] K. Zhu, Z. Tian, S. Wang, M. Leng, and X. Mao, “Atomizer: An llm-based collaborative multi-agent framework for intent-driven commit untangling,” *arXiv preprint arXiv:2601.01233*, 2026.
- [227] Y. Chen, Y. Yuan, Z. Zhang, Y. Zheng, J. Liu, F. Ni, and J. Hao, “Sheetagent: A generalist agent for spreadsheet reasoning and manipulation via large language models,” in *Proc. ICML Workshop*, 2024.
- [228] D. Jin, Z. Jin, X. Chen, and C. Wang, “Mare: Multi-agents collaboration framework for requirements engineering,” *arXiv preprint arXiv:2405.03256*, 2024.
- [229] H. Yao, L. Da, V. Nandam, J. Turnau, Z. Liu, L. Pang, and H. Wei, “Comal: Collaborative multi-agent large language models for mixed-autonomy traffic,” in *Proc. SDM*. SIAM, 2025, pp. 409–418.
- [230] Z. Wang, Y. Zhu, H. Zhao, X. Zheng, D. Sui, T. Wang, W. Tang, Y. Wang, E. Harrison, C. Pan *et al.*, “Colacare: Enhancing electronic health record modeling through large language model-driven multi-agent collaboration,” in *Proc. WWW*, 2025, pp. 2250–2261.
- [231] H. Wei, J. Qiu, H. Yu, and W. Yuan, “Medco: Medical education copilots based on a multi-agent framework,” in *Proc. ECCV*. Springer, 2024, pp. 119–135.
- [232] Shi Jinxin, Zhao Jiabao, Wang Yilei, Wu Xingjiao, Li Jiawen, and He Liang, “Cgmi: Configurable general multi-agent interaction framework,” *arXiv preprint arXiv:2308.12503*, 2023.
- [233] Y. Xu, J. Hu, Z. Zhao, Z. Duan, X. Sun, and X. Yang, “Multiagentesc: A llm-based multi-agent collaboration framework for emotional support conversation,” in *Proc. EMNLP*, 2025, pp. 4665–4681.
- [234] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. Karlsson, J. Fu, and Y. Shi, “Autoagents: a framework for automatic agent generation,” in *Proc. IJCAI*, 2024, pp. 22–30.
- [235] D. Yang, J. Wei, M. Li, J. Liu, L. Liu, M. Hu, J. He, Y. Ju, W. Zhou, Y. Liu *et al.*, “Medaide: information fusion and anatomy of medical intents via llm-based agent collaboration,” *Information Fusion*, p. 103743, 2025.
- [236] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang, S. Wang, P. Li, Y.-Q. Zhang, W. Ma *et al.*, “Agent hospital: A simulacrum of hospital with evolvable medical agents,” *arXiv preprint arXiv:2405.02957*, 2024.
- [237] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian *et al.*, “Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors,” in *Proc. ICLR*, 2024.
- [238] R. Perera, A. Basnayake, and M. Wickramasinghe, “Auto-scaling llm-based multi-agent systems through dynamic integration of agents,” *Frontiers in Artificial Intelligence*, vol. 8, p. 1638227, 2025.
- [239] T. Wang, H. Dong, V. Lesser, and C. Zhang, “Roma: multi-agent reinforcement learning with

- emergent roles,” in *Proc. ICML*, 2020, pp. 9876–9886.
- [240] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Proc. NeurIPS*, 2017.
- [241] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, “Value-decomposition networks for cooperative multi-agent learning,” *arXiv preprint arXiv:1706.05296*, 2017.
- [242] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, “Monotonic value function factorisation for deep multi-agent reinforcement learning,” *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.
- [243] Z. Zhou, Z. Liu, J. Liu, Q. Shao, Y. Wang, K. Shao, D. Jin, and F. Xu, “Resmas: Resilience optimization in llm-based multi-agent systems,” *arXiv preprint arXiv:2601.04694*, 2026.
- [244] C. Tu, Z. Yu, J. Huang, F. Huang, Y. Wu, L. Han, L. Wang, and R. Huang, “Adaptive role learning with evolutionary multi-agent reinforcement learning for uav-vehicle collaboration in sparse mobile crowdsensing,” *IEEE Internet of Things Journal*, 2025.
- [245] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large language model based multi-agents: a survey of progress and challenges,” in *Proc. IJCAI*, 2024, pp. 8048–8057.
- [246] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” in *Proc. ICLR*, 2023.
- [247] Z. Wang, S. Moriyama, W.-Y. Wang, B. Gangopadhyay, and S. Takamatsu, “Talk structurally, act hierarchically: A collaborative framework for llm multi-agent systems,” *arXiv preprint arXiv:2502.11098*, 2025.
- [248] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, “Magis: Llm-based multi-agent framework for github issue resolution,” in *Proc. NeurIPS*, vol. 37, 2024, pp. 51 963–51 993.
- [249] T. Ma, Y. Qian, Z. Zhang, Z. Wang, X. Qian, F. Bai, Y. Ding, X. Luo, S. Zhang, K. Murugesan *et al.*, “Autodata: A multi-agent system for open web data collection,” in *Proc. NeurIPS*, 2025.
- [250] X. Mou, C. Qian, W. Liu, X. Huang, and Z. Wei, “EcoLANG: Efficient and effective agent communication language induction for social simulation,” in *Findings of EMNLP*, 2025, pp. 5287–5304.
- [251] Q. Wang, T. Wang, Z. Tang, Q. Li, N. Chen, J. Liang, and B. He, “Megaagent: A large-scale autonomous llm-based multi-agent system without predefined sops,” in *Findings of ACL*, 2025, pp. 4998–5036.
- [252] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, “Cut the crap: An economical communication pipeline for llm-based multi-agent systems,” in *Proc. ICLR*, 2025.
- [253] W. Fan, T. Tognoli, H. P. Zou, C. Miao, Y. Wang, and X. Zhang, “Todycomm: Task-oriented dynamic communication for multi-round llm-based multi-agent system,” *arXiv preprint arXiv:2602.03688*, 2026.
- [254] N. Nascimento, P. Alencar, and D. Cowan, “Self-adaptive large language model (llm)-based multiagent systems,” in *Proc. ACSOS-C. IEEE*, 2023, pp. 104–109.
- [255] J. Chen, S. Saha, and M. Bansal, “Reconcile: Round-table conference improves reasoning via

- consensus among diverse llms,” in *Proc. ACL*, 2024, pp. 7066–7085.
- [256] L. Yang, S. Li, and A. Deng, “Dynamic consensus communication mechanism for large language model-based multi-agent systems,” *Journal of Signal Processing Systems*, vol. 98, no. 1, p. 10, 2026.
 - [257] B. Pan, J. Lu, K. Wang, L. Zheng, Z. Wen, Y. Feng, M. Zhu, and W. Chen, “Agentcoord: Visually exploring coordination strategy for llm-based multi-agent collaboration,” *Computers & graphics*, p. 104338, 2025.
 - [258] C. Zhu, M. Dastani, and S. Wang, “A survey of multi-agent deep reinforcement learning with communication,” *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 1, p. 4, 2024.
 - [259] S. Shaw, E. Wenzel, A. Walker, and G. Sartoretti, “Formic: Foraging via multiagent rl with implicit communication,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4877–4884, 2022.
 - [260] A. Buscemi, D. Proverbio, A. Di Stefano, T. A. Han, G. Castignani, and P. Liò, “When numbers start talking: Implicit numerical coordination among llm-based agents,” *arXiv preprint arXiv:2601.03846*, 2026.
 - [261] Y. Wang, fangwei zhong, J. Xu, and Y. Wang, “Tom2c: Target-oriented multi-agent communication and cooperation with theory of mind,” in *Proc. ICLR*, 2022.
 - [262] D. Ge, H. Ji, and B. Huang, “Efficient training in multi-agent reinforcement learning: A communication-free framework for the box-pushing problem,” in *Proc. IDETC/CIE*, vol. 89213. American Society of Mechanical Engineers, 2025, p. V02BT02A039.
 - [263] J. Zou, X. Yang, R. Qiu, G. Li, K. Tieu, P. Lu, K. Shen, H. Tong, Y. Choi, J. He *et al.*, “Latent collaboration in multi-agent systems,” *arXiv preprint arXiv:2511.20639*, 2025.
 - [264] C. Hunt, *TCP/IP network administration*. " O'Reilly Media, Inc.", 2002, vol. 2.
 - [265] L. DeNardis, *The global war for internet governance*. Yale University Press, 2014.
 - [266] D. Gourley and B. Totty, *HTTP: the definitive guide*. " O'Reilly Media, Inc.", 2002.
 - [267] J. Chen, J. Sun, and G. Wang, “From unmanned systems to autonomous intelligent systems,” *Engineering*, vol. 12, pp. 16–19, 2022.
 - [268] I.-L. Stefan and I. Dumitrache, “Interconnected autonomous systems,” in *Proc. CSCS*. IEEE, 2025, pp. 149–154.
 - [269] Y. Tang and T. Runkler, “Llm-based agentic systems for software engineering: Challenges and opportunities,” *arXiv preprint arXiv:2601.09822*, 2026.
 - [270] Y. Yang, Q. Peng, J. Wang, Y. Wen, and W. Zhang, “Llm-based multi-agent systems: Techniques and business perspectives,” *arXiv preprint arXiv:2411.14033*, 2024.
 - [271] W. Chen, Z. You, R. Li, Y. Guan, C. Qian, C. Zhao, C. Yang, R. Xie, Z. Liu, and M. Sun, “Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence,” in *Proc. ICLR*, 2025.
 - [272] D. C. Kwon and X. Zhang, “Cp-agentnet: Autonomous and explainable communication protocol design using generative agents,” in *Proc. ICNP*. IEEE, 2025, pp. 1–12.
 - [273] A. Ehtesham, A. Singh, G. K. Gupta, and S. Kumar, “A survey of agent interoperability protocols: Model context protocol (mcp), agent communication protocol (acp), agent-to-agent protocol (a2a), and agent network protocol (anp),” *arXiv preprint arXiv:2505.02279*, 2025.
 - [274] S. Marro, E. La Malfa, J. Wright, G. Li, N. Shadbolt, M. Wooldridge, and P. Torr, “A scalable communication protocol for networks of large language models,” *arXiv preprint arXiv:2410.11905*,

- 2024.
- [275] Anthropic, “Model Context Protocol,” <https://www.anthropic.com/news/model-contextprotocol>, 2024.
 - [276] G. Chang, “ANP: Agent Network Protocol,” <https://www.agent-networkprotocol.com/>, 2024.
 - [277] Eclipse, “Language Model Operating System (LMOS),” <https://eclipse.dev/lmos/>, 2025.
 - [278] L. F. AI and I. Data, “ACP: Agent Communication Protocol,” <https://agentcommunicationprotocol.dev/introduction/welcome>, 2025.
 - [279] AGI, Inc., “Agent protocol,” <https://agentprotocol.ai/>, 2025, originally developed by the AI Engineer Foundation.
 - [280] J. Liu, Z. Kong, C. Yang, F. Yang, T. Li, P. Dong, J. Nanjey, H. Tang, G. Yuan, W. Niu *et al.*, “Rcr-router: Efficient role-aware context routing for multi-agent llm systems with structured memory,” *arXiv preprint arXiv:2508.04903*, 2025.
 - [281] J. Zhao, C. Chen, C. Qiao, L. Zheng, M. Han, Y. Liu, Y. Xu, X. Xu, and M. Zhang, “Tcandon-router: Adaptive reasoning router for multi-agent collaboration,” *arXiv preprint arXiv:2601.04544*, 2026.
 - [282] Y. Yang, H. Chai, S. Shao, Y. Song, S. Qi, R. Rui, and W. Zhang, “Agentnet: Decentralized evolutionary coordination for LLM-based multi-agent systems,” in *Proc. NeurIPS*, 2025.
 - [283] R. Zhang, X. Jiang, Z. Yang, Z. Zhang, J. Gao, Y. Xiao, H. Lai, X. Chu, J. Zhao, and Y. Wang, “Stackplanner: A centralized hierarchical multi-agent system with task-experience memory management,” *arXiv preprint arXiv:2601.05890*, 2026.
 - [284] A. Li, Y. Xie, S. Li, F. Tsung, B. Ding, and Y. Li, “Agent-oriented planning in multi-agent systems,” in *Proc. ICLR*, 2025.
 - [285] Y. Dang, C. Qian, X. Luo, J. Fan, Z. Xie, R. Shi, W. Chen, C. Yang, X. Che, Y. Tian *et al.*, “Multi-agent collaboration via evolving orchestration,” in *Proc. NeurIPS*, 2025.
 - [286] Á. Aso-Mollar and E. Onaindia, “An efficient approach for cooperative multi-agent learning problems,” in *Proc. ICTAI*. IEEE, 2024, pp. 694–701.
 - [287] B. Jin, T. Collins, D. Yu, M. Cemri, S. Zhang, M. Li, J. Tang, T. Qin, Z. Xu, J. Lu *et al.*, “Controlling performance and budget of a centralized multi-agent llm system with reinforcement learning,” *arXiv preprint arXiv:2511.02755*, 2025.
 - [288] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, “Centrally coordinated multi-agent reinforcement learning for power grid topology control,” in *Proc. e-Energy*, 2025, pp. 460–475.
 - [289] J. Wang, K. Chen, X. Song, K. Zhang, L. Ai, E. Yang, and B. Shi, “Symphony: A decentralized multi-agent framework for scalable collective intelligence,” *arXiv preprint arXiv:2508.20019*, 2025.
 - [290] S. Liu, T. Chen, R. Amiri, and C. Amato, “Learning decentralized llm collaboration with multi-agent actor critic,” *arXiv preprint arXiv:2601.21972*, 2026.
 - [291] H. Yang, J. Chen, M. Siew, T. L. Botran, and C. Joe-Wong, “LLM-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning,” in *Proc. AAAI Workshop on Multi-Agent AI in the Real World*, 2025.
 - [292] L. Ratnabala, R. Peter, A. Fedoseev, and D. Tsetserukou, “Hippo-mat: Decentralized task allocation using graphsage and multi-agent deep reinforcement learning,” *arXiv preprint arXiv:2503.07662*, 2025.
 - [293] C. H. Chen and M. F. Shiu, “Agentflow: Resilient adaptive cloud-edge framework for multi-agent

- coordination,” *arXiv preprint arXiv:2505.07603*, 2025.
- [294] W. Zhang, C. Cui, Y. Zhao, Y. Liu, and B. An, “Agentorchestra: A hierarchical multi-agent framework for general-purpose task solving,” *arXiv preprint arXiv:2506.12508*, 2025.
 - [295] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proc. ACL*, 2023, pp. 2609–2634.
 - [296] S. Yu, X. Xu, K. Deng, L. Li, and L. Tian, “Tree of agents: Improving long-context capabilities of large language models through multi-perspective reasoning,” in *Findings of EMNLP*. Association for Computational Linguistics, 2025, pp. 4574–4592.
 - [297] S. Yun, J. Peng, P. Li, W. Fan, J. Chen, J. Zou, G. Li, and T. Chen, “Graph-of-agents: A graph-based framework for multi-agent llm collaboration,” in *Proc. ICLR*, 2026.
 - [298] G. Yu, “Adaptorch: Task-adaptive multi-agent orchestration in the era of llm performance convergence,” *arXiv preprint arXiv:2602.16873*, 2026.
 - [299] W. Gu, J. Han, H. Wang, X. Li, and B. Cheng, “Explain-analyze-generate: A sequential multi-agent collaboration method for complex reasoning,” in *Proc. COLING*, 2025, pp. 7127–7140.
 - [300] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Ö. Arik, “Chain of agents: Large language models collaborating on long-context tasks,” in *Proc. NeurIPS*, vol. 37, 2024, pp. 132 208–132 237.
 - [301] S. Wang, Z. Tan, Z. Chen, S. Zhou, T. Chen, and J. Li, “Anymac: Cascading flexible multi-agent collaboration via next-agent prediction,” in *Proc. EMNLP*, 2025, pp. 11 566–11 578.
 - [302] Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang, “Macrec: A multi-agent collaboration framework for recommendation,” in *Proc. SIGIR*, 2024, pp. 2760–2764.
 - [303] L. Xu, Z. Hu, D. Zhou, H. Ren, Z. Dong, K. Keutzer, S. K. Ng, and J. Feng, “Magic: Investigation of large language model powered multi-agent in cognition, adaptability, rationality and collaboration,” in *Proc. EMNLP*, 2024, pp. 7315–7332.
 - [304] A. Barrak, “Traceability and accountability in role-specialized multi-agent llm pipelines,” in *Proc. ASE Workshops*. IEEE, 2025, pp. 315–322.
 - [305] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in *Proc. ICML*, 2024.
 - [306] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-agents enhances large language model capabilities,” in *Proc. ICLR*, 2025.
 - [307] E. Zhang, E. Zhu, G. Bansal, A. Fourney, H. Mozannar, and J. Gerrits, “Optimizing sequential multi-step tasks with parallel llm agents,” *arXiv preprint arXiv:2507.08944*, 2025.
 - [308] Y. Li, S. Liu, T. Zheng, L. Sun, and M. Song, “Parallelized planning-acting for efficient llm-based multi-agent systems in minecraft,” *arXiv preprint arXiv:2503.03505*, 2025.
 - [309] Z. Gu, X. Zhu, Y. Cai, H. Shen, X. Chen, Q. Wang, J. Li, X. Shi, H. Guo, W. Huang *et al.*, “Agentgroupchat-v2: Divide-and-conquer is what llm-based multi-agent system need,” *arXiv preprint arXiv:2506.15451*, 2025.
 - [310] J. Yu, Y. Ding, and H. Sato, “Dyntaskmas: A dynamic task graph-driven framework for asynchronous and parallel llm-based multi-agent systems,” in *Proc. ICAPS*, vol. 35, no. 1, 2025, pp. 288–296.
 - [311] S. Liu, Y. Lu, S. Chen, X. Hu, J. Zhao, T. Fu, and Y. Zhao, “Drugagent: Automating AI-aided drug

- discovery programming through LLM multi-agent collaboration,” in *Proc. AI4Research Workshop*, 2024.
- [312] C. Zhang, K. Yang, S. Hu, Z. Wang, G. Li, Y. Sun, C. Zhang, Z. Zhang, A. Liu, S.-C. Zhu, X. Chang, J. Zhang, F. Yin, Y. Liang, and Y. Yang, “Proagent: Building proactive cooperative agents with large language models,” in *Proc. AAAI*, vol. 38, 2024, pp. 17 591–17 599.
- [313] J. Zhang, X. Xu, N. Zhang, R. Liu, B. Hooi, and S. Deng, “Exploring collaboration mechanisms for LLM agents: A social psychology view,” in *Proc. ACL*, 2024, pp. 14 544–14 607.
- [314] Z. Li, W. Zhao, and J. Pajarinen, “Cooperative multi-agent planning with adaptive skill synthesis,” *arXiv preprint arXiv:2502.10148*, 2025.
- [315] X. Bo, Z. Zhang, Q. Dai, X. Feng, L. Wang, R. Li, X. Chen, and J.-R. Wen, “Reflective multi-agent collaboration based on large language models,” in *Proc. NeurIPS*, vol. 37, 2024, pp. 138 595–138 631.
- [316] X. Zhao, K. Wang, and W. Peng, “An electoral approach to diversify LLM-based multi-agent collective decision-making,” in *Proc. EMNLP*, 2024, pp. 2712–2727.
- [317] Y. Zhang, X. Yang, S. Feng, D. Wang, Y. Zhang, and K. Song, “Can llms beat humans in debating? a dynamic multi-agent framework for competitive debate,” *arXiv preprint arXiv:2408.04472*, 2024.
- [318] Y. Fu, H. Peng, T. Khot, and M. Lapata, “Improving language model negotiation with self-play and in-context learning from ai feedback,” *arXiv preprint arXiv:2305.10142*, 2023.
- [319] S. Mao, Y. Cai, Y. Xia, W. Wu, X. Wang, F. Wang, Q. Guan, T. Ge, and F. Wei, “ALYMPICS: LLM agents meet game theory,” in *Proc. COLING*, 2025, pp. 2845–2866.
- [320] L. Cai, J. He, Y. Li, J. Liang, Y. Lin, Z. Quan, Y. Zeng, and J. Xu, “Rtbagent: A llm-based agent system for real-time bidding,” in *Proc. WWW*, 2025, p. 104–113.
- [321] T. Liang, Z. He, W. Jiao, X. Wang, Y. Wang, R. Wang, Y. Yang, S. Shi, and Z. Tu, “Encouraging divergent thinking in large language models through multi-agent debate,” in *Proc. EMNLP*, 2024, pp. 17 889–17 904.
- [322] R. Ye, Y. Zhang, Y. Zhang, H. Kuang, Z. Wei, and P. Sun, “Multi-agent kto: Reinforcing strategic interactions of large language model in language game,” *arXiv preprint arXiv:2501.14225*, 2025.
- [323] T. Zhu, B. Chen, J. Zhou, H. Zhou, S. Sankararaman, and X. Dai, “Align: Aligned delegation with performance guarantees for multi-agent llm reasoning,” *arXiv preprint arXiv:2602.00127*, 2026.
- [324] J. Lee, R. Chang, D. Kwon, H. Singh, and N. Verma, “Gemmas: Graph-based evaluation metrics for multi agent systems,” in *Proc. EMNLP*, 2025, pp. 1522–1532.
- [325] L. Gioacchini, G. Siracusano, D. Sanvito, K. Gashteovski, D. Friede, R. Bifulco, and C. Lawrence, “Agentquest: A modular benchmark framework to measure progress and improve llm agents,” in *Proc. NAACL*, 2024, pp. 185–193.
- [326] S. Agashe, Y. Fan, A. Reyna, and X. E. Wang, “LLM-coordination: Evaluating and analyzing multi-agent coordination abilities in large language models,” in *Findings of NAACL*, 2025, pp. 8053–8072.
- [327] J. Chen, X. Hu, S. Liu, S. Huang, W.-W. Tu, Z. He, and L. Wen, “LLMArena: Assessing capabilities of large language models in dynamic multi-agent environments,” in *Proc. ACL*, 2024, pp. 13 055–13 077.
- [328] W. Wang, D. Zhang, T. Feng, B. Wang, and J. Tang, “Battleagentbench: A benchmark for evaluating cooperation and competition capabilities of language models in multi-agent systems,” *arXiv*

- preprint arXiv:2408.15971*, 2024.
- [329] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, Z. Wang, Z. Wang, C. Qian, X. Tang, H. Ji, and J. You, “MultiAgentBench : Evaluating the collaboration and competition of LLM agents,” in *Proc. ACL*, 2025, pp. 8580–8622.
 - [330] S. Schmidgall, R. Ziaei, C. Harris, E. Reis, J. Jopling, and M. Moor, “Agentclinic: a multimodal agent benchmark to evaluate ai in simulated clinical environments,” *arXiv preprint arXiv:2405.07960*, 2025.
 - [331] Z. Fan, L. Wei, J. Tang, W. Chen, Wang Siyuan, Z. Wei, and F. Huang, “AI hospital: Benchmarking large language models in a multi-agent medical interaction simulator,” in *Proc. COLING*. Abu Dhabi, UAE: Association for Computational Linguistics, 2025, pp. 10 183–10 213.
 - [332] R. Shu, N. Das, M. Yuan, M. Sunkara, and Y. Zhang, “Towards effective genai multi-agent collaboration: design and evaluation for enterprise applications,” *arXiv preprint arXiv:2412.05449*, 2024.
 - [333] L. Geng and E. Y. Chang, “Realm-bench: A real-world planning benchmark for llms and multi-agent systems,” *arXiv preprint arXiv:2502.18836*, 2025.
 - [334] J. Zhang, Y. Fan, K. Cai, X. Sun, and K. Wang, “OSC: Cognitive orchestration through dynamic knowledge alignment in multi-agent LLM collaboration,” in *Findings of EMNLP*, 2025, pp. 6320–6337.
 - [335] A. A. Jafari, C. Ozcinar, and G. Anbarjafari, “A lightweight modular framework for constructing autonomous agents driven by large language models: Design, implementation, and applications in agentforge,” *arXiv preprint arXiv:2601.13383*, 2026.
 - [336] Y. Liu, J. Cai, Y. Li, Q. Meng, Z. Liu, X. Li, C. Qian, C. Shi, and C. Yang, “Masfactory: A graph-centric framework for orchestrating llm-based multi-agent systems with vibe graphing,” *arXiv preprint arXiv:2603.06007*, 2026.
 - [337] C. P. Bechlioulis and K. J. Kyriakopoulos, “Collaborative multi-robot transportation in obstacle-cluttered environments via implicit communication,” *Frontiers in Robotics and AI*, vol. 5, p. 90, 2018.
 - [338] H. Zhao, X. Zhou, B. Ivanovic, and N. Mehr, “Udon: Uncertainty-weighted distributed optimization for multi-robot neural implicit mapping under extreme communication constraints,” *arXiv preprint arXiv:2509.12702*, 2025.
 - [339] E. Yang and C. Mavrogiannis, “Implicit communication in human-robot collaborative transport,” in *Proc. HRI*. IEEE, 2025, pp. 23–33.
 - [340] Y. Kim, K. Gu, C. Park, C. Park, S. Schmidgall, A. A. Heydari, Y. Yan, Z. Zhang, Y. Zhuang, M. Malhotra *et al.*, “Towards a science of scaling agent systems,” *arXiv preprint arXiv:2512.08296*, 2025.
 - [341] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, “Diagnosing failure root causes in platform-orchestrated agentic systems: Dataset, taxonomy, and benchmark,” *arXiv preprint arXiv:2509.23735*, 2025.
 - [342] B. Lin, K. Yang, Y. Lai, Y. Zhang, C. Zhang, G. Zhang, X. Yu, M. Yu, X. Wang, and Y. Wang, “Agentask: Multi-agent systems need to ask,” *arXiv preprint arXiv:2510.07593*, 2025.
 - [343] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, “Which agent causes task failures and when? on automated failure attribution of LLM multi-agent

- systems,” in *Proc. ICML*, 2025, pp. 76 583—76 599.
- [344] M. Chen, J. Wang, F. Mu, Y. Wang, Z. Liu, H. Feng, and Q. Wang, “Seeing the whole elephant: A benchmark for failure attribution in llm-based multi-agent systems,” *arXiv preprint arXiv:2604.22708*, 2026.
 - [345] Y. In, M. Tanjim, J. Subramanian, S. Kim, U. Bhattacharya, W. Kim, S. Park, S. Sarkhel, and C. Park, “Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation,” *arXiv preprint arXiv:2603.25001*, 2026.
 - [346] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, “Agentracer: Who is inducing failure in the llm agentic systems?” *arXiv preprint arXiv:2509.03312*, 2025.
 - [347] Y. Wang, W. Wu, J. Wang, and Q. Wang, “From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems,” *arXiv preprint arXiv:2602.23701*, 2026.
 - [348] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, “Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis,” *arXiv preprint arXiv:2509.13782*, 2025.
 - [349] F. Kong, R. Zhang, H. Yin, G. Zhang, X. Zhang, Z. Chen, Z. Zhang, X. Zhang, S.-C. Zhu, and X. Feng, “Aegis: Automated error generation and attribution for multi-agent systems,” *arXiv preprint arXiv:2509.14295*, 2025.
 - [350] K. Zhu, Z. Liu, B. Li, M. Tian, Y. Yang, J. Zhang, P. Han, Q. Xie, F. Cui, W. Zhang *et al.*, “Where llm agents fail and how they can learn from failures,” *arXiv preprint arXiv:2509.25370*, 2025.
 - [351] R. Lu, Y. Li, and Y. Huo, “Exploring autonomous agents: A closer look at why they fail when completing tasks,” *arXiv preprint arXiv:2508.13143*, 2025.
 - [352] X. Ma, X. Xie, Y. Wang, J. Wang, B. Wu, M. Li, and Q. Wang, “Demystifying the lifecycle of failures in platform-orchestrated agentic workflows,” *arXiv preprint arXiv:2509.23735*, 2026.
 - [353] J. Bi, Y. Gao, T. Xu, F. Yang, and M. Yang, “Diagnosing with insights: Structured analysis of agent failures via behavioral abstractions,” *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026, under review.
 - [354] J. Wang, Y. Wang, M. Chen, X. Xie, C. Chen, F. Mu, Z. Liu, and Q. Wang, “A survey for llm agent trajectory analysis: From failure attribution to enhancement,” 2026.
 - [355] A. West, Y. Weng, M. Zhu, Z. Lin, Z. Ning, and Y. Zhang, “Abduct, act, predict: Scaffolding causal inference for automated failure attribution in multi-agent systems,” *arXiv preprint arXiv:2509.10401*, 2025.
 - [356] H. Zhang, Y. Shi, X. Gu, H. You, Z. Zhang, L. Gan, Y. Yuan, and J. Huang, “Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search,” *arXiv preprint arXiv:2510.10581*, 2025.
 - [357] Y. Yu, M. Li, S. Xu, J. Fu, X. Hou, F. Lai, and B. Wang, “Correct: Condensed error recognition via knowledge transfer in multi-agent systems,” *arXiv preprint arXiv:2509.24088*, 2025.
 - [358] X. Shen, Q. Zhang, S. Wang, Z. Tan, X. Zhao, L. Yao, V. Tadiparthi, H. N. Mahjoub, E. M. Pari, K. Lee *et al.*, “Metacognitive self-correction for multi-agent system via prototype-guided next-execution reconstruction,” *arXiv preprint arXiv:2510.14319*, 2025.
 - [359] L. Advani, “Trajectory guard—a lightweight, sequence-aware model for real-time anomaly detection in agentic ai,” *arXiv preprint arXiv:2601.00516*, 2026.
 - [360] X. Zhao, S. Liu, Y. Zhang, Q. Ma, G. Cheng, N. Wang, and C. Liu, “Promas: Proactive error forecast-

- ing for multi-agent systems using markov transition dynamics," *arXiv preprint arXiv:2603.20260*, 2026.
- [361] A. Banerjee, A. Nair, and T. Borogovac, "Where did it all go wrong? a hierarchical look into multi-agent error attribution," *arXiv preprint arXiv:2510.04886*, 2025.
 - [362] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, "Dover: Intervention-driven auto debugging for llm multi-agent systems," *arXiv preprint arXiv:2512.06749*, 2025.
 - [363] S. Barke, A. Goyal, A. Khare, A. Singh, S. Nath, and C. Bansal, "Agentrx: Diagnosing ai agent failures from execution trajectories," *arXiv preprint arXiv:2602.02475*, 2026.
 - [364] X. Wang, M. Yin, E. Koh, and M. D. Dogan, "Xagen: An explainability tool for identifying and correcting failures in multi-agent workflows," *arXiv preprint arXiv:2512.17896*, 2025.
 - [365] K. Sun, W. Li, B. Dong, Y. Lin, J. Zhang, and B. Shi, "Scope delineation before localization: A two-stage framework for enhancing failure attribution in multi-agent systems," in *Proc. AAAI*, vol. 40, no. 39, 2026, pp. 33 108–33 116.
 - [366] J. Chen, Y. Wang, J. Wang, X. Xie, J. Hu, Q. Wang, and F. Xu, "Understanding individual agent importance in multi-agent system via counterfactual reasoning," in *Proc. AAAI*, vol. 39, no. 15, 2025, pp. 15 785–15 794.
 - [367] W. Epperson, G. Bansal, V. C. Dibia, A. Fournay, J. Gerrits, E. Zhu, and S. Amershi, "Interactive debugging and steering of multi-agent ai systems," in *Proc. CHI*, 2025, pp. 1–15.
 - [368] G. Ma, J. Zhu, H. Guo, W. Shi, J. Shen, J. Liu, and Y. Liang, "Automatic failure attribution and critical step prediction method for multi-agent systems based on causal inference," *arXiv preprint arXiv:2509.08682*, 2025.
 - [369] S. Triantafyllou, A. Sukovic, D. Mandal, and G. Radanovic, "Agent-specific effects: A causal effect propagation analysis in multi-agent mdps," *arXiv preprint arXiv:2310.11334*, 2023.
 - [370] X. Zheng, Z. Chen, C. Lin, H. Wei, H. Chen, W. Cheng, and D. Luo, "Trajectory graph copilot: Pre-action error diagnosis in llm agents," *arXiv preprint arXiv:2505.08638*, 2025.
 - [371] R. Sheng, Y. Yang, C. Shi, Y. Lin, Z. Chen, H. Qu, and F. Cheng, "Dills: Interactive diagnosis of llm-based multi-agent systems via layered summary of agent behaviors," *arXiv preprint arXiv:2602.05446*, 2026.
 - [372] S. Triantafyllou, A. Sukovic, Y. Zolfimoselo, and G. Radanovic, "Counterfactual effect decomposition in multi-agent sequential decision making," *arXiv preprint arXiv:2410.12539*, 2024.
 - [373] C. Zhu, S. Hong, J. Wu, K. Chawla, Y. Tang, Y. Yin, N. Wolfe, E. Babinsky, and D. Liu, "Raffles: Reasoning-based attribution of faults for llm systems," in *Proc. UIST*, 2026, pp. 7659–7688.
 - [374] Z. G. Wang, "Agenttrace: Causal graph tracing for root cause analysis in deployed multi-agent systems," *arXiv preprint arXiv:2603.14688*, 2026.
 - [375] H. D. Le, X. Xia, and Z. Chen, "Multi-agent causal discovery using large language models," *arXiv preprint arXiv:2407.15073*, 2024.
 - [376] H. V. D. Parunak *et al.*, "' go to the ant': Engineering principles from natural multi-agent systems," *Ann. Oper. Res.*, vol. 75, pp. 69–101, 1997.
 - [377] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm intelligence: from natural to artificial systems*. Oxford university press, 1999, no. 1.
 - [378] G. D. M. Serugendo, M.-P. Irit, and A. Karageorgos, "Self-organisation and emergence in mas:

- An overview," *Informatica*, vol. 30, no. 1, 2006.
- [379] R. S. Galhardo, P. J. Hastings, and S. M. Rosenberg, "Mutation as a stress response and the regulation of evolvability," *Critical reviews in biochemistry and molecular biology*, vol. 42, no. 5, pp. 399–435, 2007.
 - [380] S. Goss, S. Aron, J.-L. Deneubourg, and J. M. Pasteels, "Self-organized shortcuts in the argentine ant," *Naturwissenschaften*, vol. 76, no. 12, pp. 579–581, 1989.
 - [381] H. Y. Leong, Y. Li, Y. Wu, W. Ouyang, W. Zhu, J. Gao, and W. Han, "Amas: Adaptively determining communication topology for llm-based multi-agent system," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2025, pp. 2061–2070.
 - [382] Z. Xiang, C. Yang, Z. Chen, Z. Wei, Y. Tang, Z. Teng, Z. Peng, Z. Li, C. Huang, Y. He *et al.*, "A systematic survey of self-evolving agents: From model-centric to environment-driven co-evolution," 2026.
 - [383] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 21 344–21 377.
 - [384] S. Lu, J. Shao, B. Luo, and T. Lin, "Morphagent: Empowering agents through self-evolving profiles and decentralized collaboration," *arXiv preprint arXiv:2410.15048*, 2024.
 - [385] W. Chen, J. Yuan, C. Qian, C. Yang, Z. Liu, and M. Sun, "Optima: Optimizing effectiveness and efficiency for llm-based multi-agent system," in *Findings of ACL*, 2025, pp. 11 534–11 557.
 - [386] X. Xue, Y. Zhou, G. Zhang, Z. Zhang, Y. Li, C. Zhang, Z. Yin, P. Torr, W. Ouyang, and L. Bai, "Comas: Co-evolving multi-agent systems via interaction rewards," *arXiv preprint arXiv:2510.08529*, 2025.
 - [387] S. Mangla, C. Hokamp, J. Boylan, D. G. Ghalandari, Y. Jauhari, L. Cassidy, and O. Duffy, "Negotiationgym: Self-optimizing agents in a multi-agent social simulation environment," *arXiv preprint arXiv:2510.04368*, 2025.
 - [388] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
 - [389] G. Chen, L. Fan, Z. Gong, N. Xie, Z. Li, Z. Liu, C. Li, Q. Qu, H. Alinejad-Rokny, S. Ni *et al.*, "Agentcourt: Simulating court with adversarial evolvable lawyer agents," in *Findings of ACL*, 2025, pp. 5850–5865.
 - [390] Z. Guan, X. Kong, F. Zhong, and Y. Wang, "Richelieu: Self-evolving llm-based agents for ai diplomacy," in *Proc. NeurIPS*, vol. 37, 2024, pp. 123 471–123 497.
 - [391] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, H. Wang, H. Wang *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, vol. 2, no. 1, p. 32, 2023.
 - [392] Y. Chen, Y. Wang, S. Zhu, H. Yu, T. Feng, M. Zhan, M. Patwary, and J. You, "Multi-agent evolve: Llm self-improve through co-evolution," *arXiv preprint arXiv:2510.23595*, 2025.
 - [393] Z. Pan, Y. Zhang, Z. Liu, Y. Y. Tang, Z. Zhang, H. Luo, Y. Han, J. Zhang, D. Wu, H.-Y. Chen *et al.*, "Advevo-marl: Shaping internalized safety through adversarial co-evolution in multi-agent reinforcement learning," *arXiv preprint arXiv:2510.01586*, 2025.
 - [394] R. Zhou, M. Song, X. Wu, S. Cheng, X. Yin, Y. Xie, Z. Hao, W. Hua, L. Pan, S. Poria *et al.*,

- “Epistemic context learning: Building trust the right way in llm-based multi-agent systems,” *arXiv preprint arXiv:2601.21742*, 2026.
- [395] I. Kar, S. Zonunpuia, and Z. Ralte, “Towards agi a pragmatic approach towards self evolving agent,” *arXiv preprint arXiv:2601.11658*, 2026.
- [396] X. Li, C. Bai, Z. Li, J. Zheng, T. Xiao, and J. Zhang, “Learn as individuals, evolve as a team: Multi-agent llms adaptation in embodied environments,” *arXiv preprint arXiv:2506.07232*, 2025.
- [397] G. Zhang, Y. Yue, X. Sun, G. Wan, M. Yu, J. Fang, K. Wang, T. Chen, and D. Cheng, “G-designer: Architecting multi-agent communication topologies via graph neural networks,” *arXiv preprint arXiv:2410.11782*, 2024.
- [398] H. Zhou, X. Wan, R. Sun, H. Palangi, S. Iqbal, I. Vulić, A. Korhonen, and S. Ö. Arık, “Multi-agent design: Optimizing agents with better prompts and topologies,” *arXiv preprint arXiv:2502.02533*, 2025.
- [399] N. Tastan, S. Horváth, and K. Nandakumar, “Stochastic self-organization in multi-agent systems,” *arXiv preprint arXiv:2510.00685*, 2025.
- [400] G. Chen, S. Dong, Y. Shu, G. Zhang, J. Sesay, B. F. Karlsson, J. Fu, and Y. Shi, “Autoagents: A framework for automatic agent generation,” *arXiv preprint arXiv:2309.17288*, 2023.
- [401] S. Yuan, K. Song, J. Chen, X. Tan, D. Li, and D. Yang, “Evoagent: Towards automatic multi-agent generation via evolutionary algorithms,” in *Proc. NAACL*, 2025, pp. 6192–6217.
- [402] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen, “Self-evolving multi-agent collaboration networks for software development,” *arXiv preprint arXiv:2410.16946*, 2024.
- [403] X. Ma, C. Lin, Y. Zhang, V. Tresp, and Y. Ma, “Agentic neural networks: Self-evolving multi-agent systems via textual backpropagation,” *arXiv preprint arXiv:2506.09046*, 2025.
- [404] R. Pryzant, D. Iter, J. Li, Y. Lee, C. Zhu, and M. Zeng, “Automatic prompt optimization with “gradient descent” and beam search,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 7957–7968.
- [405] G. Zhang, M. Fu, G. Wan, M. Yu, K. Wang, and S. Yan, “G-memory: Tracing hierarchical memory for multi-agent systems,” *arXiv preprint arXiv:2506.07398*, 2025.
- [406] J. Rosser and J. Foerster, “Agentbreeder: Mitigating the ai safety risks of multi-agent scaffolds via self-improvement,” *arXiv preprint arXiv:2502.00757*, 2025.
- [407] Z. Ke, A. Xu, Y. Ming, X.-P. Nguyen, R. Chin, C. Xiong, and S. Joty, “Mas-zero: Designing multi-agent systems with zero supervision,” *arXiv preprint arXiv:2505.14996*, 2025.
- [408] H. Gao, Y. Liu, Y. He, L. Dou, C. Du, Z. Deng, B. Hooi, M. Lin, and T. Pang, “Flowreasoner: Reinforcing query-level meta-agents,” *arXiv preprint arXiv:2504.15257*, 2025.
- [409] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, “Multi-agent architecture search via agentic supernet,” *arXiv preprint arXiv:2502.04180*, 2025.
- [410] R. Ye, S. Tang, R. Ge, Y. Du, Z. Yin, S. Chen, and J. Shao, “Mas-gpt: Training llms to build llm-based multi-agent systems,” *arXiv preprint arXiv:2503.03686*, 2025.
- [411] D. T. Campbell, “Blind variation and selective retentions in creative thought as in other knowledge processes.” *Psychological review*, vol. 67, no. 6, p. 380, 1960.
- [412] H. Zhang, W. Du, J. Shan, Q. Zhou, Y. Du, J. B. Tenenbaum, T. Shu, and C. Gan, “Building cooperative embodied agents modularly with large language models,” *arXiv preprint arXiv:2307.02485*,

- 2023.
- [413] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” *arXiv preprint arXiv:2310.06770*, 2023.
 - [414] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
 - [415] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
 - [416] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt, “Measuring mathematical problem solving with the math dataset,” *arXiv preprint arXiv:2103.03874*, 2021.
 - [417] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
 - [418] L. Li, B. Dong, R. Wang, X. Hu, W. Zuo, D. Lin, Y. Qiao, and J. Shao, “Salad-bench: A hierarchical and comprehensive safety benchmark for large language models,” in *Findings of ACL*, 2024, pp. 3923–3954.
 - [419] G. Booch, F. Fabiano, L. Horesh, K. Kate, J. Lenchner, N. Linck, A. Loreggia, K. Murgesan, N. Mattei, F. Rossi *et al.*, “Thinking fast and slow in ai,” in *Proc. AAAI*, vol. 35, no. 17, 2021, pp. 15 042–15 046.
 - [420] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” in *Proc. NeurIPS*, vol. 35, 2022, pp. 27 730–27 744.
 - [421] A. Chan, R. Salganik, A. Markelius, C. Pang, N. Rajkumar, D. Krasheninnikov, L. Langosco, Z. He, Y. Duan, M. Carroll *et al.*, “Harms from increasingly agentic algorithmic systems,” in *Proc. FAccT*, 2023, pp. 651–666.
 - [422] R. Wang, J. Lehman, J. Clune, and K. O. Stanley, “Paired open-ended trailblazer (poet): Endlessly generating increasingly complex and diverse learning environments and their solutions,” *arXiv preprint arXiv:1901.01753*, 2019.